



CS5003NI Data Structure and Specialist Programming

AY 2024-25

30 Credit Year Long Module

30% Individual Coursework 01

Student Name: Piyush Karn

Project Title: Bake Ease

London Met ID: 23048660

College ID: NP01AI4A230119

Assignment Due Date: Monday, January 20, 2025

Assignment Submission Date: Monday, January 20, 2025

Submitted to: Mr. Prithivi Maharjan

Word Count: 10545

GitHub Link	https://github.com/piiyuushh/Bake-Ease	
--------------------	---	--

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

23048660 Piyush karn.docx



Islington College, Nepal

Document Details

Submission ID

trn:oid::3618:79597804

Submission Date

Jan 19, 2025, 11:30 PM GMT+5:45

Download Date

Jan 20, 2025, 9:06 AM GMT+5:45

File Name

23048660 Piyush karn.docx

File Size

53.9 KB

64 Pages

8,407 Words

45,252 Characters

37% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **130 Not Cited or Quoted 35%**
Matches with neither in-text citation nor quotation marks
-  **12 Missing Quotations 3%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **1 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7%  Internet sources
- 2%  Publications
- 35%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 130 Not Cited or Quoted 35%**
Matches with neither in-text citation nor quotation marks
- 12 Missing Quotations 3%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 1 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7% Internet sources
- 2% Publications
- 35% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	islingtoncollege on 2025-01-10	21%
2	Internet	www.javatpoint.com	1%
3	Internet	ntnuopen.ntnu.no	1%
4	Submitted works	islingtoncollege on 2025-01-10	1%
5	Submitted works	Canterbury Institute of Management on 2024-04-20	<1%
6	Submitted works	islingtoncollege on 2025-01-10	<1%
7	Internet	origin.geeksforgeeks.org	<1%
8	Internet	www.geeksforgeeks.org	<1%
9	Submitted works	islingtoncollege on 2025-01-10	<1%
10	Submitted works	Hardin Simmons University on 2019-07-19	<1%

Abstract

This project is submitted as part of coursework for the CS5003NI Data Structures and Specialist Programming module, focuses on creating a bakery management system called Bake Ease. The report begins with an introduction that explains the current challenges faced by bakery owners and how this project offers a solution.

The report showcases the system's structure, emphasizing the graphical user interface (GUI) design and the tools and technologies used following MVC architecture. It explains the step-by-step development process, including features like product management, sorting, searching, and validation. Additionally, it highlights the use of algorithms such as merge sort, selection sort, insertion sort and binary search, which are integrated into the system. Finally, the report describes the challenges encountered during the project and how they were resolved. This document provides a complete overview of the work and effort put into developing this project.

Acknowledgement

I would like to express my sincere gratitude to Mr. Prithivi Maharjan, for his guidance and support throughout this project. Also I would like to thank my friends Arpan Nepal, Aashutosh Dhakal, Pranav Sharma. Their insights, references, help, expertise for a similar project, were very helpful in overcoming challenges during the development process. I am also thankful to my peers for their advice and encouragement, which helped me navigate obstacles, particularly in implementing complex features like sorting and searching algorithms.

Table of Contents

1. Introduction.....	1
1.1 Overview and purpose	1
1.2 Audience	1
1.3 Aims and objectives	1
1.4 Features.....	2
2. Design	2
2.1 Loading page	2
2.1.1 Loading page reference	2
2.1.2 Wireframe design of loading page	3
2.1.2 Actual design of loading page	4
2.2 Login page	5
2.2.1 Login page reference	5
2.2.2 Wireframe design of login page	6
2.2.3 Actual design of login page	6
2.3 Home page.....	7
2.3.1 Wireframe design of Home page	7
2.3.2 Actual design of Home page	8
2.4 Admin page.....	9
2.4.1 Reference of Admin page	9
2.4.2 Wireframe of Admin page	10
2.4.3 Actual design of Admin page.....	11
2.5 Product page	12
2.5.1 Reference of product page.....	12
2.5.2 Wireframe of product page	13
2.5.3 Actual design of product page	13
2.6 Color Pallet	14
3. Java Classes and Methods	16
3.1 Overall Class Diagram	16
3.2 Individual Class Diagram.....	16
3.2.1 Model Class.....	18
3.2.2 Algorithms Class	21
3.2.3 Views Class	26
4. Testing.....	37
4.1 Test Planning	37
4.2 Test Cases.....	38
4.2.1 Progress Bar.....	38
4.2.2 Login Validation	40
4.2.3 Product Information	43
4.2.4 Submit Data	47
4.2.5 Update Data.....	50
4.2.6 Clear Data.....	52
4.2.7 Delete Record	54

4.2.8	Sort by Quantity	56
4.2.9	Sort by Price	58
4.2.10	Sort by Name.....	60
4.2.11	Search Button.....	62
5.	Coursework Development	64
5.1	Canva	64
5.1.1	Usage.....	66
5.2	Apache Netbeans	68
5.3	Java Programming	71
5.3.1	Project Development	71
5.3.2	Implementation of MVC Architecture	72
5.4	Algorithms.....	73
5.4.1	Insertion Sort	73
5.4.2	Selection Sort.....	78
5.4.3	Merge Sort.....	82
5.4.4	Binary Search	86
5.5	Data Structure	89
5.5.1	Linked List.....	89
5.5.2	Array List	92
6.	Critical Analysis.....	97
6.1	UI/UX Design & Development.....	97
6.2	CRUD Operations	105
6.2.1	Adding data to admin dashboard	105
6.2.1.1	Solution to the challenge	106
6.2.1.2	Screenshots of adding data	108
6.2.2	Deleting data from admin dashboard.....	109
6.2.2.1	Solution to the problem	109
6.2.3	Updating data from admin dashboard.....	112
6.2.3.1	Solution to the problem	112
6.3	Validations.....	115
6.3.1	Login Page Validation	115
6.3.2	Empty Fields Validation	116
6.3.3	Incorrect Credentials Validation.....	117
6.3.4	Correct Credentials Validation	118
6.3.2	Admin Dashboard Validation.....	119
6.3.2.1	Admin Empty Fields Validation	119
6.3.2.2	Incorrect data type validation	120
6.3.2.3	Correct data type validation.....	121
6.3.2.4	Clear text fields validation	122
6.3.2.5	Delete data validation.....	123
6.3.2.6	Search data validation	124
6.4	Sorting.....	125
6.4.1	Merge Sort.....	126
6.4.2	Insertion Sort	128
6.4.3	Selection Sort.....	129
6.4.4	Searching Algorithm	129
7.	Conclusion	130

8. Reference	131
---------------------------	------------

Table of Figures

Figure 1: reference for loading page.....	3
Figure 2: wireframe of loading page	3
Figure 3: actual loading page.....	4
Figure 4: login page reference	5
Figure 5: wireframe of login page	6
Figure 6: actual login page.....	6
Figure 7: wireframe of home page	7
Figure 8: actual home page	8
Figure 9: reference of home page.....	9
Figure 10: wireframe of admin page design	10
Figure 11: actual home page	11
Figure 12: reference of product page	12
Figure 13: wireframe of product page.....	13
Figure 14: Actual product page	13
Figure 15: choosing color pallet.....	14
Figure 16: Actual color pallet.....	15
Figure 17: overall class diagram	16
Figure 18: login page class diagram	16
Figure 19: admin dashboard class diagram	16
Figure 20: home page class diagram	17
Figure 21: product page class diagram	17
Figure 22: admin model class diagram.....	17
Figure 23: binary search class diagram.....	17
Figure 24: insertion sort class diagram.....	17
Figure 25: merge sort class diagram	17
Figure 26: selection sort class diagram	18
Figure 27: get product id()	18
Figure 28: getProductname().....	18
Figure 29: getQuantity()	18
Figure 30: getProductPrice()	19
Figure 31: setProductId().....	19
Figure 32: setProductName().....	19
Figure 33: setProductQuantity()	19
Figure 34: setPrice().....	20
Figure 35: SearchByName()	21
Figure 36: sortByName()	21
Figure 37: mergeSort()	22
Figure 38: merge()	23
Figure 39: sortByQuantity()	24
Figure 40: shouldSwap()	24
Figure 41: sortByPrice()	25
Figure 42: findPriceIndex().....	25
Figure 43: Swap	26
Figure 44: loginButtonActionPerformed()	26
Figure 45: chkBoxShowPasswordActionPerformed()	27
Figure 46:orderPageButtonActionPerformed	27
Figure 47: adminPageButtonActionPerformed()	27
Figure 48: viewProductsButtonActionPerformed()	28
Figure 49: exitButtonActionPerformed()	28

Figure 50: addBakeryItems()	28
Figure 51: dislayPreLoadedItems()	29
Figure 52: checkDuplicateProduct()	30
Figure 53: btnSubmitActionPerformed()	30
Figure 54: btnDeleteActionPerformed()	31
Figure 55: btnReturnActionPerformed()	31
Figure 56: updateItems()	32
Figure 57: btnUpdateActionPerformed()	32
Figure 58: btnClearActionPerformed()	33
Figure 59: loadSortedProductData()	34
Figure 60: btnSortPriceActionPerformed()	34
Figure 61: btnSortQuantityActionPerformed()	35
Figure 62: btnSortNameActionPerformed()	35
Figure 63: btnSearchActionPerformed()	36
Figure 64: Progress Bar test	39
Figure 65: Empty fields validation	40
Figure 66: incorrect details validation	41
Figure 67: Providing correct credentials	42
Figure 68: after providing correct credentials	42
Figure 69: method to validate product information	44
Figure 70: product information empty fields validation	44
Figure 71: Integer manipulation validation	45
Figure 72: product added successfully	45
Figure 73: Empty fields validation for submit data	47
Figure 74: Integer manipulation validation for submit data	48
Figure 75: Product succesfully added message	48
Figure 76: Checking duplicate data	49
Figure 77: Same product validation	49
Figure 78: Targeted Product	50
Figure 79: Updated data of targeted product	51
Figure 80: Asking user option	52
Figure 81: After choosing NO	53
Figure 82: After choosing YES	53
Figure 83: Targeted element to be deleted	54
Figure 84: Not selecting table row while deleting data	55
Figure 85: After selecting table row of targeted element	55
Figure 86: before sorting by quantity	56
Figure 87: after sorting by quantity	57
Figure 88: implementing insertion sort in sort by quantity button	57
Figure 89: before sorting by price	58
Figure 90: after sorting by price	59
Figure 91: imeplementing selection sort to sort by price	59
Figure 92: before sorting by name	60
Figure 93: after sorting by name	61
Figure 94: implementing merge sort to sort by name	61
Figure 95: message showing product not found in GUI	62
Figure 96: message showing product not found in terminal	63
Figure 97: product found message in GUI	63
Figure 98: name of founded product in terminal	63
Figure 99: Canva as designing tool	64

Figure 100: designing home page wireframe	66
Figure 101: designing login page wireframe.....	66
Figure 102: designing admin page wireframe	67
Figure 103: designing loading page wireframe	67
Figure 104: Using apache netbeans.....	68
Figure 105: Integrated GUI elements in netbeans	69
Figure 106: Java support in Netbeans.....	70
Figure 107: open source link of Netbeans.....	70
Figure 108: Implementing MVC architecture	72
Figure 109: Proof of citation explanation	73
Figure 110: Using insertion sort to sort by quantity	75
Figure 111: Implementing insertion sort	76
Figure 112: Evidence of selection sort citation explanation	78
Figure 113: method to sort by price	79
Figure 114: Calculating price index.....	79
Figure 115: swapping	80
Figure 116: implementing Selection sort	80
Figure 117: Evidence of citation explanation of merge sort	82
Figure 118: validating empty fields	83
Figure 119: Splitting into two sub lists	83
Figure 120: Merging sub lists	84
Figure 121: Implementing merge sort.....	84
Figure 122: Evidence of citation explanation of binary search	86
Figure 123: initializing binary search	87
Figure 124: comparing search values.....	87
Figure 125: implementing binary search.....	87
Figure 126: Evidence of citation explanation of linked list	89
Figure 127: Initializing linked list in this program.....	90
Figure 128: using Linked list to display bakery items.....	90
Figure 129: using linked list to check duplicate data	91
Figure 130: using linked list to update items information	91
Figure 131: evidence of explanation of citation of array list	92
Figure 132: initializing array list in selection sort	93
Figure 133: using array list to sort by price.....	93
Figure 134: calculating price index	94
Figure 135: checking swapped values.....	94
Figure 136: Initializing array list in insertion sort	94
Figure 137: performing insertion sort in QuantitySortedList.....	95
Figure 138: evidence of citation of UI/UX development.....	97
Figure 139: Views package.....	98
Figure 140: contents of loading screen.....	98
Figure 141: designing loading screen.....	99
Figure 142: contents of home page	99
Figure 143: designing home page	100
Figure 144: contents of login page.....	100
Figure 145: designing home page	101
Figure 146: Contents of admin page	102
Figure 147: designing admin page.....	103
Figure 148: contents of order page.....	103
Figure 149: Contents of product page	104

Figure 150: Evidence of using youtube tutorial	106
Figure 151: evidence of using GitHub as reference	106
Figure 152: importing default table model	107
Figure 153: creating table model	107
Figure 154: adding data to model	107
Figure 155: screenshot before adding data.....	108
Figure 156: Evidence of adding data	108
Figure 157: evidence of using youtube tutorial for deleting data	109
Figure 158: importing default table model	110
Figure 159: creation of table method	110
Figure 160: select a row to be deleted.....	110
Figure 161: deleting data from table row	111
Figure 162: creating update method.....	112
Figure 163: update button action listener method	113
Figure 164: targeted product to update information	113
Figure 165: updating product information from GUI	114
Figure 166: empty fields validation in login page	116
Figure 167: incorrect credentials validation	117
Figure 168: After correct credentials validation	118
Figure 169: empty fields validation in admin page	119
Figure 170: incorrect data type validation.....	120
Figure 171: correct data type validation in admin page	121
Figure 172: clear text fields validation in admin page	122
Figure 173: delete data validation in admin page.....	123
Figure 174: item not found in search bar	124
Figure 175: item found validation in search bar	124
Figure 176: evidence of sorting algorithm citation.....	125
Figure 177: implementing Merge sort in program (i)	126
Figure 178: creating sub lists	127
Figure 179: implementing selection sort in the program	128
Figure 180: Implementing selection sort.....	129
Figure 181: implementing binary search.....	129

Table of Tables

Table 1: Planning test cases	37
Table 2: Test 1: Progress Bar	38
Table 3: Test 2: Login validation	40
Table 4: Test 3: Product Information	43
Table 5: Test 4: Submit data	47
Table 6: Test 5: Update data.....	50
Table 7: Test 6: Clear data.....	52
Table 8: Test 7: Delete record.....	54
Table 9: Test 8: sort by quantity.....	56
Table 10: Test 9: Sort by price	58
Table 11: Test 10: Sort by name.....	60
Table 12: Test 11: Search button.....	62

1.Introduction

Bake Ease, a bakery management system application built using Java Awt and swing package. This project specializes in management of bakery related operations for customer convenience.

1.1 Overview and purpose

The project is built using Eclipse IDE (NetBeans, n.d.) for designing we have used Canva for wireframe development and various references articles for research purpose. To maintain the dynamicity of the project, special care has been taken care, and many things have been taken into consideration in frontend and DSA concepts implementations. The source code for this project has been published on GitHub as a repository, so that the user can access it.

The project follows MVC architecture which separates the application into three different components which are **model**, **view** and **controller**. The **model** component is used to manage all the operations, the **view** component is where all the frontend operations are handled, and **controller** component is used to coordinate model and view component to retrieve or update data and determine appropriate view to display.

1.2 Audience

The project is made for bakery service providers who specialize in building system application for their user convenience.

1.3 Aims and objectives

1.3.1 Aims

The aim of this project this to provide user convenience as per their need. To maintain the bakery management system easy and comfortable for both user and the bakery owner and since this project is opensource so in future anyone from the internet can help to make it better.

1.3.2 Objectives

- To make the bakery management system convenient

- To make user experience better and better
- Streamline inventory operations
- Real time updates

1.4 Features

The features of this bakery management system application are:

- Admin dashboard: the dashboard can add and remove products of the bakery.
- View Products: this option allows user to see all the currently available products and make their order decision.
- Order Page: this is the page where the user can make their order about what they need and that order will be delivered.

2. Design

2.1 Loading page

2.1.1 Loading page reference

For loading page to be designed, I have used a simple yet minimalistic design. A simple page with a plain background color and with a logo above progress bar. For this I went to search on image gallery websites like Pinterest. (Pinterest, 2024)

Reason behind choosing pinterest as research platform is that Pinterest contains image gallery for related images with all types of formats. This website contains all sorts of image type, categorization, trending designs etc.

For designing wireframe, I have used Canva. Canva is an online designing tool that allows users to create their own designs using their creativity and online reference. (Canva, 2024)

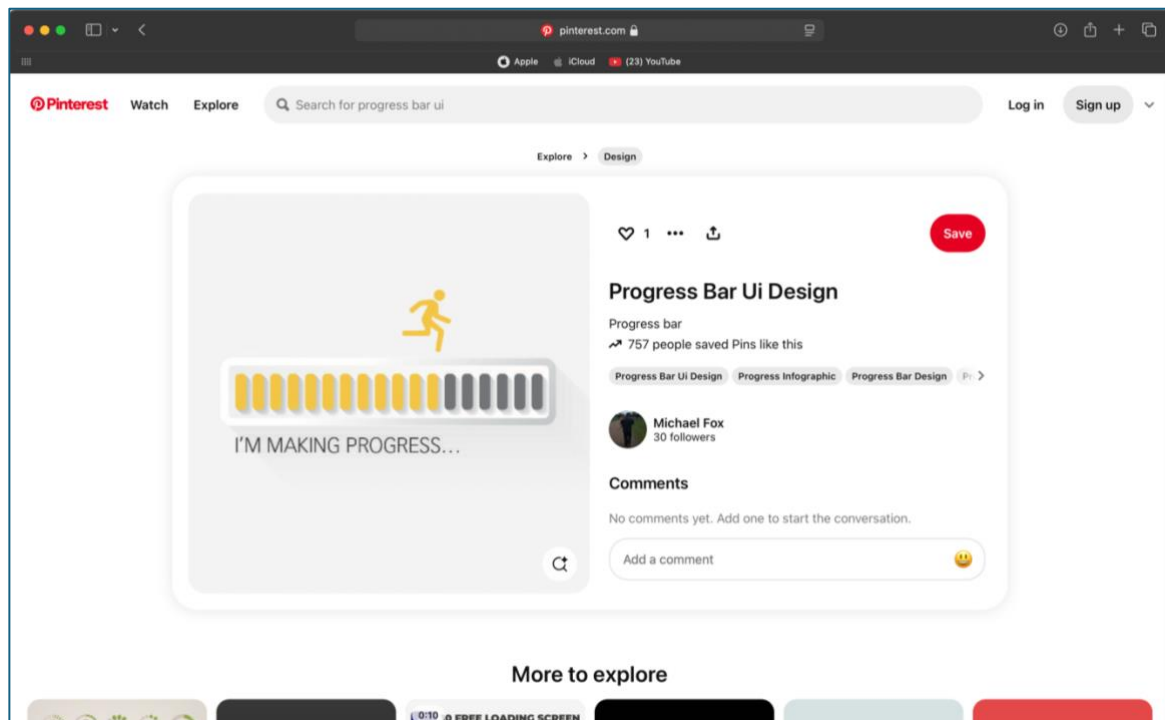


Figure 1: reference for loading page

2.1.2 Wireframe design of loading page

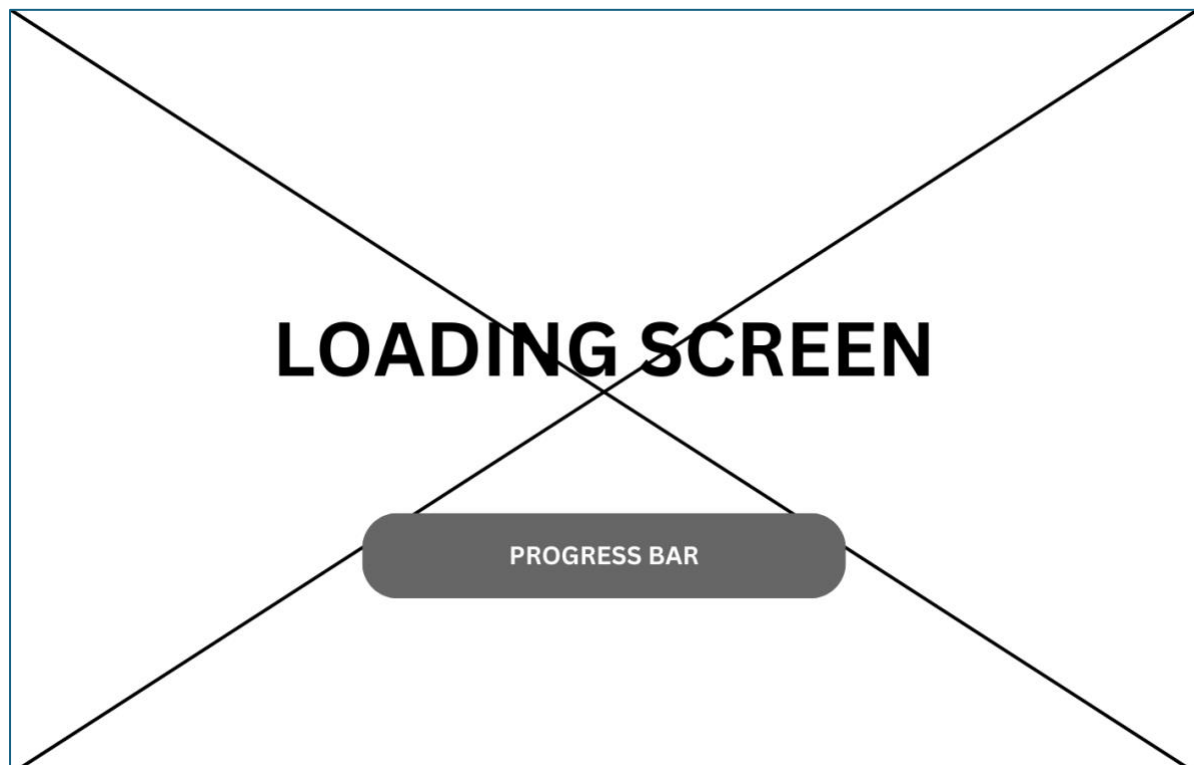


Figure 2: wireframe of loading page

2.1.2 Actual design of loading page



Figure 3: actual loading page

2.2 Login page

2.2.1 Login page reference

For the reference of login page, we searched online designs in Pinterest and came across many visual designs which represents my project.

For login page, I wanted it to look as simple as possible as it is the entry point of the program and users from well-educated to not well educated will be using the program. So, it requires the page to be simple and minimal. (reference, 2024)

For designing the wireframe of the page, we have used Canva as designing tool. This tool allows users to create their own designs.

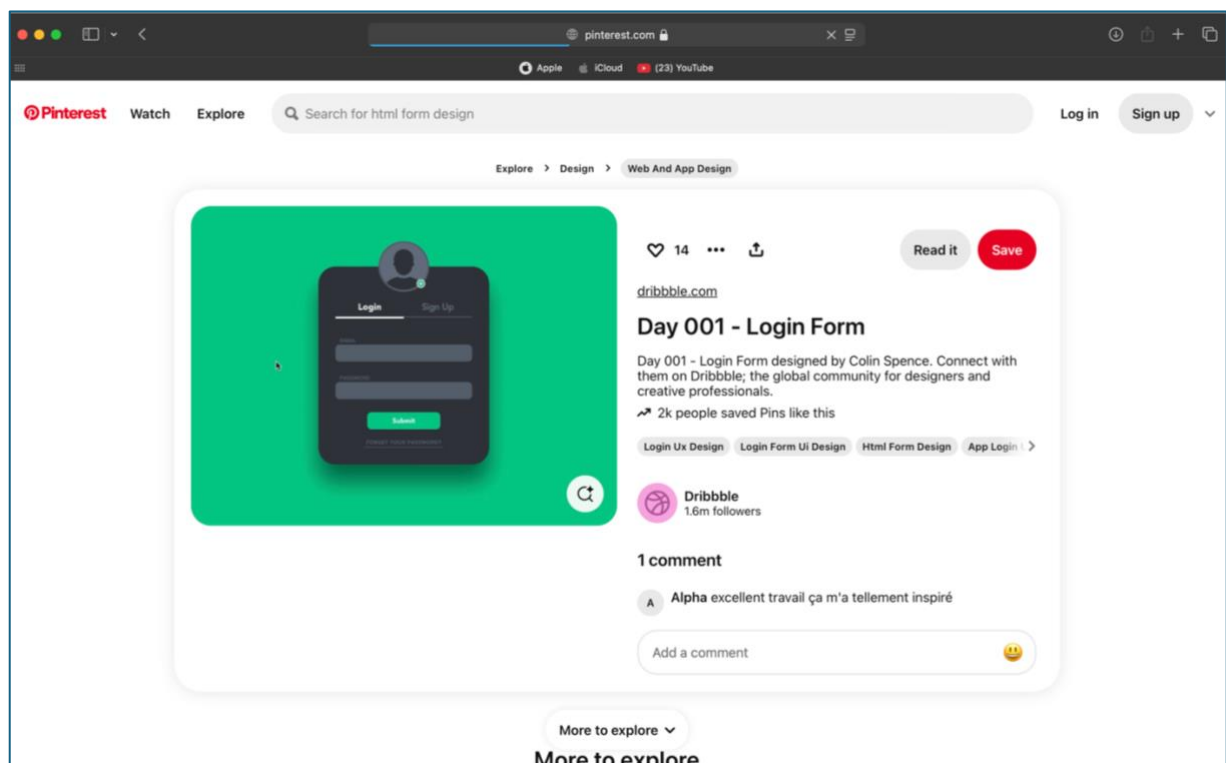
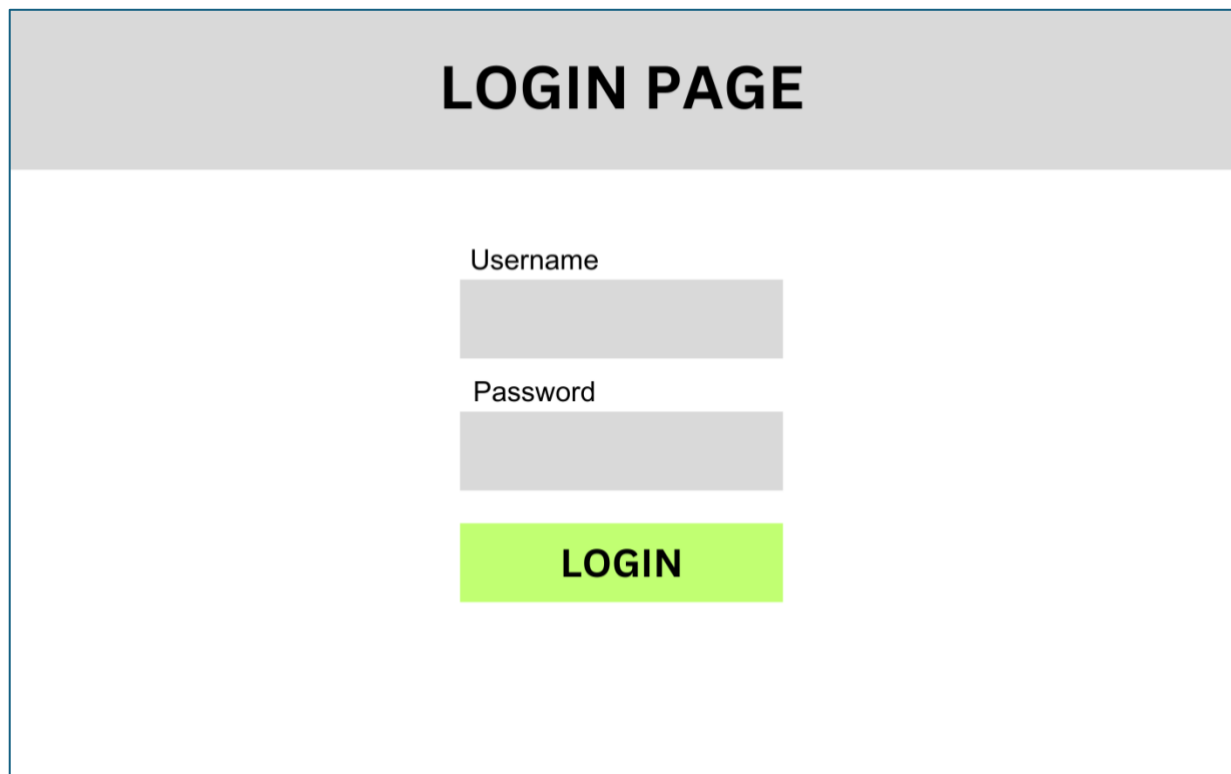


Figure 4: login page reference

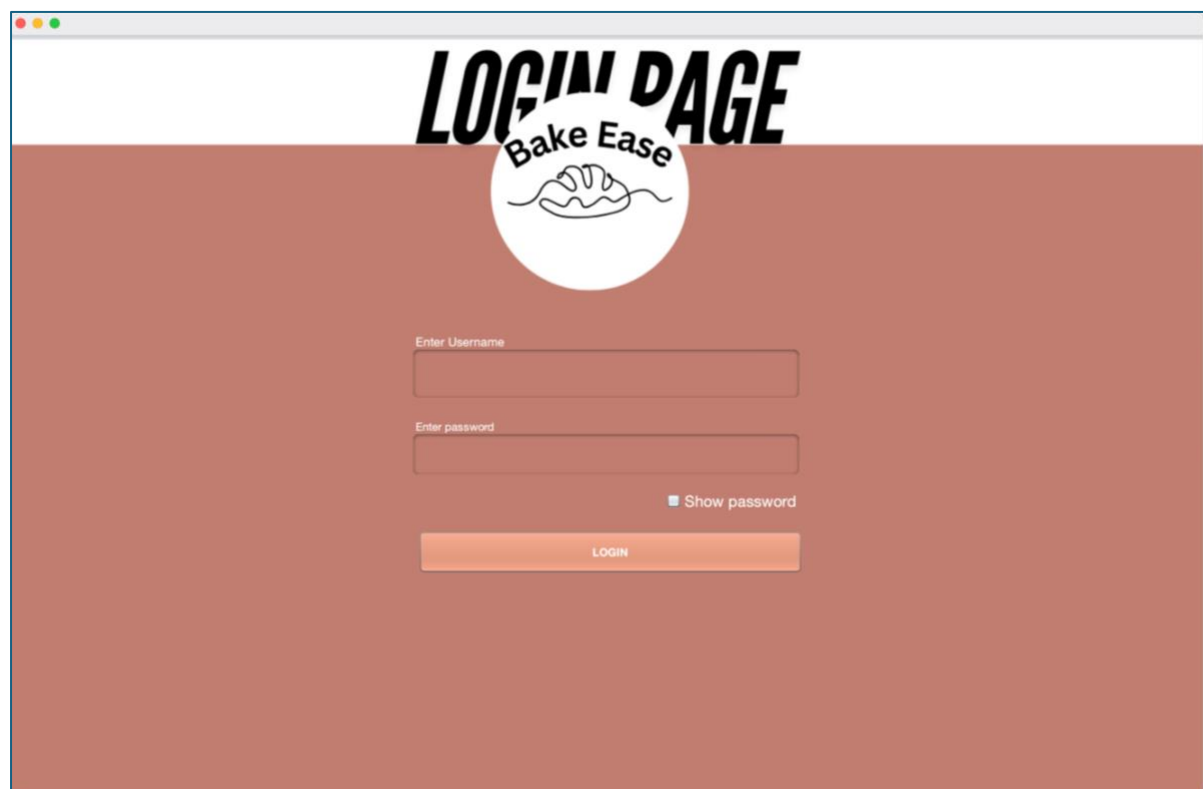
2.2.2 Wireframe design of login page



A wireframe design of a login page. It features a light gray header bar with the text "LOGIN PAGE" in bold black font. Below the header, the page has a white background. In the center, there are two input fields: the first is labeled "Username" and the second is labeled "Password". Both fields are represented by light gray rectangular boxes. Below these fields is a bright green rectangular button with the text "LOGIN" in bold black font.

Figure 5: wireframe of login page

2.2.3 Actual design of login page



An actual design of a login page. It is displayed within a browser window with a light gray title bar. The page has a white header area with the text "LOGIN PAGE" in a large, bold, black, stylized font. Below the header, there is a circular logo with the text "Bake Ease" and a line drawing of a cake. The main body of the page has a solid reddish-brown background. In the center, there are two input fields: the first is labeled "Enter Username" and the second is labeled "Enter password". Below the password field is a small checkbox labeled "Show password". At the bottom of the form is a light orange rectangular button with the text "LOGIN" in bold black font.

Figure 6: actual login page

2.3 Home page

For designing wireframe of the home page we used Canva, an online designing tool which is used to create designs as user wants. (Canva, n.d.)

2.3.1 Wireframe design of Home page

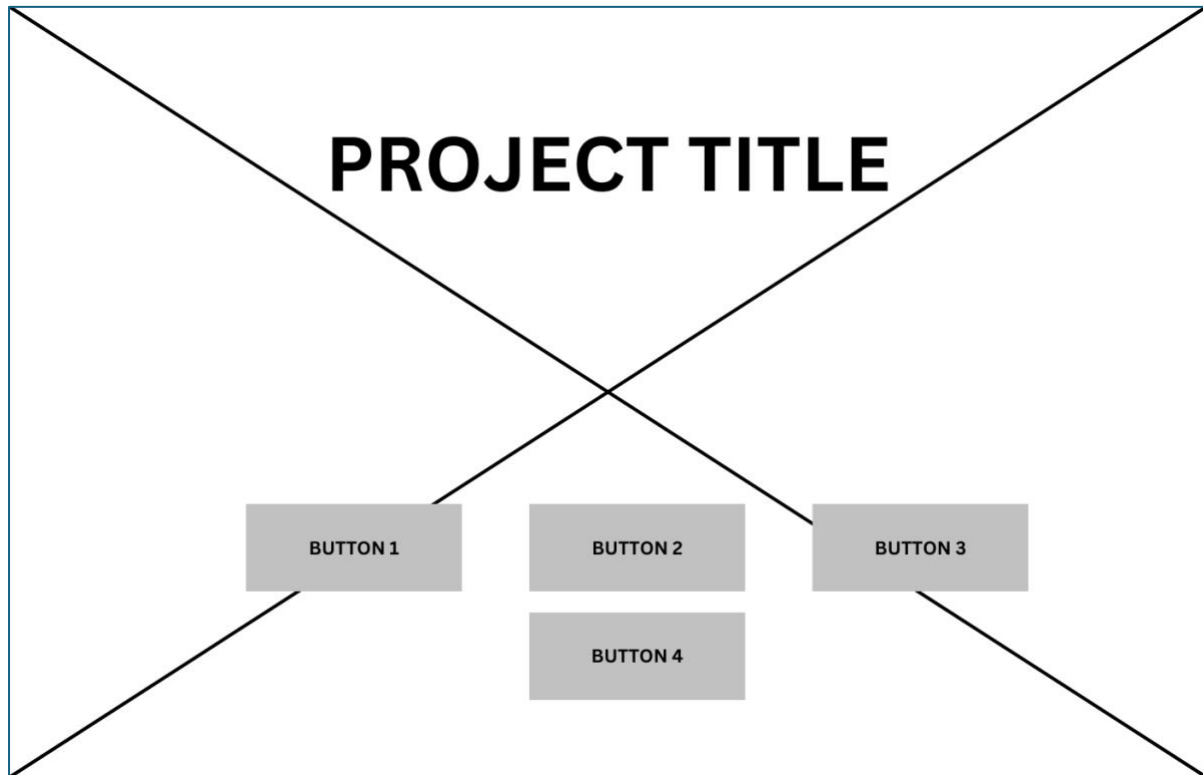


Figure 7: wireframe of home page

2.3.2 Actual design of Home page

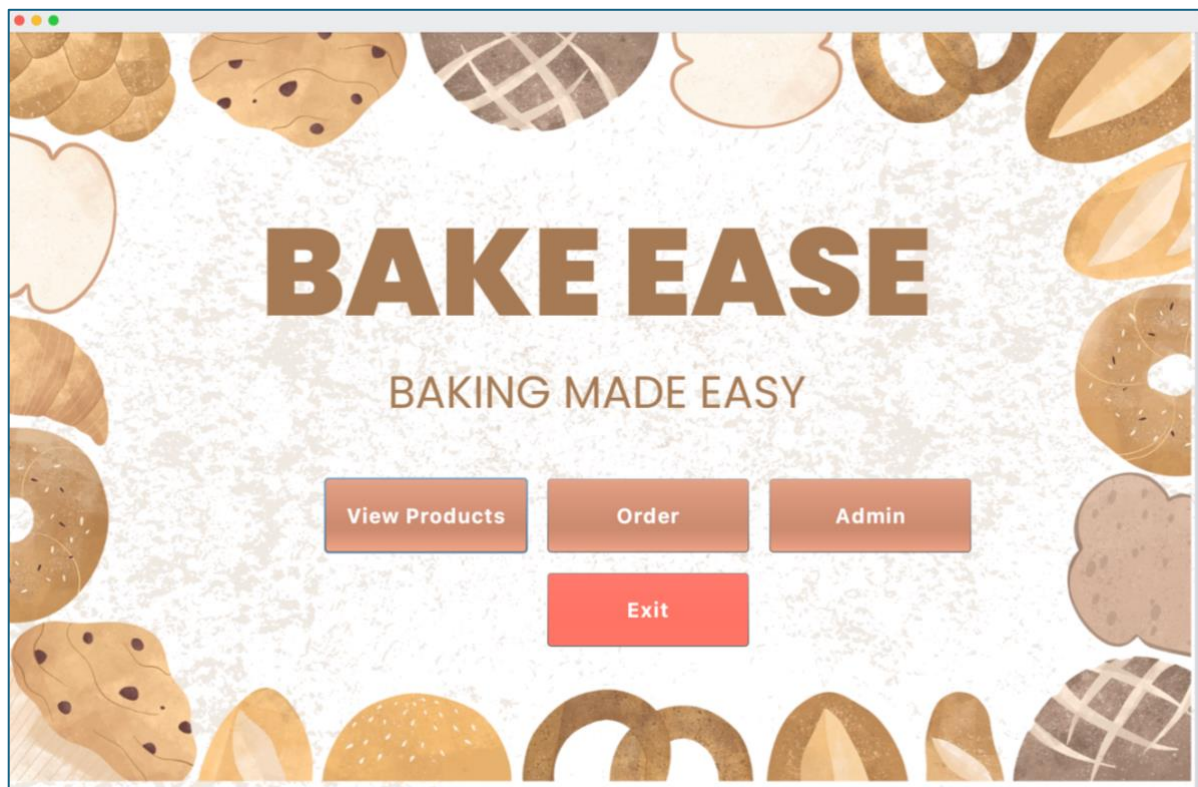


Figure 8: actual home page

2.4 Admin page

2.4.1 Reference of Admin page

For designing admin page, it is necessary for the page to be simple, effective and full of operations based. To find the reference of the admin page I went to stack overflow, an online forum for developer's problem solution. In this website I found the design that would somehow suit my GUI application. (Overflow, n.d.)

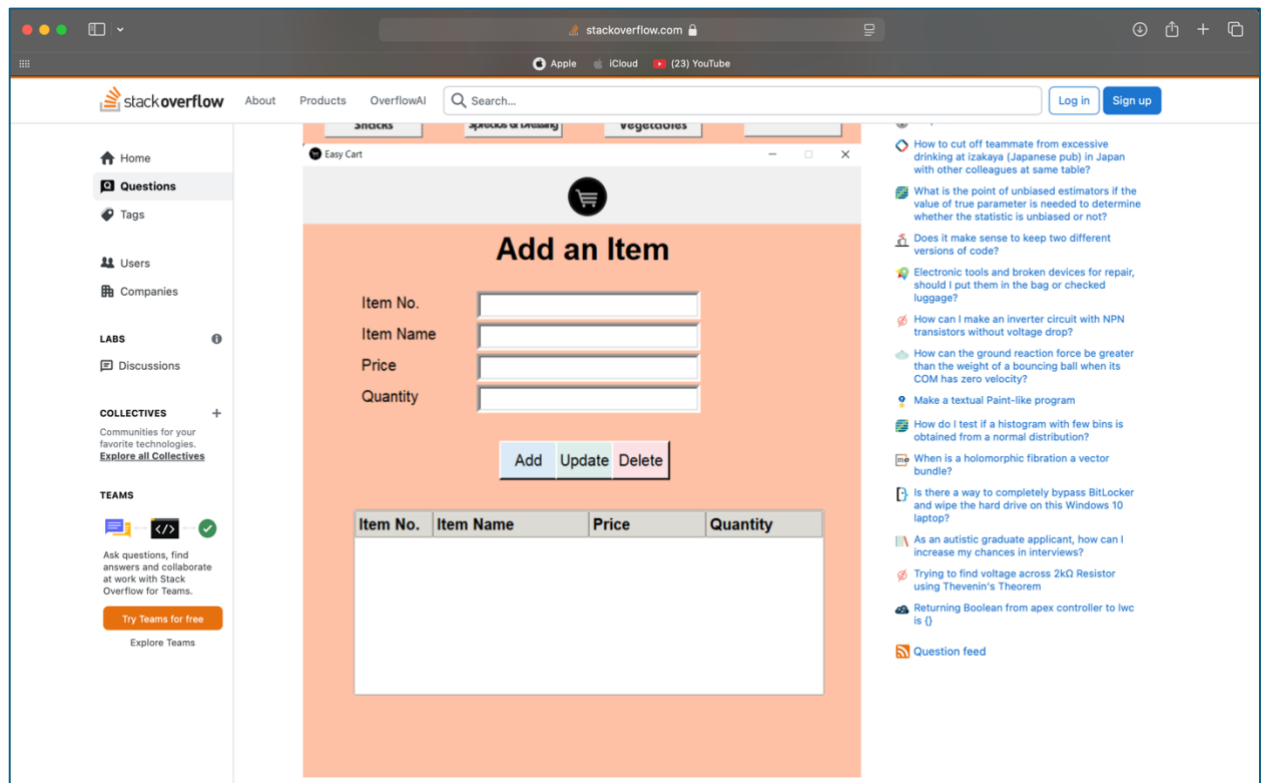


Figure 9: reference of home page

2.4.2 Wireframe of Admin page

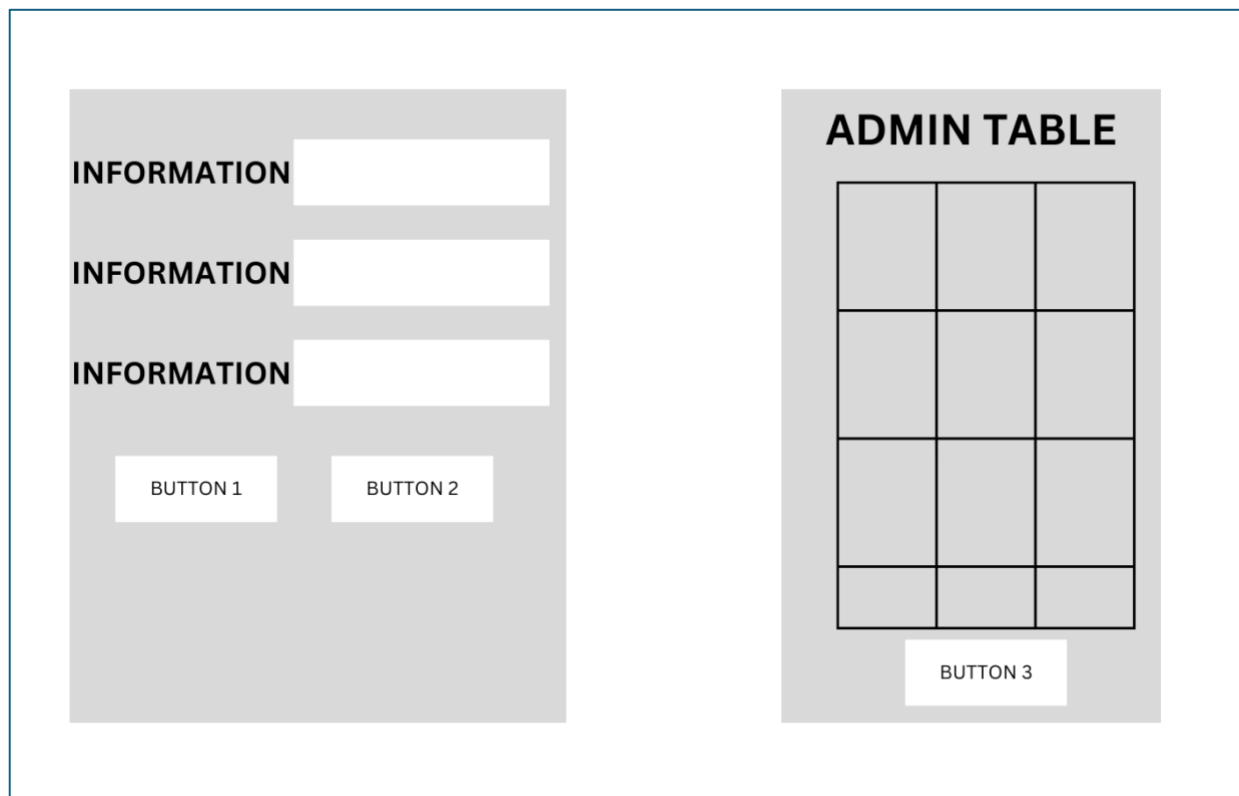


Figure 10: wireframe of admin page design

2.4.3 Actual design of Admin page

The Admin page design features a top navigation bar with a search bar labeled 'Search Product' and a magnifying glass icon. Below the navigation bar, the page is divided into two main sections: 'Enter Details' and 'Admin Dashboard'.

Enter Details

Enter product ID:

Enter product Name:

Enter Quantity:

Enter Price:

Buttons: Submit, Update, Clear

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Buttons: Delete Record, Sort by Quantity, Sort by Name, Sort by Price

Figure 11: actual home page

2.5 Product page

2.5.1 Reference of product page

For the product page, it should display available products in the store within the app. For this it requires nothing but product image, its name and description (optional). For designing product page, I visited a bakery website named Magnolia Bakery. There I found the matching product page for my GUI.

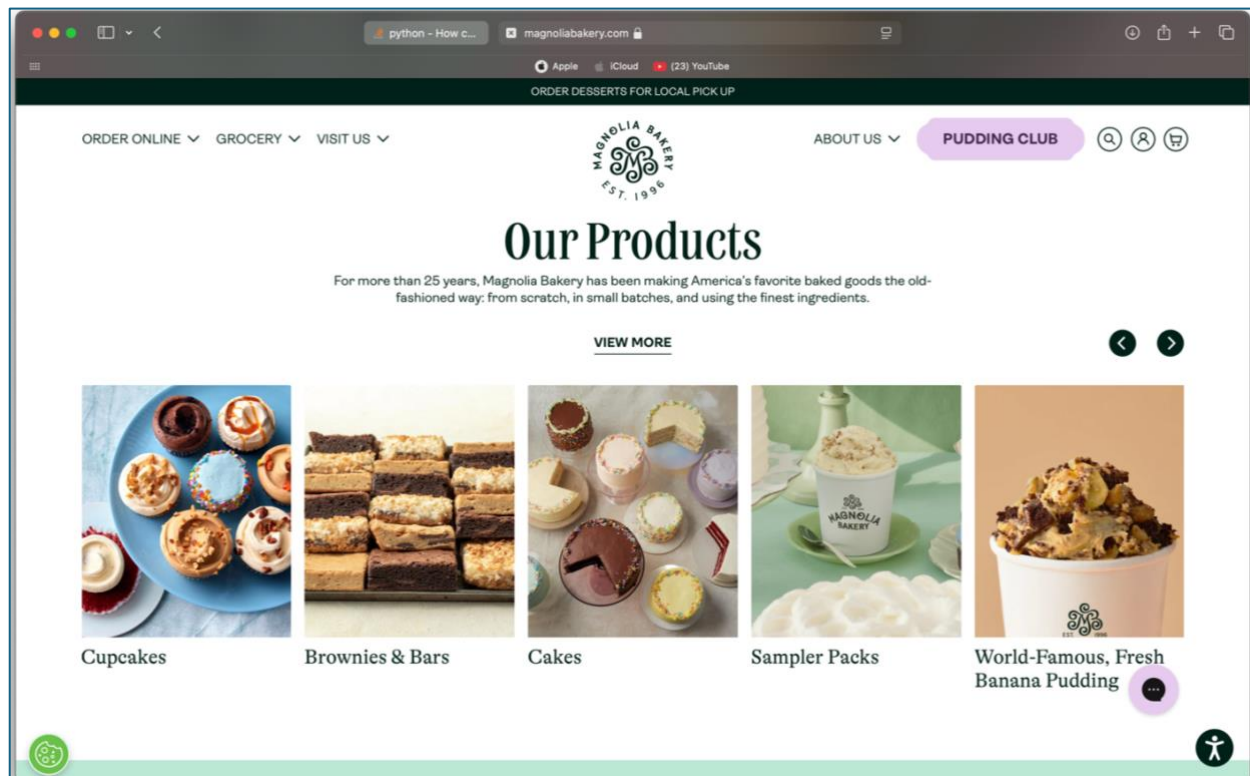


Figure 12: reference of product page

2.5.2 Wireframe of product page

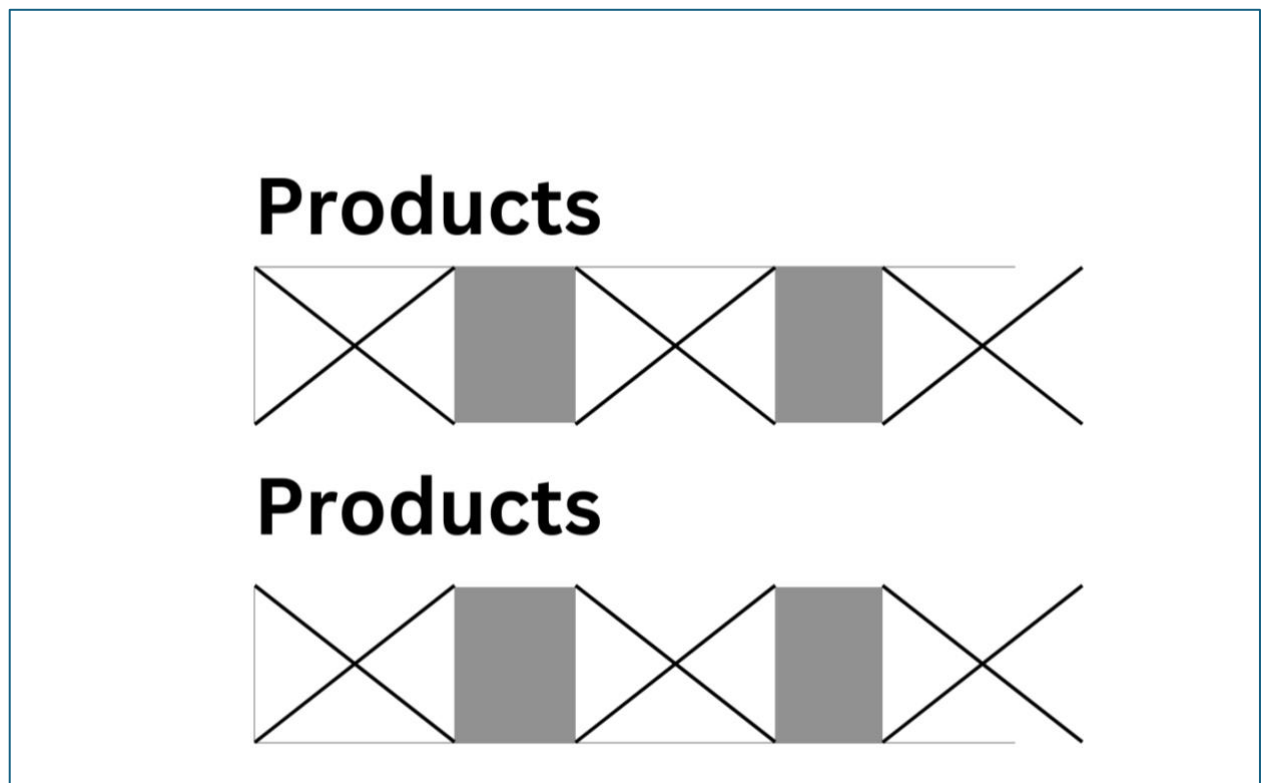


Figure 13: wireframe of product page

2.5.3 Actual design of product page

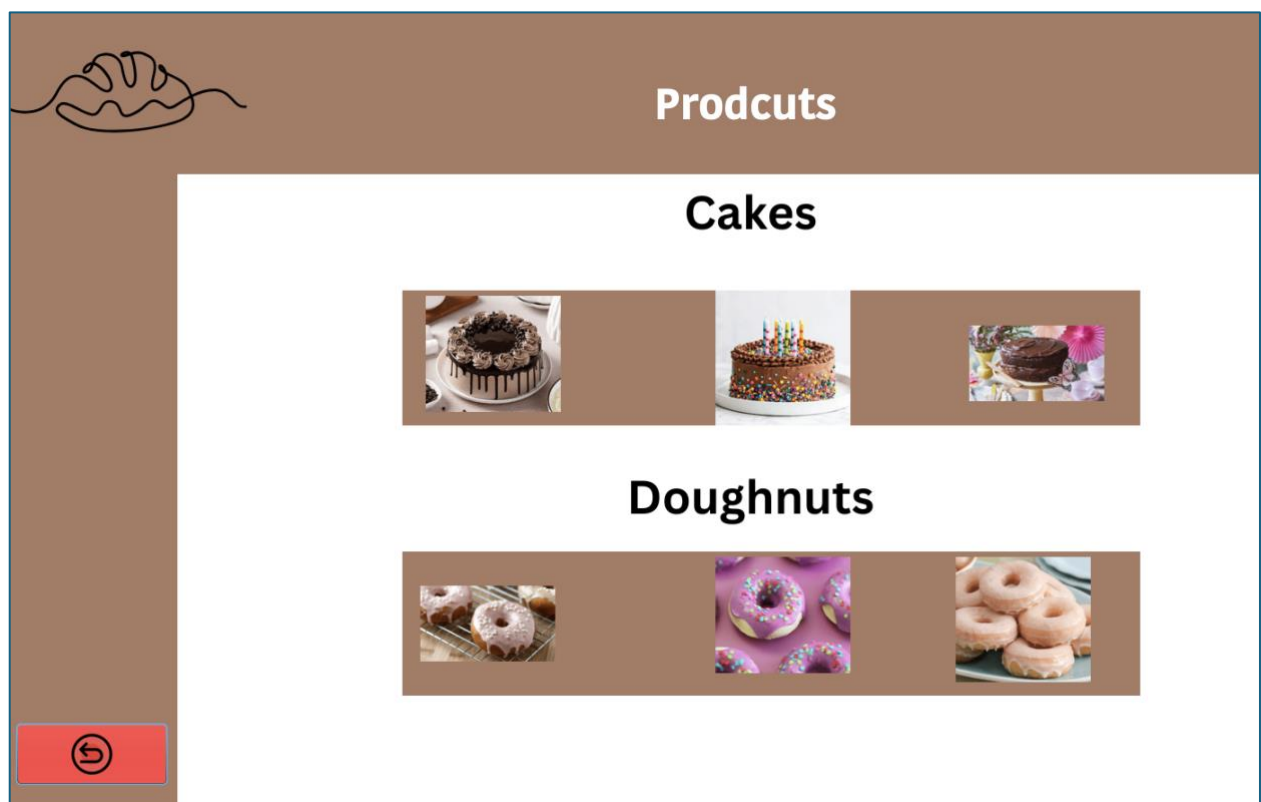


Figure 14: Actual product page

2.6 Color Pallet

A bakery resembles taste of bread, buns, cream, sweet, quick snacks etc. The color pallet is an essential component in designing an application as it provides clarity to the user. It resembles the type of software the user is using. To choose color pallet for this software I searched on Pinterest as this platform contains image gallery for every suitable image.

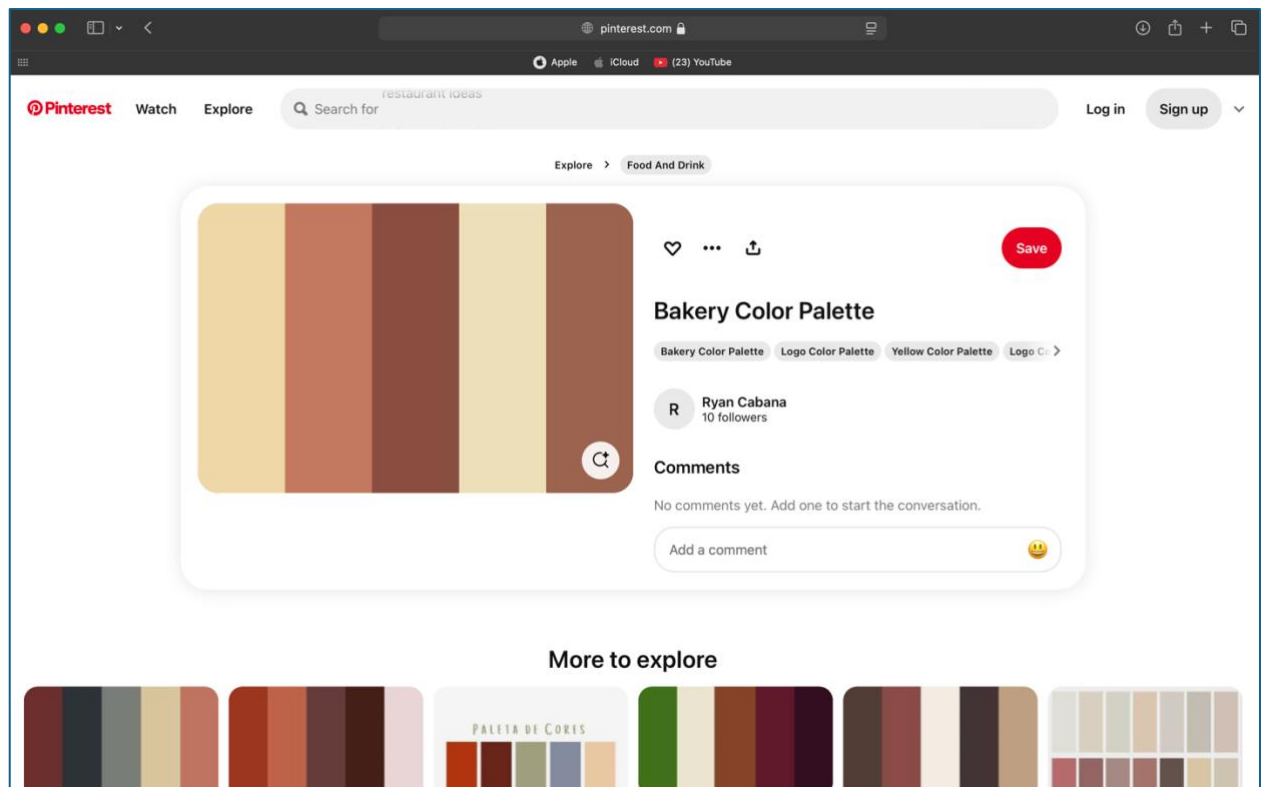


Figure 15: choosing color pallet

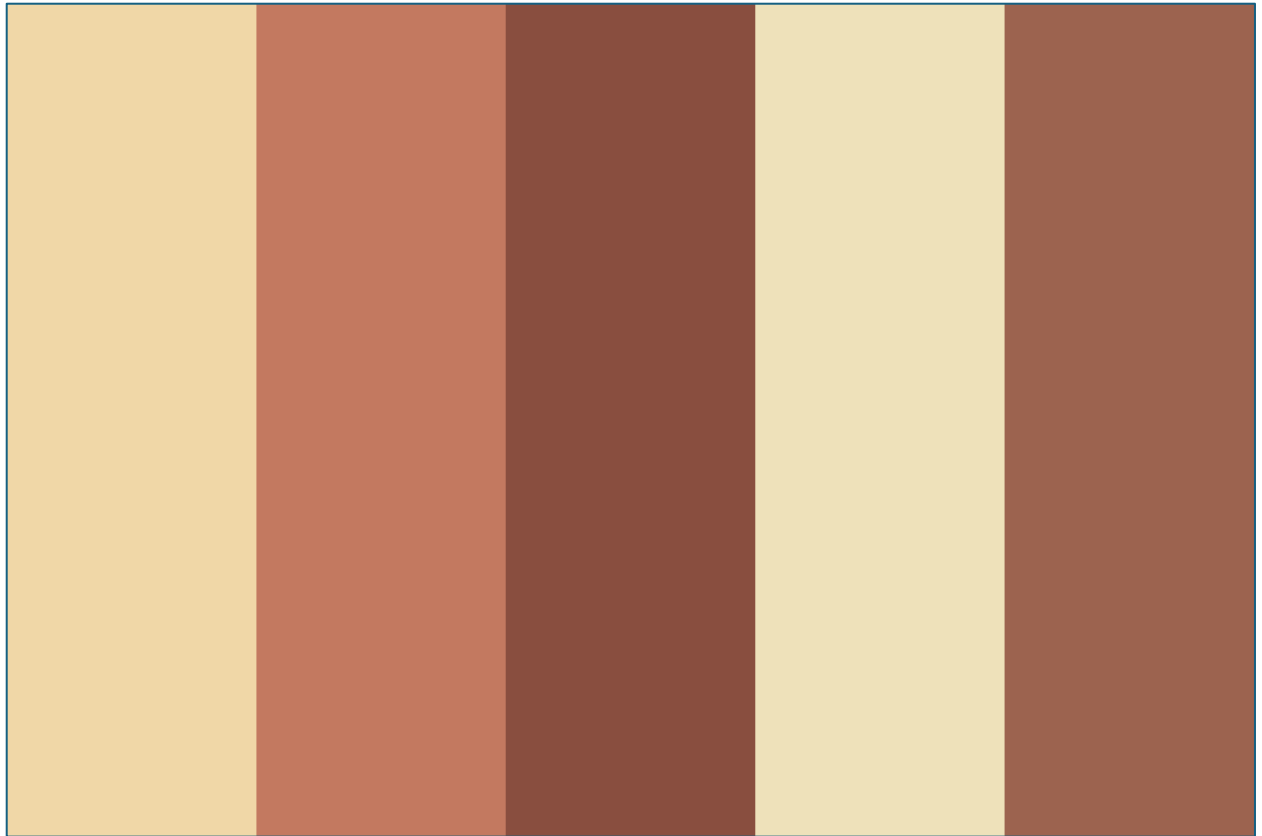


Figure 16: Actual color pallet

3. Java Classes and Methods

3.1 Overall Class Diagram

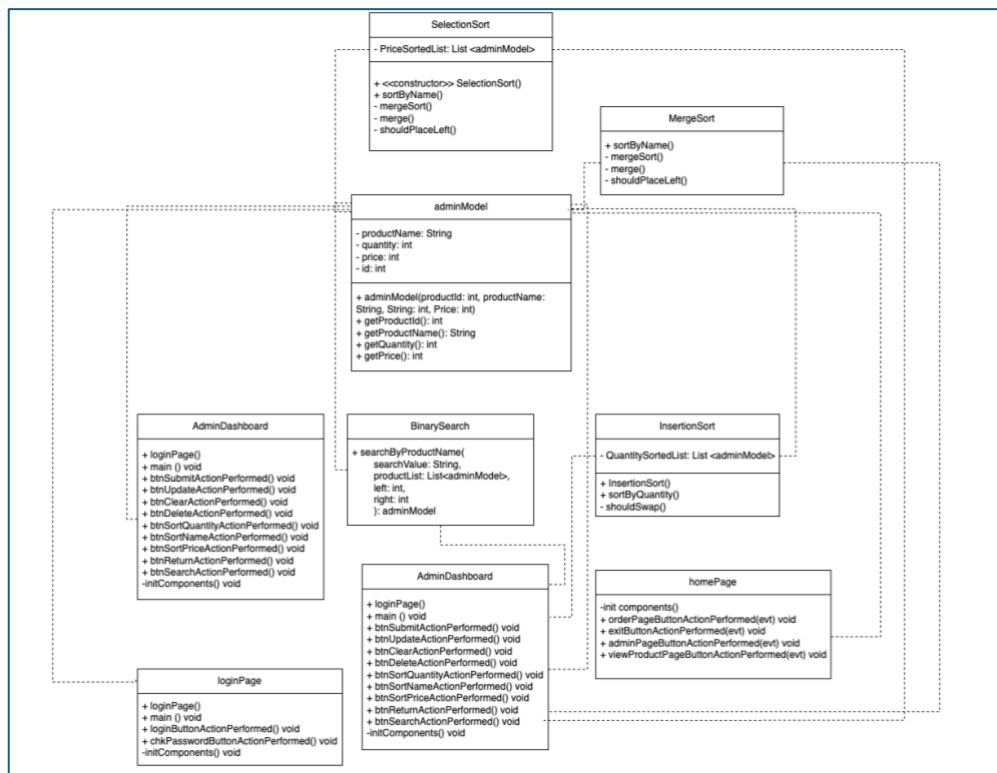


Figure 17: overall class diagram

3.2 Individual Class Diagram

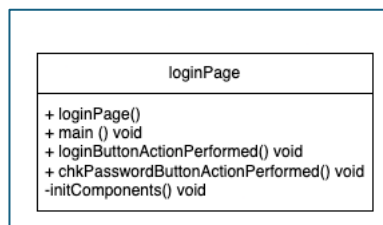


Figure 18: login page class diagram

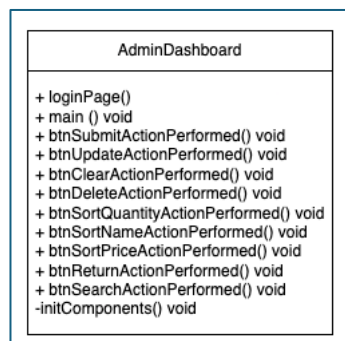


Figure 19: admin dashboard class diagram

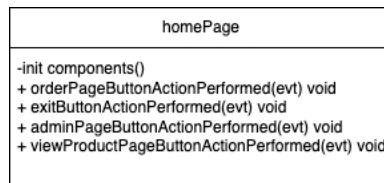


Figure 20: home page class diagram

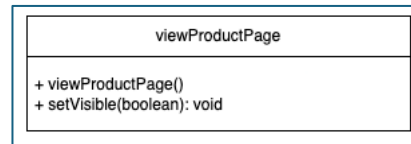


Figure 21: product page class diagram

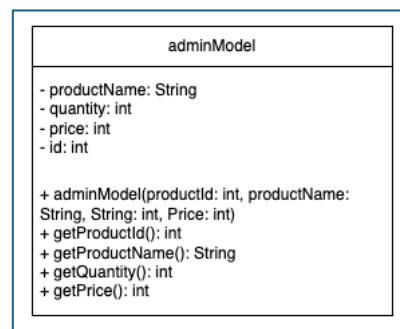


Figure 22: admin model class diagram

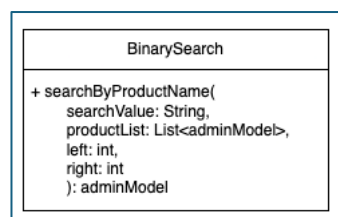


Figure 23: binary search class diagram

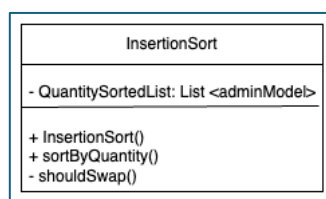


Figure 24: insertion sort class diagram

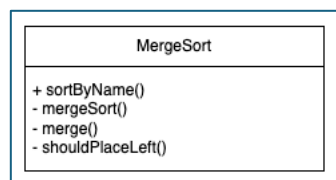


Figure 25: merge sort class diagram

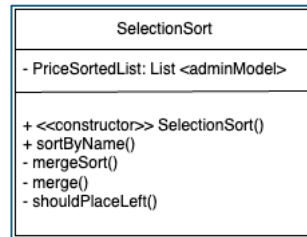


Figure 26: selection sort class diagram

3.2.1 Model Class

getProductId()

```

// declaring public getter method for product Id
public int getProductId() {
    return productId;
}
  
```

Figure 27: get product id()

This method retrieves the product id data.

getProductName()

```

// getter method for product name
public String getProductName() {
    return productName;
}
  
```

Figure 28: getproductname()

This method retrieves product name.

getQuantity()

```

// getter method for product quantity
public int getQuantity() {
    return quantity;
}
  
```

Figure 29: getQuantity()

This method retrieves product quantity.

getProductPrice()

```
// getter method for product price
public int getPrice() {
    return price;
}
```

Figure 30: getProductPrice()

This method retrieves product price.

setProductId()

```
// setter method for product name
public void setProductId(int productId) {
    this.productId = productId;
}
```

Figure 31: setProductId()

This method sets product Id.

setProductName()

```
// setter method for product name
public void setProductName(String productName) {
    this.productName = productName;
}
```

Figure 32: setProductName()

This method sets product name.

setProductQuantity()

```
// setter method for product quantity
public void setQuantity(int quantity) {
    this.quantity = quantity;
}
```

Figure 33: setProductQuantity()

This method sets product quantity.

setProductPrice()

```
// setter method for product price
public void setPrice(int price) {
    this.price = price;
}
```

Figure 34: setPrice()

This method sets product Price.

3.2.2 Algorithms Class

SearchByName()

```
public adminModel searchByProductName(String searchValue, List<adminModel> studentList,
    int left, int right) {

    // if the search range is invalid, return null
    if (right < left) {
        return null;
    }

    // mid value of current search range
    int mid = (left + right) / 2;

    // Compare the search value with the product name at the middle index
    if (searchValue.compareToIgnoreCase(studentList.get(mid).getProductName()) == 0) {
        // If the search value matches the product name, return the object
        return studentList.get(mid);
    } else if (searchValue.compareToIgnoreCase(studentList.get(mid).getProductName()) < 0) {
        // If the search value is less than the middle product name, search in the left sublist
        return searchByProductName(searchValue, studentList, left, mid - 1);
    } else {
        // If the search value is greater than the middle product name, search in the right sublist
        return searchByProductName(searchValue, studentList, mid + 1, right);
    }
}
```

Figure 35: SearchByName()

This method uses merge sort algorithm of merge sort class to identify the product based on its name. It is based on binary search algorithm. Its access modifier is public and it accepts parameters as list from admin model, left and right index. It calculates mid index and compares the values.

SortByName()

```
public List<adminModel> sortByName(List<adminModel> product, boolean isDesc) {
    if (product == null || product.isEmpty()) {
        throw new IllegalArgumentException("Product list is empty. Fix it!");
    }

    return mergeSort(product, isDesc);
}
```

Figure 36: sortByName()

This method validates the array list and throws exception if array list is found empty and returns merge sort values in descending order.

Mergesort()

```
/**
 * Recursively divides and sorts the list of adminModel objects using Merge Sort.
 *
 * @param product the list of adminModel items to be sorted
 * @param isDesc Boolean indicating the sort order
 * @return the sorted list of adminModel objects
 */
private List<adminModel> mergeSort(List<adminModel> product, boolean isDesc) {
    if (product.size() <= 1) {
        return product;
    }

    int mid = product.size() / 2;
    List<adminModel> left = new ArrayList<>(product.subList(0, mid));
    List<adminModel> right = new ArrayList<>(product.subList(mid, product.size()));

    // Recursively split and merge
    left = mergeSort(left, isDesc);
    right = mergeSort(right, isDesc);

    return merge(left, right, isDesc);
}
```

Figure 37: mergeSort()

This method implements merge sort algorithm. It divides the list into two sublists and returns those sub lists. It creates two sublists left and right and arranges values in descending alphabetical order. Its access modifier is private and it accepts parameters as admin model list named product, and a boolean value for descending order.

Merge()

```
/**
 * Merges two sorted sub lists into a single sorted list.
 *
 * @param left the left sub list
 * @param right the right sub list
 * @param isDesc Boolean indicating the sort order
 * @return the merged and sorted list
 */
private List<adminModel> merge(List<adminModel> left, List<adminModel> right, boolean isDesc) {
    List<adminModel> merged = new ArrayList<>();
    int i = 0, j = 0;
    // Merge based on the model names
    while (i < left.size() && j < right.size()) {
        if (shouldPlaceLeft(left.get(i).getProductName(), right.get(j).getProductName(), isDesc)) {
            merged.add(left.get(i));
            i++;
        } else {
            merged.add(right.get(j));
            j++;
        }
    }

    // Add remaining elements
    while (i < left.size()) {
        merged.add(left.get(i));
        i++;
    }

    while (j < right.size()) {
        merged.add(right.get(j));
        j++;
    }

    return merged;
}
```

Figure 38: merge()

This method merges two sublists if indexed element is found. Its access modifier is private and it accepts parameters in admin model list, left and right index, and boolean condition for descending order.

SortByQuantity()

```

/**
 * Sorts a list of adminModel objects by quantity.
 *
 * @param productList the list of adminModel objects to be sorted
 * @param isDesc      Boolean indicating whether to sort in descending order (true)
 *                   or ascending order (false)
 * @return the sorted list of adminModel objects
 * @throws IllegalArgumentException if the provided list is null or empty
 */
public List<adminModel> sortByQuantity(List<adminModel> productList, boolean isDesc) {
    this.QuantitySortedList.clear();
    this.QuantitySortedList.addAll(productList);
    if (QuantitySortedList == null || QuantitySortedList.isEmpty()) {
        throw new IllegalArgumentException("Product list is empty. Insert data!!");
    }

    // Perform the insertion sort
    for (int i = 1; i < QuantitySortedList.size(); i++) {
        adminModel key = QuantitySortedList.get(i);
        int j = i - 1;

        // Move elements of QuantitySortedList[0..i-1] that are greater (or smaller depending on isDesc) than key
        while (j >= 0 && shouldSwap(QuantitySortedList.get(j).getQuantity(), key.getQuantity(), isDesc)) {
            QuantitySortedList.set(j + 1, QuantitySortedList.get(j));
            j = j - 1;
        }
        // place key in correct position
        QuantitySortedList.set(j + 1, key);
    }

    return QuantitySortedList;
}

```

Figure 39: sortByQuantity()

This method uses insertion sort to sort the product based on its quantity. It validates empty list, sorts the product and returns sorted list. Its access modifier is public and it accepts parameters as product list, boolean flag. It returns sorted list for quantity.

shouldSwap()

```

/** checks if values are to be swapped or not based on isDesc flag
 * @param current the current quantity being compared
 * @param key      the quantity to compare against
 * @param isDesc   boolean indicating the sort order
 * @return true if the elements should be swapped, false otherwise
 */
private boolean shouldSwap(int current, int key, boolean isDesc) {
    return isDesc ? current < key : current > key;
}

```

Figure 40: shouldSwap()

This method checks if the values are swapped or not.

SortByPrice()

```

/*
price sorting starts.
@param array list priceSortedList
validates empty sort
*/
public List<adminModel> sortByPrice(List<adminModel> productList, boolean isDesc) {
    this.PriceSortedList.clear();
    this.PriceSortedList.addAll(productList);
    if (PriceSortedList == null || PriceSortedList.isEmpty()) {
        throw new IllegalArgumentException("Product list is empty. Insert data!!");
    }

    for (int i = 0; i < PriceSortedList.size() - 1; i++) {
        int priceIndex = findPriceIndex(PriceSortedList, i, isDesc);
        if (i != priceIndex) {
            swap(PriceSortedList, i, priceIndex);
        }
    }

    return PriceSortedList;
}

```

Figure 41: sortByPrice()

This method uses selection sort to sort items based on their price from high to low. It clears all the list first, then adds all data to product list. Then validates empty list and swaps the greater and smaller values. Its access modifier is public, it accepts parameters as product list and boolean flag and returns sorted price items.

findPriceIndex()

```

/*
method to find price index
@param priceSortedList is used to get price
*/
private int findPriceIndex(List<adminModel> priceSortList, int startIndex, boolean isDesc) {
    int extremumIndex = startIndex;

    for (int j = startIndex + 1; j < priceSortList.size(); j++) {
        if (shouldSwap(PriceSortedList.get(j).getPrice(), PriceSortedList.get(extremumIndex).getPrice(), isDesc)) {
            extremumIndex = j;
        }
    }

    return extremumIndex;
}

```

Figure 42: findPriceIndex()

This method is used to calculate price index. It uses extremum to calculate it. Its access modifier is private, it accepts parameters as product list, start index and boolean flag and returns extremum index.

```

// checks is values are to be swapped or not
private boolean shouldSwap(int current, int extremum, boolean isDesc) {
    return isDesc ? current > extremum : current < extremum;
}
// swaps sorted price value
private void swap(List<adminModel> priceSortList, int i, int j) {
    adminModel temp = PriceSortedList.get(i);
    PriceSortedList.set(i, PriceSortedList.get(j));
    PriceSortedList.set(j, temp);
}

```

Figure 43: Swap

These methods help swapping the values which price index are calculated.

3.2.3 Views Class

3.2.3.1 Login Page

loginButtonActionPerformed()

```

// method to login to the system
private void loginButtonActionPerformed(java.awt.event.ActionEvent evt) {
    // validating empty fields
    if (usernameTextField.getText().equals("") || passwordTextField.getText().equals("")) {
        JOptionPane.showMessageDialog(null, "Please enter credentials.");
    } else if (usernameTextField.getText().equals("admin") && passwordTextField.getText().equals("admin")) {
        setVisible(false);
        // creating object of home page
        homePage home = new homePage();
        // Set the homePage visible
        home.setVisible(true);
        JOptionPane.showMessageDialog(null, "Welcome.");
    } else {
        // instruction to enter credentials correctly
        JOptionPane.showMessageDialog(null, "Enter credentials correctly.");
        usernameTextField.setText("");
        passwordTextField.setText("");
    }
}
}

```

Figure 44: loginButtonActionPerformed()

This is an action performed method of login button from the login page. This method validates empty fields in the GUI. It compares the default set values and allows user to enter into the system if credentials are correct. If the credentials are incorrect, it prompts user to enter credentials correctly. Incase the user enters the login button without entering any credentials it will instruct user to enter credentials.

chkBoxShowPasswordActionPerformed()

```
// method to enable check box to show and hide password
private void chkBoxShowPasswordActionPerformed(java.awt.event.ActionEvent evt) {
    // checkbox to show password
    if (chkBoxShowPassword.isSelected()) {
        passwordTextField.setEchoChar((char) 0); // Show password
    } else {
        // hide password again
        passwordTextField.setEchoChar('*'); // Mask password
    }
}
```

Figure 45: chkBoxShowPasswordActionPerformed()

This method is used to hide and show password in the GUI.

3.2.3.2 Home Page

orderPageButtonActionPerformed()

```
private void orderPageButtonActionPerformed(java.awt.event.ActionEvent evt) {
    // creating object of order page
    orderPage order = new orderPage();
    // Set the homePage visible
    order.setVisible(true);

    // set current page visible as false
    setVisible(false);
}
```

Figure 46:orderPageButtonActionPerformed

This method is used open the order page in the GUI by instance of order page and sets current page visibility as false.

adminPageButtonActionPerformed()

```
private void adminPageButtonActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    // creating object of login page
    AdminDashboard admin = new AdminDashboard();
    // setting set visible as True
    admin.setVisible(true);
    // set home page visible as false
    setVisible(false);
}
```

Figure 47: adminPageButtonActionPerformed()

This method is used open the order page in the GUI by instance of admin page and sets current page visibility as false.

viewProductsButtonActionPerformed()

```
private void viewProductsButtonActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    viewProductPage product = new viewProductPage();
    // setting the product page frame set visible as TRUE
    product.setVisible(true);
    // set current page visible as false
    setVisible(false);
}
```

Figure 48: viewProductsButtonActionPerformed()

This method is used open the order page in the GUI by instance of product page and sets current page visibility as false.

exitButtonActionPerformed()

```
private void exitButtonActionPerformed(java.awt.event.ActionEvent evt) {
    System.exit(0);
    // terminates the whole program
}
```

Figure 49: exitButtonActionPerformed()

This action performed button is used to terminate the program.

3.2.3.3 Admin Page addBakeryItems()

```
// method to add bakery Items to table row
/*
@param ProductId the product ID
@param ProductName the product name
@param Quantity the Quantity
@param Price the Price
*/
public void addBakeryItems(adminModel bakeryItems) {
    productList.add(bakeryItems);
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    Object[] bakeryData = new Object[]{
        bakeryItems.getProductId(), bakeryItems.getProductName(), bakeryItems.getQuantity(), bakeryItems.getPrice()
    };
    model.addRow(bakeryData);
}
```

Figure 50: addBakeryItems()

This method is used to initialize for the method which displays pre loaded data into the table.

dislayPreLoadedItems()

```
/*
method to display pre loaded bakery items
validates the data into table row
@param item1, item2, item3, item4, item5, item6, item7, item8
*/
public void dislayPreLoadedItems() {
    /*
    creating object of model class
    calling method addBakeryItems() to add bakery items to table row
    passing item object in the method
    */
    adminModel item1 = new adminModel(1, "Crossiant", 22, 420);
    addBakeryItems(item1);

    adminModel item2 = new adminModel(2, "Cake", 12, 300);
    addBakeryItems(item2);

    adminModel item3 = new adminModel(3, "Pastry", 10, 200);
    addBakeryItems(item3);

    adminModel item4 = new adminModel(4, "Cheese Cake", 14, 69);
    addBakeryItems(item4);

    adminModel item5 = new adminModel(5, "Rumballs", 7, 690);
    addBakeryItems(item5);

    adminModel item6 = new adminModel(6, "Whole wheat bread", 4, 120);
    addBakeryItems(item6);

    adminModel item7 = new adminModel(7, "Doughnuts", 5, 60);
    addBakeryItems(item7);

    adminModel item8 = new adminModel(8, "Muffins", 7, 70);
    addBakeryItems(item8);
}
```

Figure 51: dislayPreLoadedItems()

This method is used to show pre loaded data into the admin dashboard table. It creates object for every item and adds the data into the object and that object is sent to add items method.

checkDuplicateProduct()

```
// method to check duplicate data in the data
private boolean checkDuplicateProduct(adminModel bakeryItems) {
    for (adminModel existingBakeryItem : productList) {
        if (existingBakeryItem.getProductid() == bakeryItems.getProductid()) {
            return true;
        }
    }
    return false;
}
```

Figure 52: checkDuplicateProduct()

This method is used to check duplicate product data. It uses product id to confirm duplicate data.

btnSubmitActionPerformed()

```
*/
private void btnSubmitActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (txtFldProductId.getText().equals("") || txtFldProductName.getText().equals("") || txtFldQuantity.getText().equals("") || txtFldPrice.getText().equals("")) {
        JOptionPane.showMessageDialog(null, "Empty Fields Detected!", "Error", JOptionPane.ERROR_MESSAGE);
    } else {
        try {
            // parsing the supposed strings values into integer
            int quantityInt = Integer.parseInt(txtFldQuantity.getText());
            int priceInt = Integer.parseInt(txtFldPrice.getText());

            // Get input values
            String productName = txtFldProductName.getText();
            int quantity = Integer.parseInt(txtFldQuantity.getText());
            int price = Integer.parseInt(txtFldPrice.getText());
            int productId = Integer.parseInt(txtFldProductId.getText());

            // stating a valid condition as TRUE initially
            boolean isCorrect = true;

            if (isCorrect) {
                // creating object of AdminModel class to pass values in parameters
                adminModel newBakeryItem = new adminModel(productId, productName, quantity, price);
                if (checkDuplicateProduct(newBakeryItem)) {
                    JOptionPane.showMessageDialog(null, "Same product already exists. \nPlease change the product information or try updating pr", "Error", JOptionPane.ERROR_MESSAGE);
                } else {
                    addBakeryItems(newBakeryItem);
                    JOptionPane.showMessageDialog(null, "Product added succesfully.", "Success", JOptionPane.INFORMATION_MESSAGE);
                }
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Product Id, Quantity and Price must be in numeric form!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Figure 53: btnSubmitActionPerformed()

This method is used to submit the product information into the table. It validates empty linked list. It checks duplicate product data, and after all the conditions are met the data is added to admin dashboard table. It has multiple try and catch statements to reduce errors in the system.

btnDeleteActionPerformed()

```
/*
method to delete table row records
@param adminTable for admin table variable name
*/
private void btnDeleteActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    int selectedRow = adminTable.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(null, "Select a row to be deleted.", "Error", JOptionPane.ERROR_MESSAGE);
    } else {
        model.removeRow(selectedRow);
        JOptionPane.showMessageDialog(null, "Row deleted succesfully!", "Row Deleted", JOptionPane.INFORMATION_MESSAGE);
    }
}
```

Figure 54: btnDeleteActionPerformed()

This button is used to delete data from the table row.

btnReturnActionPerformed()

```
/*
method to return back to home page
creates object of Home frame
*/
private void btnReturnActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    // creating object of home page
    HomePage home = new HomePage();
    // Set the homePage visible
    home.setVisible(true);
    // set current page visible as false
    setVisible(false);
}
```

Figure 55: btnReturnActionPerformed()

This button action performed method is used to return to home page form the current page.

updateItems()

```

/*
method to update table item
@param product id
@param product name
@param product quantity
@param price
*/
private void updateItems() {
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    // Clear the table before loading new data
    // set model row count to 0 to locate to exact updation location
    model.setRowCount(0);
    // add row for every fields
    for (adminModel products : productList) {
        model.addRow(new Object[]{
            products.getProductid(),
            products.getProductname(),
            products.getQuantity(),
            products.getPrice()
        });
    }
}

```

Figure 56:updateItems()

This method is used to initialize update data method to update product information. It accepts product id, product name, product quantity, product price and sets model count to 0.

btnUpdateActionPerformed()

```

/*
update button method
*/
private void btnUpdateActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int Id = Integer.parseInt(txtFldProductId.getText());
    String Name = txtFldProductName.getText();
    int price = Integer.parseInt(txtFldPrice.getText());
    int quantity = Integer.parseInt(txtFldQuantity.getText());

    boolean isValid = true;

    if (isValid) {
        for (adminModel menu : productList) {
            // Matching product id
            if (menu.getProductid() == Id) {
                menu.setProductname(Name);
                menu.setQuantity(quantity);
                menu.setPrice(price);
                menu.setProductId(Id);

                // updates product details
                // calling above update method
                updateItems();
                JOptionPane.showMessageDialog(null, "Product information updated succesfully.", "Updated", JOptionPane.INFORMATION_MESSAGE);
                return;
            }
        }
    }
}

```

Figure 57: btnUpdateActionPerformed()

This method is used to update product information in the admin table. This method uses setter methods to set new product information. It also validates integer manipulation.

btnClearActionPerformed()

```
/*
clear text button
clears all the text fields
@param txtFldProductName
@param txtFldProductId
@param txtFldQuantity
@param txtFldPrice
*/
private void btnClearActionPerformed(java.awt.event.ActionEvent evt) {
    if (txtFldProductName.getText().equals("") && txtFldProductId.getText().equals("") && txtFldQuantity.getText().equals("") && txtFldPrice.getText().equals("")) {
        JOptionPane.showMessageDialog(null, "Nothing to clear.", "Not Cleared", JOptionPane.WARNING_MESSAGE);
    } else {
        int response = JOptionPane.showConfirmDialog(null, "Are you sure you want to clear all the text fields ?", "Confirm Clear", JOptionPane.YES_NO_OPTION);
        if (response == JOptionPane.YES_OPTION) {
            txtFldProductName.setText("");
            txtFldProductId.setText("");
            txtFldQuantity.setText("");
            txtFldPrice.setText("");
        } else {
            JOptionPane.showMessageDialog(null, "Nothing is cleared.", "Not Cleared", JOptionPane.WARNING_MESSAGE);
        }
    }
}
```

Figure 58: btnClearActionPerformed()

This method is used to clear all the text fields in the GUI.

loadSortedProductData()

```
/*
method to perform sort
sorts the data in the table according to "price", "quantity" and "name"
*/
private void loadSortedProductData(List<adminModel> sortedData) {
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();

    // Clear the table before loading new data
    model.setRowCount(0);

    // Iterate through the LinkedList and add each member to the table
    for (adminModel products : sortedData) {
        model.addRow(new Object[]{
            products.getProductID(),
            products.getProductName(),
            products.getQuantity(),
            products.getPrice()
        });
    }
}
```

Figure 59: loadSortedProductData()

This method is used as global sorting method to perform sorting algorithm.

btnSortPriceActionPerformed()

```
/*
method to execute selection sort for price
Arranges item price ranging from high to low
@param adminModel for model
creates a object sort and handles sorting
Adds data to list
*/
private void btnSortPriceActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    SelectionSort sort = new SelectionSort();
    List<adminModel> sortedList = sort.sortByPrice(productList, true);
    loadSortedProductData(sortedList);
}
```

Figure 60: btnSortPriceActionPerformed()

This method is used to perform selection sort. It creates instance of selection sort class and sorts item according to its price.

btnSortQuantityActionPerformed()

```
/*
insertion sort for sort by quantity
@param adminModel for model
creates object sortQuantity to handle sorting
Adds data to list
*/
private void btnSortQuantityActionPerformed(java.awt.event.ActionEvent evt) {
    // Create an instance of the InsertionSort class
    InsertionSort sortQuantity = new InsertionSort();

    // Ensure productList is properly populated with product data
    // Sort the product list by quantity (descending order in this case)
    List<adminModel> sortedList = sortQuantity.sortByQuantity(productList, true);

    // Load the sorted data into the table
    loadSortedProductData(sortedList);
}
```

Figure 61: btnSortQuantityActionPerformed()

This method is used to perform insertion sort. It creates instance of insertion sort class and sorts products according to its quantity.

btnSortNameActionPerformed()

```
/*
method to sort by name
@param adminModel for model
creates object merge to handle sorting
adds data to list
*/
private void btnSortNameActionPerformed(java.awt.event.ActionEvent evt) {
    MergeSort sort = new MergeSort();
    List<adminModel> merge = sort.sortByName(productList, true);
    loadSortedProductData(merge);
}
```

Figure 62: btnSortNameActionPerformed()

This method is used to perform merge sort. It creates instance of merge sort class and sorts products according to its name.

btnSearchActionPerformed()

```
/*
button to execute search feature
instructs user to provide proper details in case product is not found
Else shows success message
Also prints message in console
*/
private void btnSearchActionPerformed(java.awt.event.ActionEvent evt) {
    MergeSort selectionSort = new MergeSort();
    List<adminModel> sortedList = selectionSort.sortByName(productList, false);
    BinarySearch search = new BinarySearch();
    adminModel searchedData = search.searchByProductName(txtFldSearchBar.getText().trim().toLowerCase(), sortedList, 0, sortedList.size() - 1);
    if (searchedData != null) {
        System.out.println(searchedData.getProductName());
        JOptionPane.showMessageDialog(null, searchedData.getProductName() + " is found.", "Item Found", JOptionPane.INFORMATION_MESSAGE);
    } else {
        System.out.println("Product not found.");
        JOptionPane.showMessageDialog(null, "Item not found. Please check the table or try changing keywords.", "Product Not Found", JOptionPane
    }
}
```

Figure 63: btnSearchActionPerformed()

This method is used to implement binary search algorithm. This method uses instance of merge sort class to locate the item based on its name, and uses binary search algorithm to implement search.

4. Testing

4.1 Test Planning

Table 1: Planning test cases

S. N	Test Name	Test Description
1	Progress Bar	The loading progress bar is designed to keep user engaged until and unless the application is entirely loaded in the background.
2	Login Validation	The login page has empty fields validation, wrong credentials validation and correct credentials validation.
3	Product Information	The admin page has text fields for entering product information which is validated with empty fields, integer manipulation, and correct information entered validation.
4	Submit Data	The submit button validates if the data is already entered in the product table or not. It uses product ID to check repeated product information and after validation is completed, the data is shown in the table.
5	Update Data	The update data button is used to change the already existing product information in the dashboard.
6	Clear Data	The clear data button is used to clear all the data entered in the text fields. It makes it easy for the user to add new data rather than removing data individually from the text fields.
7	Delete Record	The delete record button is used to delete the data existing in the table.
8	Sort by Quantity	This button is used to sort the items by their quantity number in descending order using insertion sort.
9	Sort by price	This button is used to sort the items by their price high to low using selection sort.

10	Sort by name	This button is used to sort the items using merge sort in descending alphabetical order.
11	Search Button	Search button is used to search for a specific item in a table based on its name using merge sort.

4.2 Test Cases

4.2.1 Progress Bar

Table 2: Test 1: Progress Bar

Test Name	Test Description	Expected Result	Actual Result	Remarks
Progress Bar	The loading progress bar is designed to keep user engaged until and unless the application is entirely loaded in the background.	The progress bar should show until the system in the background is loaded fully and dispose.	The progress bar is seen until the program is fully loaded in the background and then vanishes.	The test was successful.



Figure 64: Progress Bar test

4.2.2 Login Validation

Table 3: Test 2: Login validation

Test Name	Test Description	Expected Result	Actual Result	Remarks
Login Validation	The login page has empty fields validation, wrong credentials validation and correct credentials validation.	The login page should show different validation messages for different cases.	The login page shows different validation messages for different cases.	The test was successful.

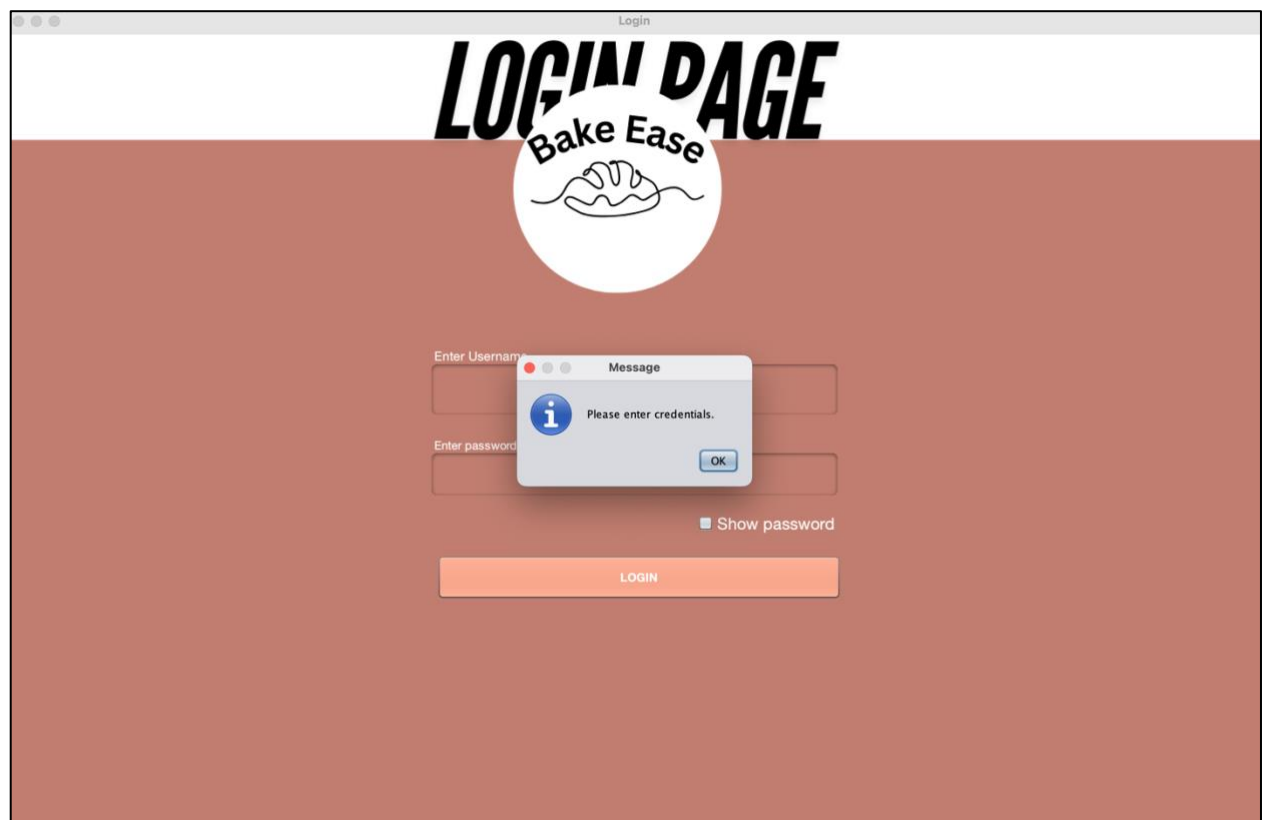


Figure 65: Empty fields validation

Here, the system default username is set to “admin” and password is set to “admin”.
So when wrong credentials are entered, following message is shown.

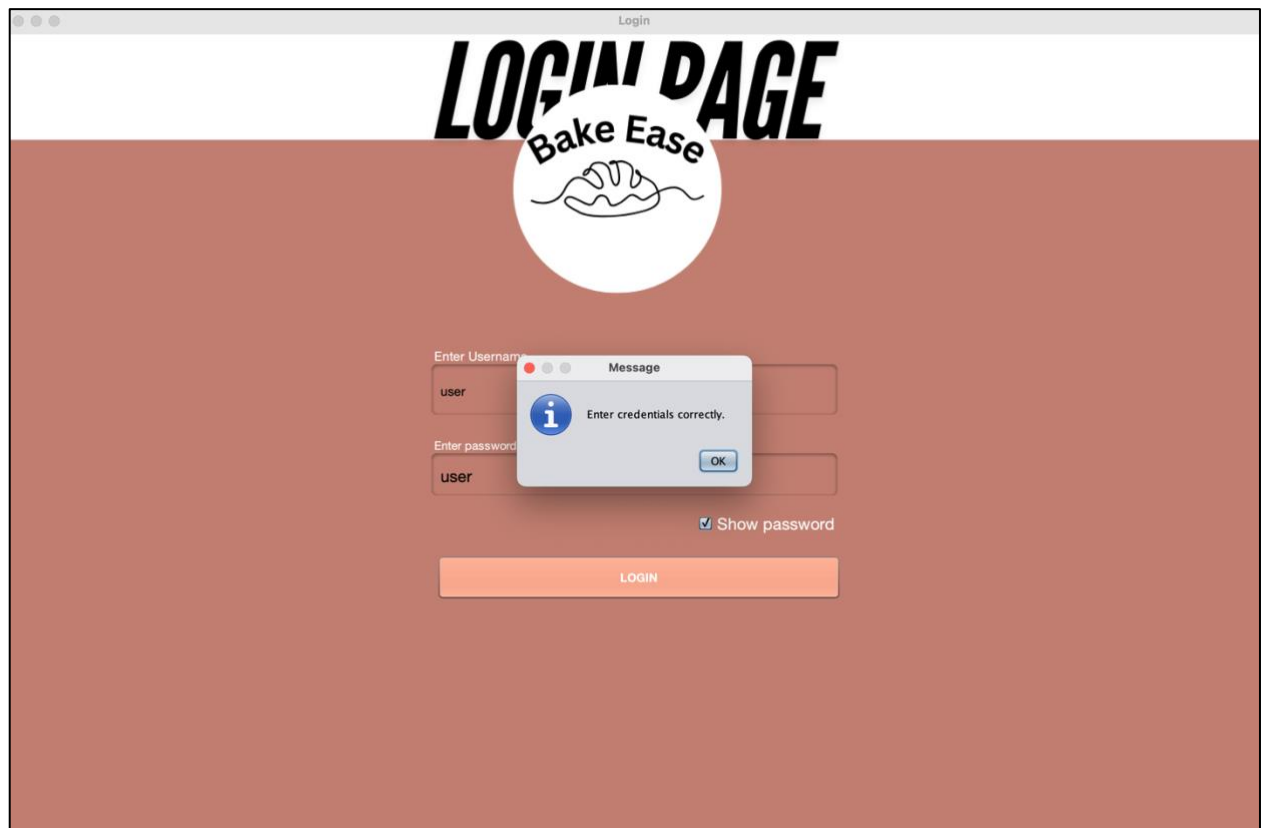


Figure 66: incorrect details validation

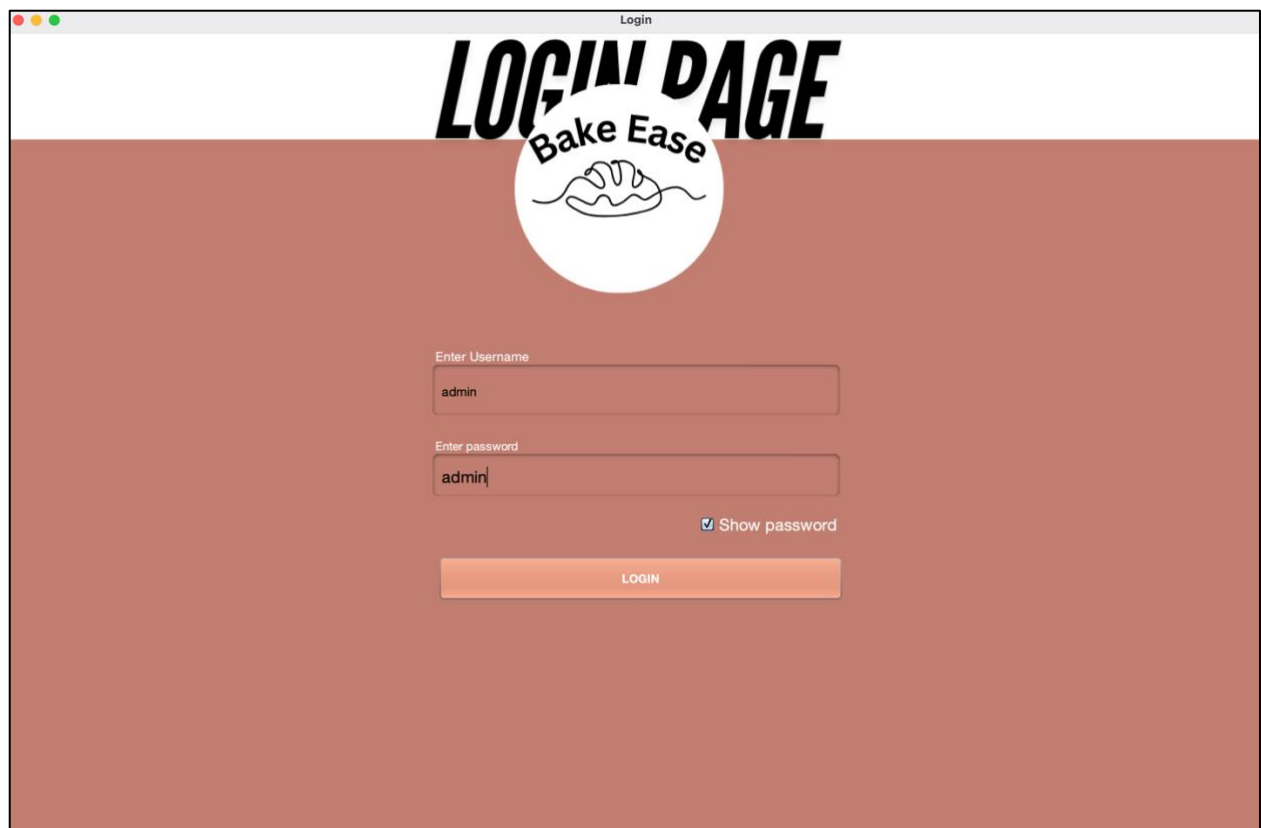


Figure 67: Providing correct credentials

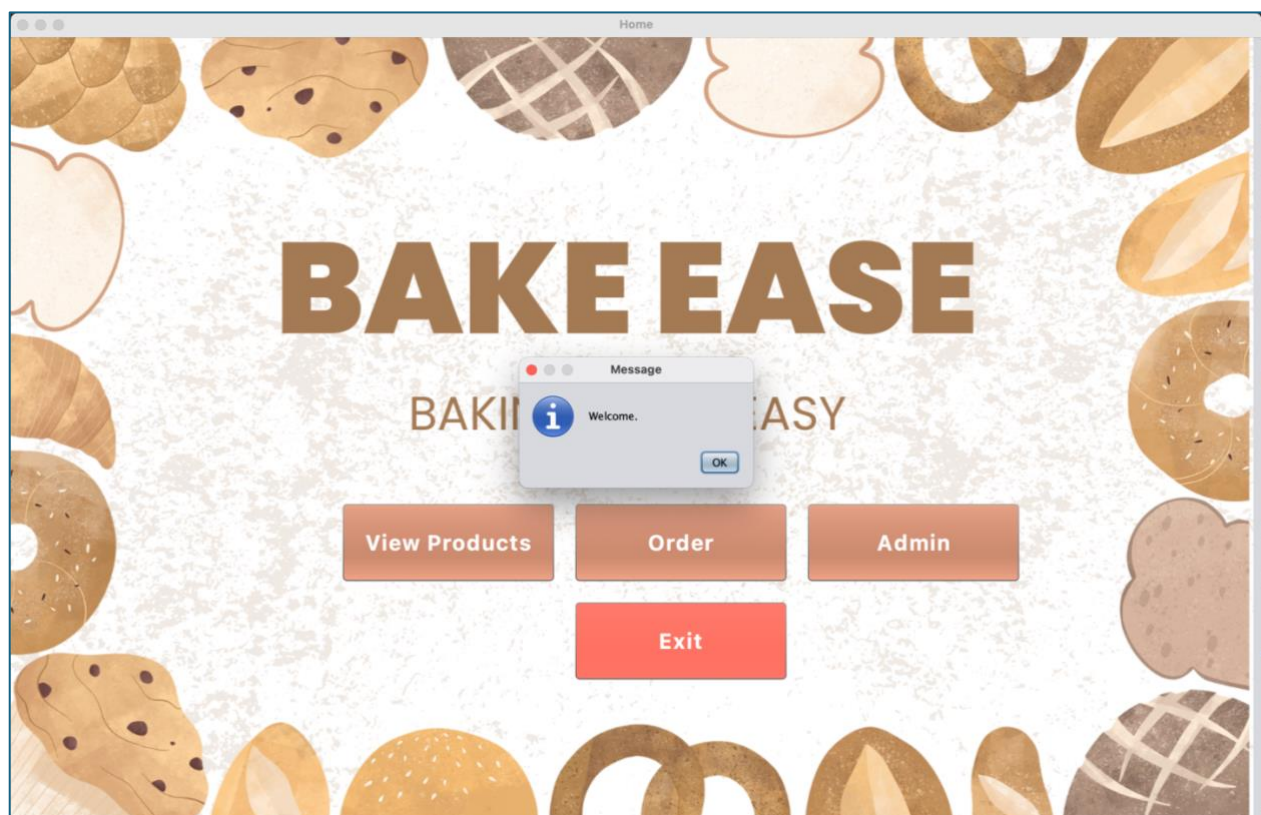


Figure 68: after providing correct credentials

4.2.3 Product Information

Table 4: Test 3: Product Information

Test Name	Test Description	Expected Result	Actual Result	Remarks
Product Information	The admin page has text fields for entering product information which is validated with empty fields, integer manipulation, and correct information entered validation.	The price, quantity, and product Id text fields are validated. Hence the system should have empty fields validation as well as integer manipulation validation after submit button is clicked.	The system shows empty fields validation as well as integer manipulation validation.	The test was successful.

After the submit button is clicked the action listener method is triggered which carries out the validation for the text fields. Further that button is also used to add data to admin dashboard.

```

private void btnSubmitActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (txtFldProductId.getText().equals("") || txtFldProductName.getText().equals("") || txtFldQuantity.getText().equals("") || txtFldPrice.getText().equals("")) {
        JOptionPane.showMessageDialog(null, "Empty Fields Detected!", "Error", JOptionPane.ERROR_MESSAGE);
    } else {
        try {
            // parsing the supposed strings values into integer
            int quantityInt = Integer.parseInt(txtFldQuantity.getText());
            int priceInt = Integer.parseInt(txtFldPrice.getText());

            // Get input values
            String productName = txtFldProductName.getText();
            int quantity = Integer.parseInt(txtFldQuantity.getText());
            int price = Integer.parseInt(txtFldPrice.getText());
            int productId = Integer.parseInt(txtFldProductId.getText());

            // stating a valid condition as TRUE initially
            boolean isCorrect = true;

            if (isCorrect) {
                // creating object of AdminModel class to pass values in parameters
                adminModel newBakeryItem = new adminModel(productId, productName, quantity, price);
                if (checkDuplicateProduct(newBakeryItem)) {
                    JOptionPane.showMessageDialog(null, "Same product already exists. \nPlease change the product", "Error", JOptionPane.ERROR_MESSAGE);
                } else {
                    addBakeryItems(newBakeryItem);
                    JOptionPane.showMessageDialog(null, "Product added succesfully.", "Success", JOptionPane.INFORMATION_MESSAGE);
                }
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Product Id, Quantity and Price must be in numeric form!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

Figure 69: method to validate product information

The screenshot shows a web application interface for managing products. On the left, the 'Enter Details' panel contains four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right, the 'Admin Dashboard' panel features a table with the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange). An 'Error' dialog box is overlaid on the table, displaying a red exclamation mark icon and the message 'Empty Fields Detected!' with an 'OK' button.

Figure 70: product information empty fields validation

The Admin Dashboard interface includes a search bar at the top. On the left, the 'Enter Details' section has four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. All fields contain the text 'user'. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right, the 'Admin Dashboard' section features a table with the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
		12	300
		10	200
	ke	14	69
		7	690
	at bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Below the table are four buttons: 'Delete Record', 'Sort by Quantity', 'Sort by Name', and 'Sort by Price'. An error dialog box is displayed in the center, stating: 'Error: Product Id, Quantity and Price must be in numeric form!'. The dialog has a red exclamation mark icon and an 'OK' button.

Figure 71: Integer manipulation validation

The Admin Dashboard interface is the same as in Figure 71, but the input fields in the 'Enter Details' section now contain the value '45'. The 'Submit' button is highlighted in green. A success dialog box is displayed in the center, stating: 'Success: Product added succesfully.'. The dialog has a blue information icon and an 'OK' button. The table in the 'Admin Dashboard' section now includes an additional row at the bottom:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
45	user	45	45

Figure 72: product added successfully

We can see that the product is added when string and integer manipulation is carried out properly.

4.2.4 Submit Data

Table 5: Test 4: Submit data

Test Name	Test Description	Expected Result	Actual Result	Remarks
Submit Data	The submit button validates if the data is already entered in the product table or not. It uses product ID to check repeated product information and after validation is completed, the data is shown in the table.	Respective messages should be shown for every different case which includes empty fields, integer manipulation, successful message and repeated data.	The system shows respective message for every different case including empty fields, integer manipulation, successful message and repeated data.	The test was successful.

The screenshot shows a web application interface for an admin dashboard. On the left, there is a form titled 'Enter Details' with four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right, there is a section titled 'Admin Dashboard' containing a table with the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange). An 'Error' dialog box is overlaid on the dashboard, displaying a red exclamation mark icon and the message 'Empty Fields Detected!' with an 'OK' button.

Figure 73: Empty fields validation for submit data

The Admin Dashboard interface includes a search bar at the top. On the left, the 'Enter Details' section has four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Each field contains the placeholder text 'user'. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right, the 'Admin Dashboard' section features a table with the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
		12	300
		10	200
	ke	14	69
		7	690
	at bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Below the table are four buttons: 'Delete Record', 'Sort by Quantity', 'Sort by Name', and 'Sort by Price'. An error dialog box is displayed in the center, with a red exclamation mark icon and the text: 'Error Product Id, Quantity and Price must be in numeric form!'. The dialog has an 'OK' button.

Figure 74: Integer manipulation validation for submit data

The Admin Dashboard interface is the same as in Figure 74, but the input fields in the 'Enter Details' section now contain the values: '66' for Product ID, 'banana' for Product Name, '56' for Quantity, and '56' for Price. A success dialog box is displayed in the center, with a blue information icon and the text: 'Success Product added succesfully.'. The dialog has an 'OK' button. The table in the 'Admin Dashboard' section now includes an additional row:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	banana	56	56

Figure 75: Product succesfully added message

Since the same product is already added, when we click the submit button again, it shows message that says same product already exists.

For this to happen, a method has been created in the program that checks repeated data in the table.

```
// method to check duplicate data in the data
private boolean checkDuplicateProduct(adminModel bakeryItems) {
    for (adminModel existingBakeryItem : productList) {
        if (existingBakeryItem.getProductid() == bakeryItems.getProductid()) {
            return true;
        }
    }
    return false;
}
```

Figure 76: Checking duplicate data

The screenshot shows an Admin Dashboard with two main sections: 'Enter Details' and 'Admin Dashboard'.

Enter Details: This section contains four input fields: 'Enter product ID:' (66), 'Enter product Name:' (banana), 'Enter Quantity:' (56), and 'Enter Price:' (56). Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red).

Admin Dashboard: This section features a table with the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
		12	300
		10	200
		14	69
		7	690
		4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	banana	56	56

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange).

A modal dialog box is overlaid on the dashboard with the following text: 'Same products found. Same product already exists. Please change the product information or try updating product information.' and an 'OK' button.

Figure 77: Same product validation

4.2.5 Update Data

Table 6: Test 5: Update data

Test Name	Test Description	Expected Result	Actual Result	Remarks
Update Data	The update data button is used to change the already existing product information in the dashboard.	The button should change the current existing product information.	The button successfully updated the already existing product information.	The test was successful.

The screenshot shows an 'Admin' interface with a search bar at the top. On the left, the 'Enter Details' form contains the following data:

- Enter product ID: 66
- Enter product Name: banana
- Enter Quantity: 56
- Enter Price: 56

Buttons at the bottom of the form are 'Submit' (green), 'Update' (blue), and 'Clear' (red). On the right, the 'Admin Dashboard' table displays the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	banana	56	56

The row for Product ID 66 is highlighted with a red border. Below the table are buttons for 'Delete Record', 'Sort by Quantity', 'Sort by Name', and 'Sort by Price'.

Figure 78: Targeted Product

As we can see that the product information of the targeted product is changed as user changed the data.

The screenshot shows an 'Admin' interface with a top navigation bar containing a search bar and a refresh button. The main content is divided into two panels. The left panel, titled 'Enter Details', contains four input fields: 'Enter product ID:' (66), 'Enter product Name:' (Apple), 'Enter Quantity:' (40), and 'Enter Price:' (40). Below these are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). The right panel, titled 'Admin Dashboard', displays a table of products. A modal dialog box is open over the table, showing a success message: 'Product information updated successfully.' with an 'OK' button. The table has columns: Product ID, Name, Quantity, and Price. The row for Product ID 66 (Apple) is highlighted with a red border. Below the table are four buttons: 'Delete Record', 'Sort by Quantity', 'Sort by Name', and 'Sort by Price'.

Product ID	Name	Quantity	Price
1	Crossiant	22	420
		12	300
	ry	10	200
	ese Cake	14	69
	iballs	7	690
	le wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	Apple	40	40

Figure 79: Updated data of targeted product

4.2.6 Clear Data

Table 7: Test 6: Clear data

Test Name	Test Description	Expected Result	Actual Result	Remarks
Clear Data	The clear data button is used to clear all the data entered in the text fields. It makes it easy for the user to add new data rather than removing data individually from the text fields.	The user should be given two options that asks if the user really wants to clear all the text fields. If the user chooses yes then all the text fields should clear, else nothing should be cleared.	The button clears all the data in the text fields if the user chooses “yes” else nothing is cleared.	The test was successful.

Admin

Search Product

Enter Details

Enter product ID: 66

Enter product Name: Apple

Enter Quantity: 40

Enter Price: 40

Submit Update Clear

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
		12	300
		10	200
	Cake	14	69
	s	7	690
	heat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	Apple	40	40

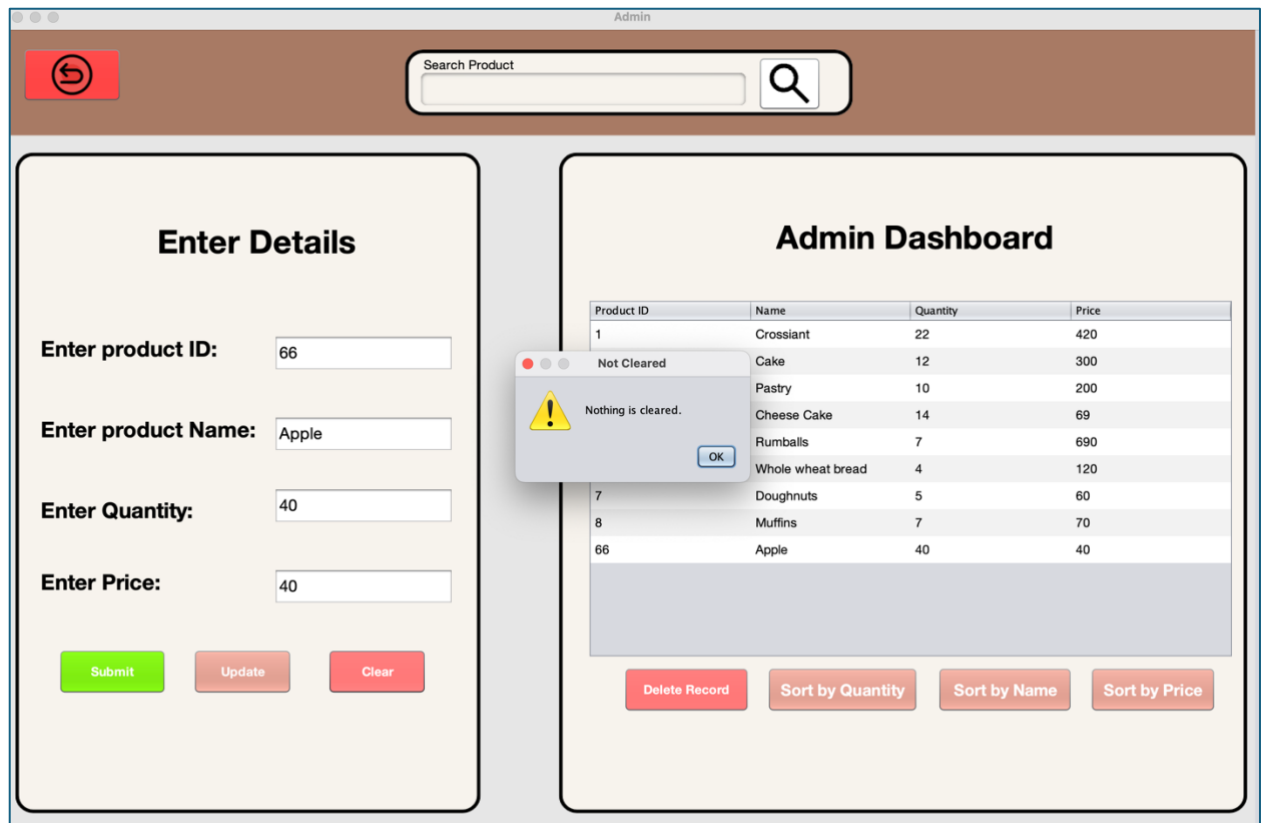
Delete Record Sort by Quantity Sort by Name Sort by Price

Confirm Clear

Are you sure you want to clear all the text fields ?

No Yes

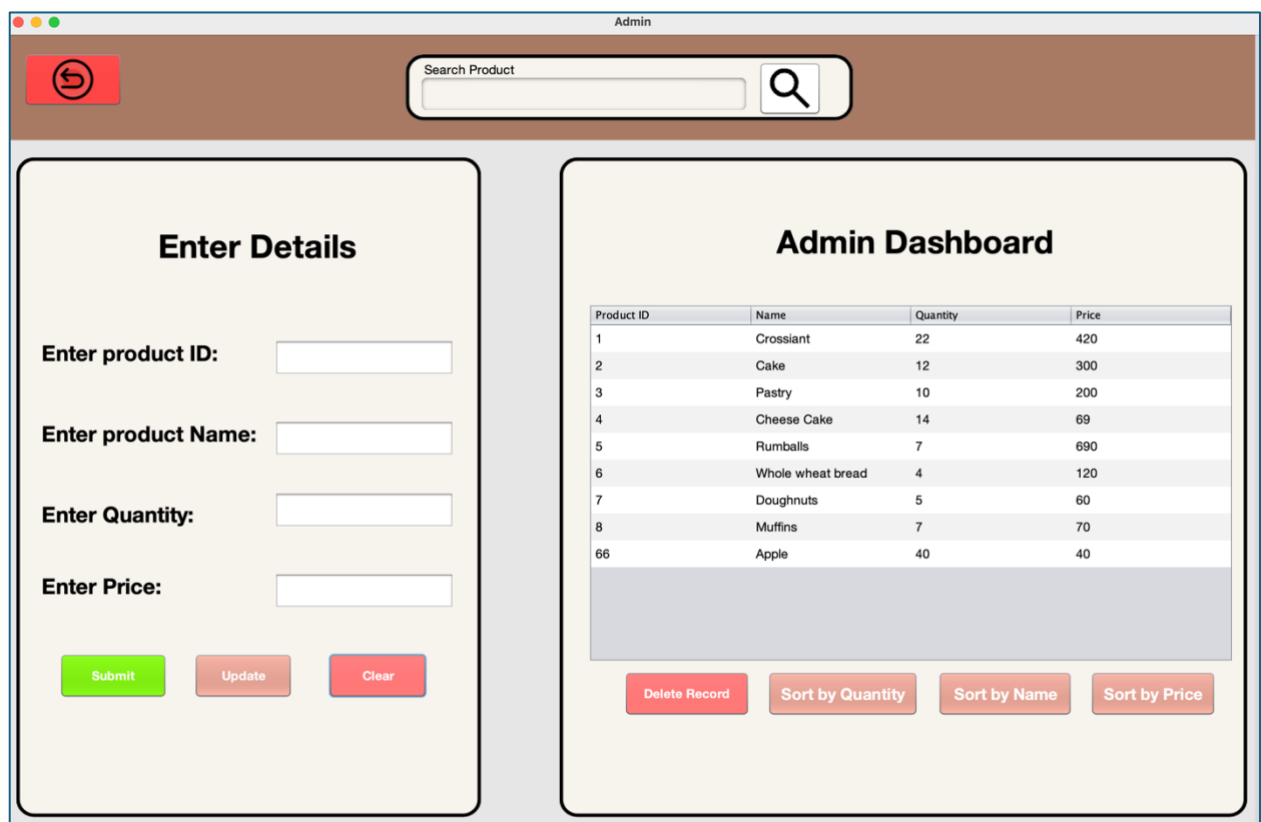
Figure 80: Asking user option



The screenshot shows the Admin Dashboard with a modal dialog box titled "Not Cleared" displaying a warning icon and the message "Nothing is cleared." with an "OK" button. The "Enter Details" form on the left has the following values: Product ID: 66, Product Name: Apple, Quantity: 40, Price: 40. The "Admin Dashboard" table on the right lists products with their IDs, names, quantities, and prices.

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	Apple	40	40

Figure 81: After choosing NO



The screenshot shows the Admin Dashboard after choosing YES. The "Enter Details" form on the left is empty. The "Admin Dashboard" table on the right lists products with their IDs, names, quantities, and prices.

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	Apple	40	40

Figure 82: After choosing YES

4.2.7 Delete Record

Table 8: Test 7: Delete record

Test Name	Test Description	Expected Result	Actual Result	Remarks
Delete Record	The delete record button is used to delete the data existing in the table.	The button should remove the item from the table after the targeted element's table row is selected.	The button successfully removes the data from the table row.	The test was successful.

The screenshot shows an Admin Dashboard with a table of products. The row for Product ID 66 (banana) is highlighted in red. The dashboard includes a search bar, an 'Enter Details' form, and buttons for 'Delete Record', 'Sort by Quantity', 'Sort by Name', and 'Sort by Price'.

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
66	banana	56	56

Figure 83: Targeted element to be deleted

To delete an item, the user should select the row of the targeted element and then only the product is able to be deleted. If the row is not selected, the data cannot be deleted.

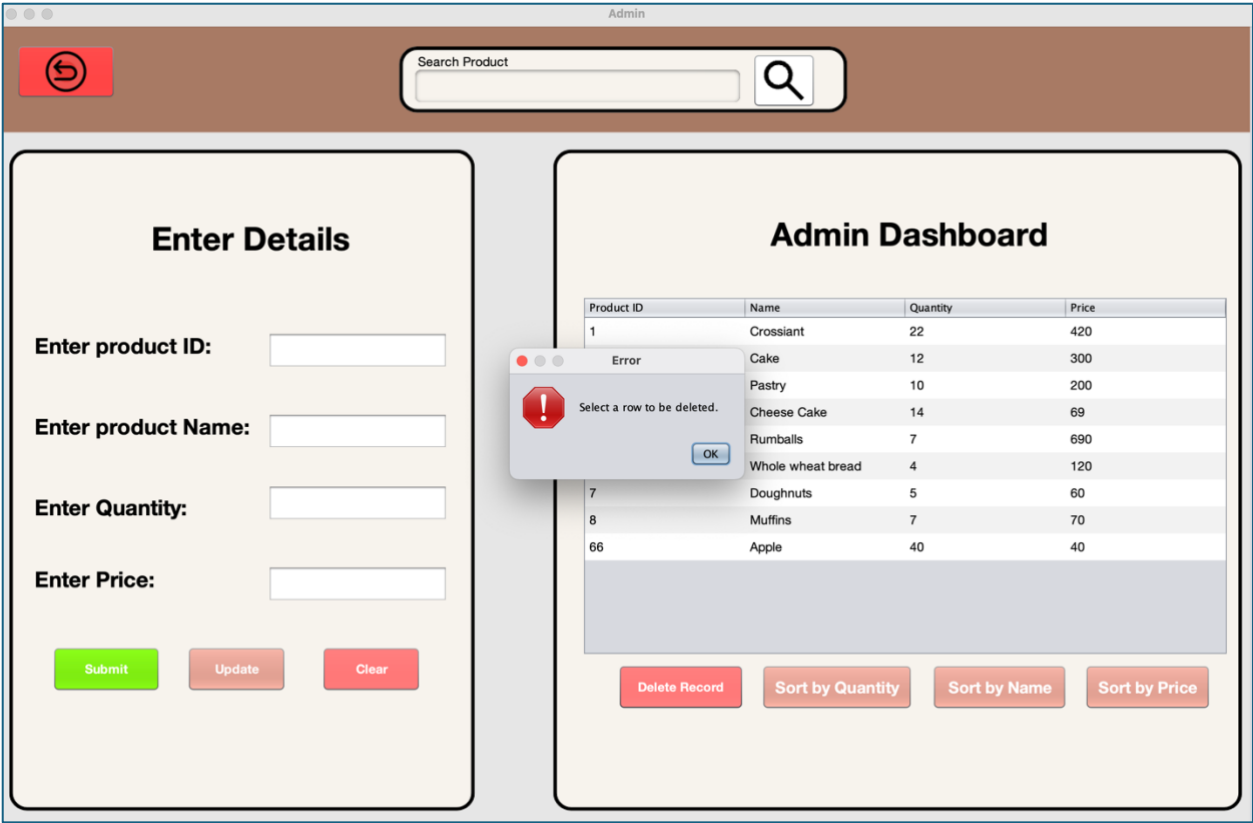


Figure 84: Not selecting table row while deleting data

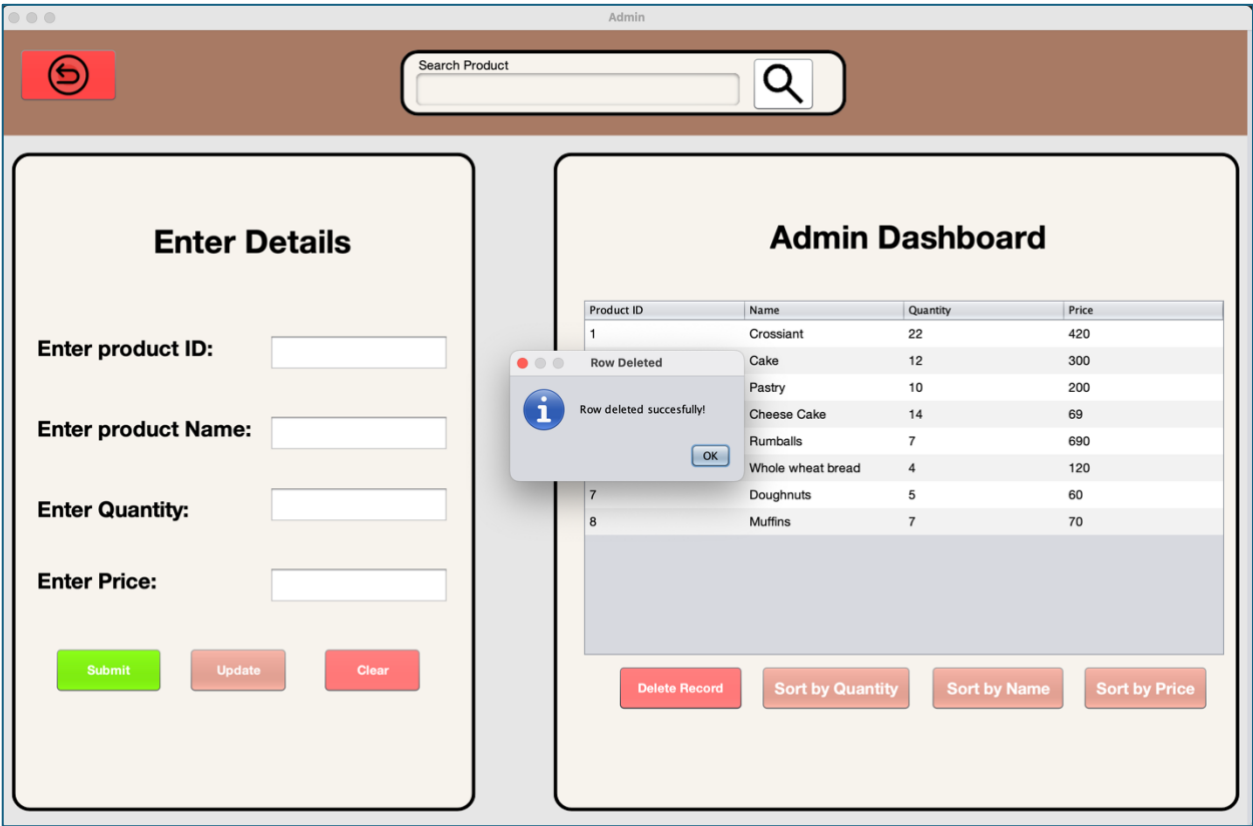


Figure 85: After selecting table row of targeted element

4.2.8 Sort by Quantity

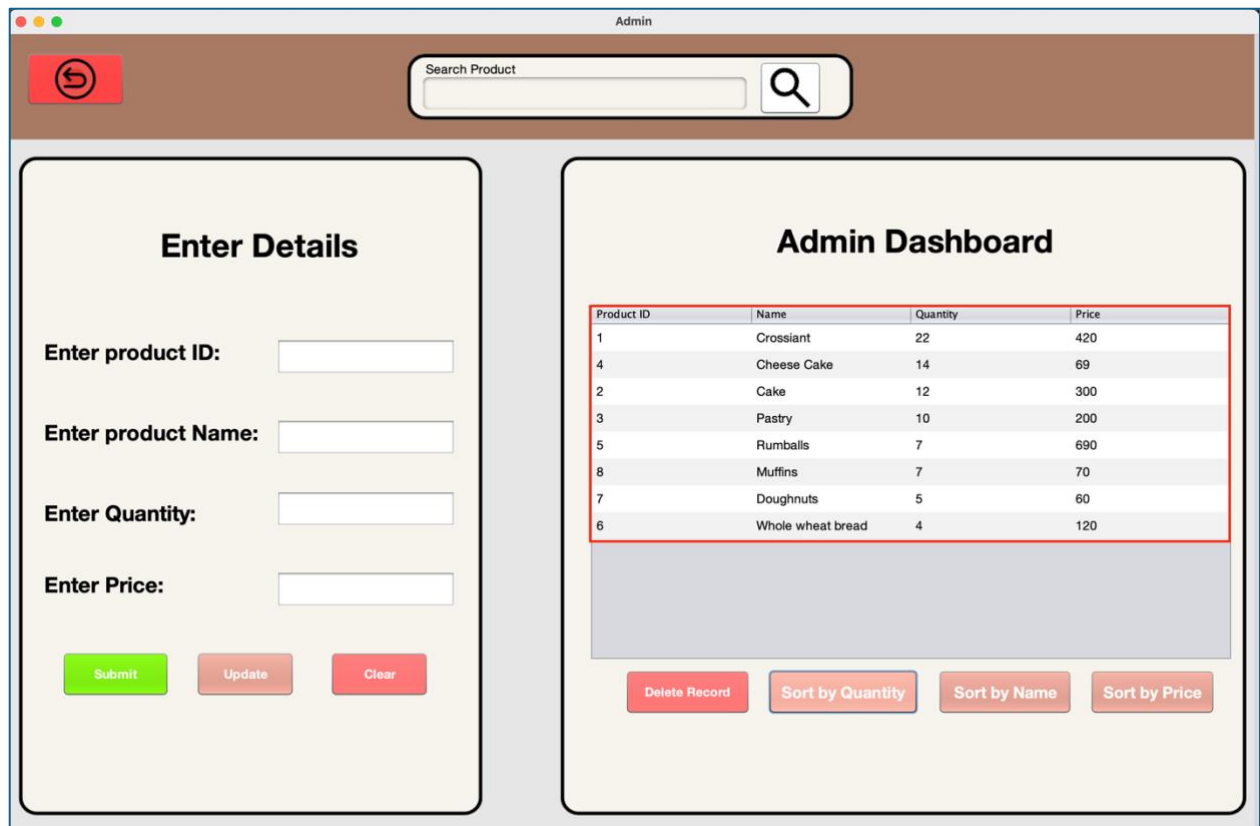
Table 9: Test 8: sort by quantity

Test Name	Test Description	Expected Result	Actual Result	Remarks
Sort by Quantity	This button is used to sort the items by their quantity number in descending order using insertion sort.	The button should sort the item based on its quantity from high to low.	The button sorts the item based on its quantity from high to low.	The test was successful.

The screenshot shows the Admin Dashboard with a search bar at the top. On the left is the 'Enter Details' form with fields for Product ID, Name, Quantity, and Price, and buttons for Submit, Update, and Clear. On the right is the 'Admin Dashboard' section containing a table of products and buttons for 'Delete Record', 'Sort by Quantity', 'Sort by Name', and 'Sort by Price'. The table data is as follows:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Figure 86: before sorting by quantity



The screenshot shows an Admin Dashboard with a search bar at the top. On the left, there is a form titled 'Enter Details' with fields for 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right, there is a table titled 'Admin Dashboard' showing product data. The table has columns for Product ID, Name, Quantity, and Price. The data is sorted by quantity in descending order. Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange).

Product ID	Name	Quantity	Price
1	Crossiant	22	420
4	Cheese Cake	14	69
2	Cake	12	300
3	Pastry	10	200
5	Rumballs	7	690
8	Muffins	7	70
7	Doughnuts	5	60
6	Whole wheat bread	4	120

Figure 87: after sorting by quantity

To sort by quantity, we have used insertion sort. It is an sorting algorithm that builds the final sorted array one at a time by repeatedly picking next element and placing in its correct position.

```

/*
insertion sort for sort by quantity
@param adminModel for model
creates object sortQuantity to handle sorting
Adds data to list
*/
private void btnSortQuantityActionPerformed(java.awt.event.ActionEvent evt) {
    // Create an instance of the InsertionSort class
    InsertionSort sortQuantity = new InsertionSort();

    // Ensure productList is properly populated with product data
    // Sort the product list by quantity (descending order in this case)
    List<adminModel> sortedList = sortQuantity.sortByQuantity(productList, true);

    // Load the sorted data into the table
    loadSortedProductData(sortedList);
}

```

Figure 88: implementing insertion sort in sort by quantity button

4.2.9 Sort by Price

Table 10: Test 9: Sort by price

Test Name	Test Description	Expected Result	Actual Result	Remarks
Sort by Price	This button is used to sort the items by their price high to low using selection sort.	The button should sort the item based on its price from high to low.	The button sorts the item based on its price from high to low.	The test was successful.

The screenshot shows an Admin Dashboard with a header bar containing a search bar and a sidebar with a navigation menu. The main content area is divided into two panels. The left panel, titled 'Enter Details', contains four input fields for 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:', each followed by a text input box. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). The right panel, titled 'Admin Dashboard', contains a table with the following data:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange).

Figure 89: before sorting by price

The screenshot shows an 'Admin' window with a search bar at the top. On the left, the 'Enter Details' section has four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right, the 'Admin Dashboard' section displays a table of products sorted by price. The table has columns for Product ID, Name, Quantity, and Price. Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange).

Product ID	Name	Quantity	Price
5	Rumballs	7	690
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
6	Whole wheat bread	4	120
8	Muffins	7	70
4	Cheese Cake	14	69
7	Doughnuts	5	60

Figure 90: after sorting by price

```

/*
method to execute selection sort for price
Arranges item price ranging from high to low
@param adminModel for model
creates a object sort and handles sorting
Adds data to list
*/
private void btnSortPriceActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    SelectionSort sort = new SelectionSort();
    List<adminModel> sortedList = sort.sortByPrice(productList, true);
    loadSortedProductData(sortedList);
}

```

Figure 91: imeplementing selection sort to sort by price

4.2.10 Sort by Name

Table 11: Test 10: Sort by name

Test Name	Test Description	Expected Result	Actual Result	Remarks
Sort by Name	This button is used to sort the items using merge sort in descending alphabetical order.	The button should sort items according to its name.	The button sorts items according to its name.	The test was successful.

The screenshot shows an 'Admin' interface with a top navigation bar containing a search bar and a search icon. Below the navigation bar, there are two main sections: 'Enter Details' on the left and 'Admin Dashboard' on the right.

Enter Details Section:

- Enter product ID:
- Enter product Name:
- Enter Quantity:
- Enter Price:
- Buttons: Submit (green), Update (orange), Clear (red)

Admin Dashboard Section:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Buttons below the table: Delete Record (red), Sort by Quantity (orange), Sort by Name (orange), Sort by Price (orange).

Figure 92: before sorting by name

The image shows a web application interface for an admin dashboard. It has a header bar with a search bar and a title 'Admin'. The main content is divided into two panels. The left panel, titled 'Enter Details', contains four input fields for 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). The right panel, titled 'Admin Dashboard', contains a table with the following data:

Product ID	Name	Quantity	Price
6	Whole wheat bread	4	120
5	Rumballs	7	690
3	Pastry	10	200
8	Muffins	7	70
7	Doughnuts	5	60
1	Crossiant	22	420
4	Cheese Cake	14	69
2	Cake	12	300

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange). The table is sorted by name, as indicated by the caption.

Figure 93: after sorting by name

```

/*
method to sort by name
@param adminModel for model
creates object merge to handle sorting
adds data to list
*/
private void btnSortNameActionPerformed(java.awt.event.ActionEvent evt) {
    MergeSort sort = new MergeSort();
    List<adminModel> merge = sort.sortByName(productList, true);
    loadSortedProductData(merge);
}

```

Figure 94: implementing merge sort to sort by name

4.2.11 Search Button

Table 12: Test 11: Search button

Test Name	Test Description	Expected Result	Actual Result	Remarks
Search Button	Search button is used to search for a specific item in a table based on its name using merge sort.	The button should display a message that says the searched product is found in the GUI as well as in terminal if found and should do the same If product not found.	The button displays a message that says the searched product is found in the GUI as well as in terminal if found and does the same If product not found.	The test was successful.

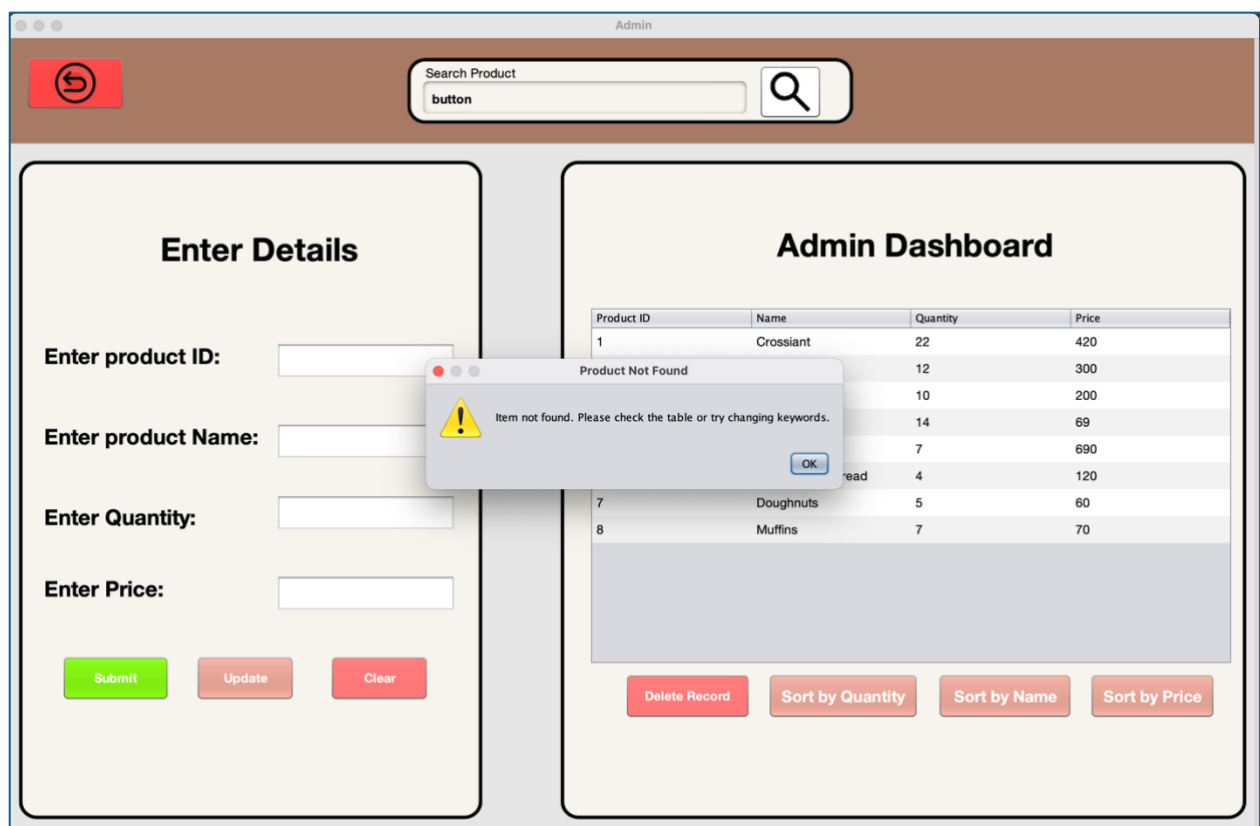


Figure 95: message showing product not found in GUI


```

Output - BakeEase (run)
run:
2025-01-17 23:57:12.698 java[20715:5558322] +[IMKClient subclass]: chose IMKClient_Modern
2025-01-17 23:57:12.698 java[20715:5558322] +[IMKInputSession subclass]: chose IMKInputSession_Modern
Product not found.

```

Figure 96: message showing product not found in terminal

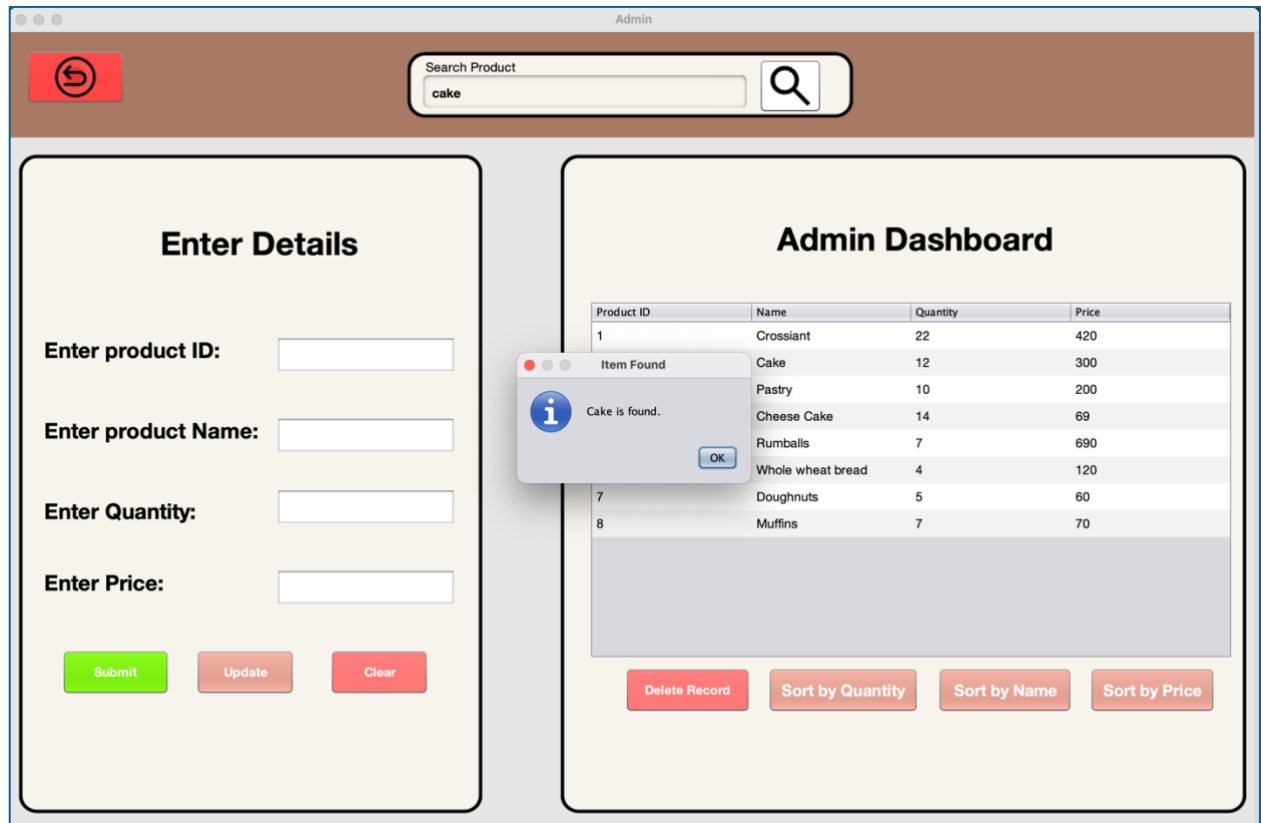


Figure 97: product found message in GUI

```

Output - BakeEase (run)
run:
2025-01-17 23:57:12.698 java[20715:5558322] +[IMKClient subclass]: chose IMKClient_Modern
2025-01-17 23:57:12.698 java[20715:5558322] +[IMKInputSession subclass]: chose IMKInputSession_Modern
Product not found.
Cake
|

```

Figure 98: name of founded product in terminal

5.Coursework Development

5.1 Canva

Canva is an online designing tool that is mostly free to use for everyone which is used to make designs, presentations or any poster graphics related work. It provides a wide variety of fonts, color options, templates, size options etc which makes it accesible for any kind of designing elements. In the context of this system, Canva is used to design all the wireframes and background images of the pages in the system. It makes brainstorming ideas easier which is why Canva is used here as designing tool.

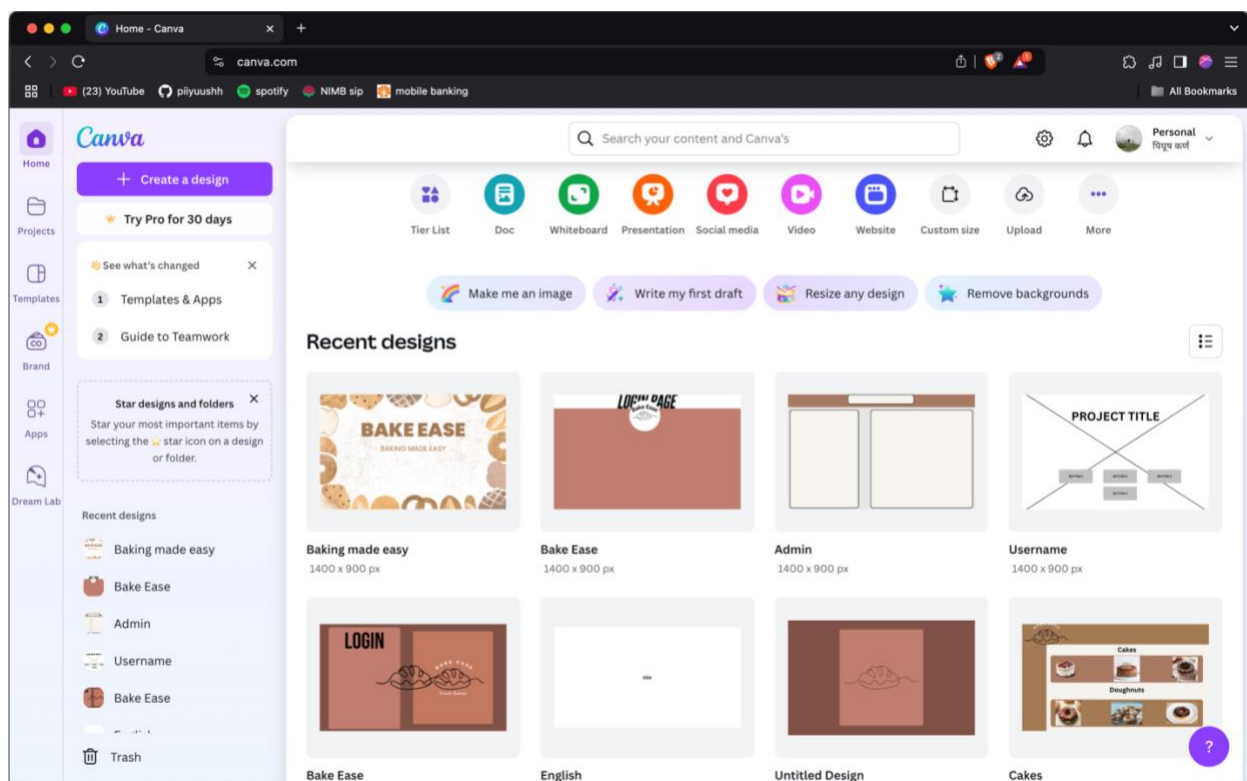


Figure 99: Canva as designing tool

There are many reasons for choosing Canva as designing tool. Some of them are listed below:

- **User friendly interface:** Canva provides user with user friendly interface which helps even a rookie designer to make his/her design much more efficiently.
- **Export options:** Canva provides user with different export options for his/her design such as pdf, jpg, jpeg, etc.

- **Time efficiency:** Canva is easier to user because of its user-friendly interface and consists of ready-made templates.
- **Cloud based:** Canva is cloud-based application which means that it doesn't use internal memory of the device to save user progress and doesn't require installation to access user content.
- **Wide range of templates and fonts:** Canva provides user with wide range of templates and fonts which gives user with large number of options to create the design of his own.
- **Cost effective:** Canva offers large number of free elements which are actually good to use and also provides affordable premium features.
- **Versatility:** Canva is a versatile tool which can be used to design wireframes, backgrounds, wallpapers, posters, social media banners etc. Hence a single tool can be used for multiple purpose.

5.1.1 Usage

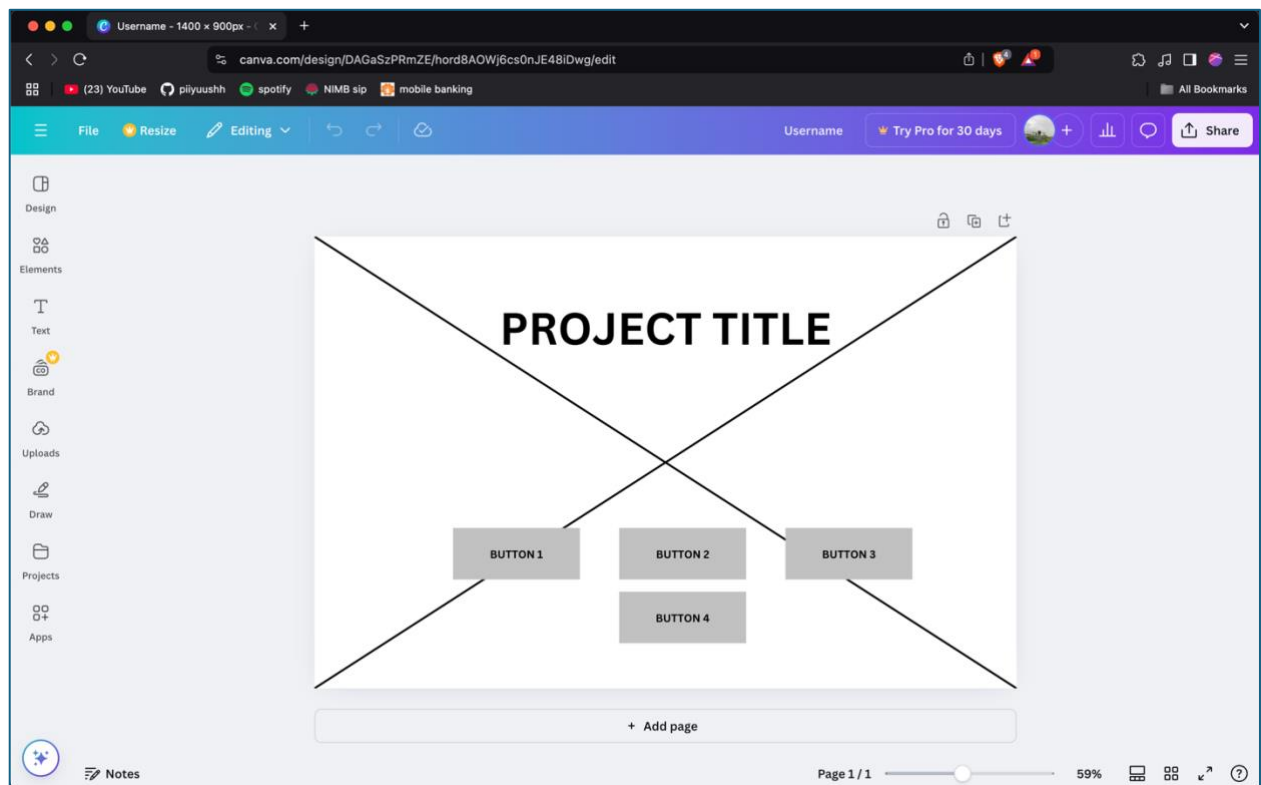


Figure 100: designing home page wireframe

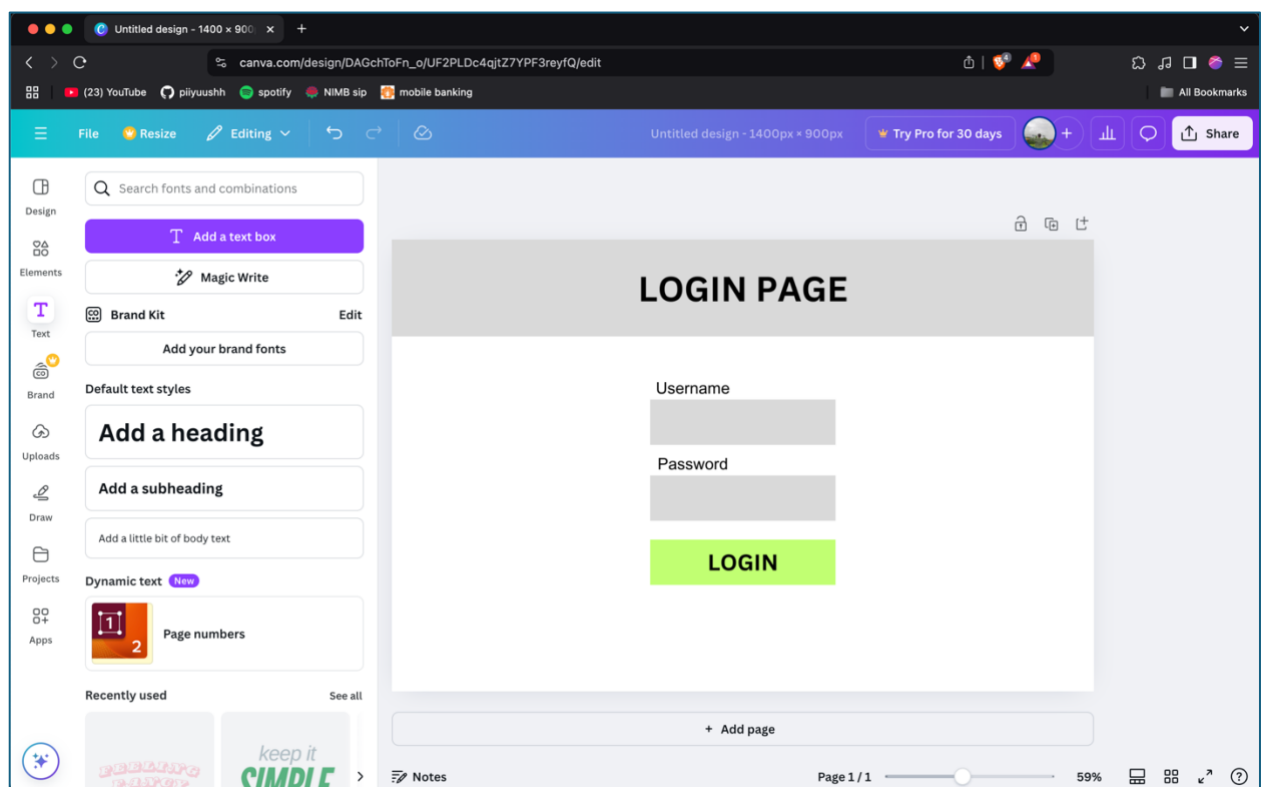


Figure 101: designing login page wireframe

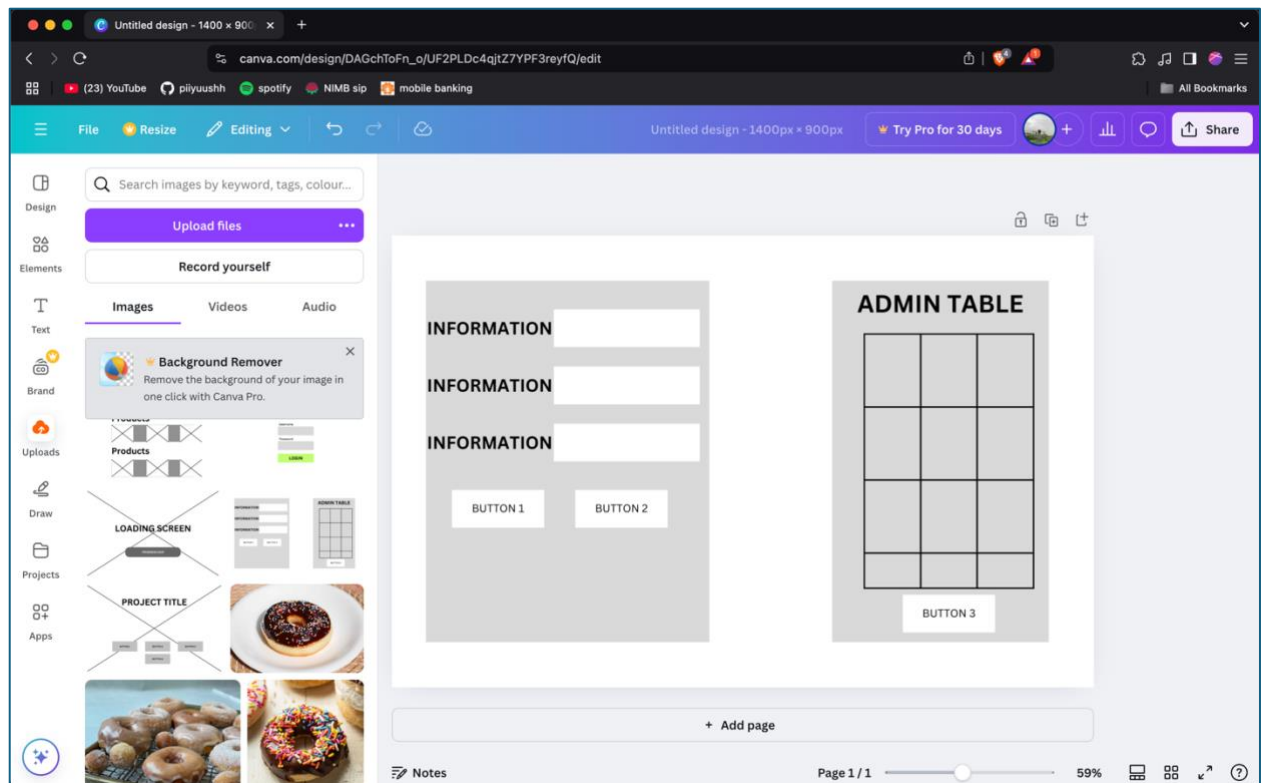


Figure 102: designing admin page wireframe

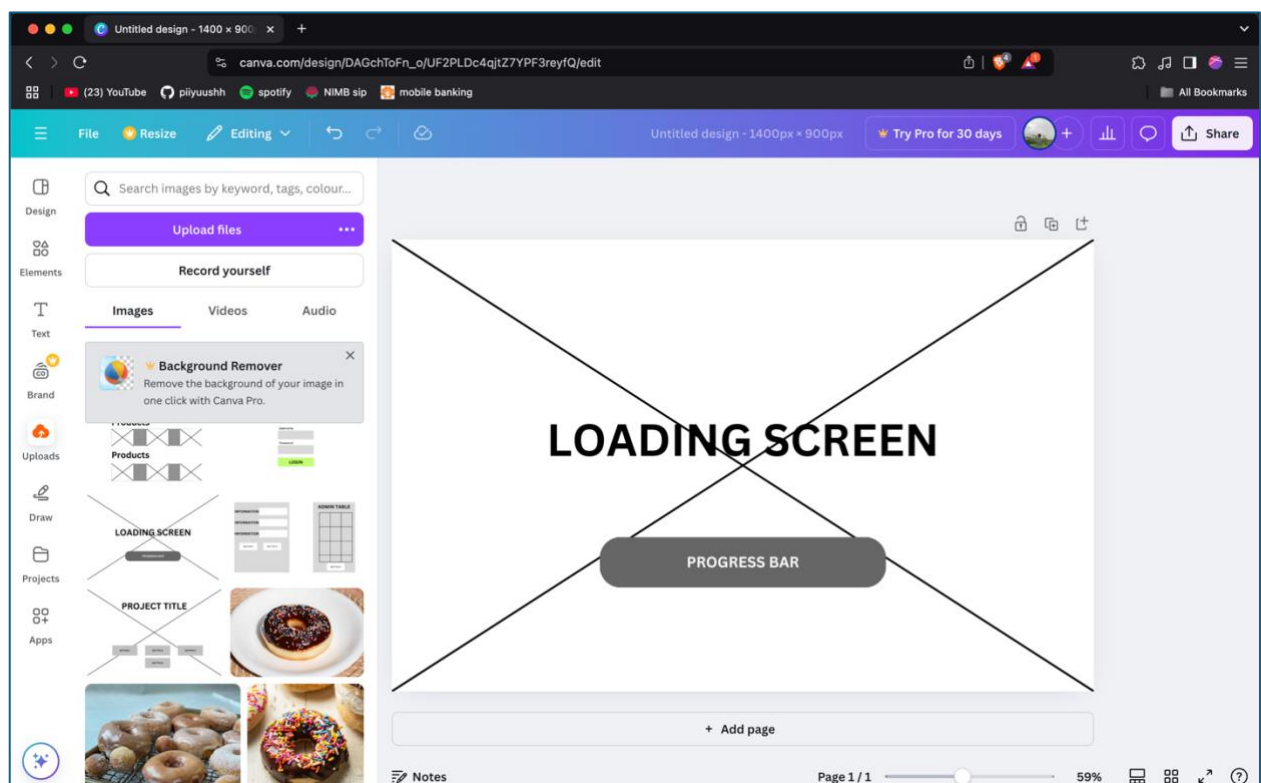


Figure 103: designing loading page wireframe

5.2 Apache Netbeans

The integrated development environment (IDE) used for developing this software is Apache Netbeans IDE. (Anon., n.d.) It is a powerful open-source IDE which is used to develop Java applications but also supports multiple languages like C, C++, PHP, JavaScript and python. It is widely used because of its multiple language support, allows user to install plugins, and many more similar features. It has some integrated development tools like GUI builder, debugger etc. It also supports version control for git/GitHub, and it works across all platforms such as windows, MacOS and Linux. This IDE is a great choice for developers seeking a free, versatile, and user-friendly IDE that balances simplicity with powerful development features.

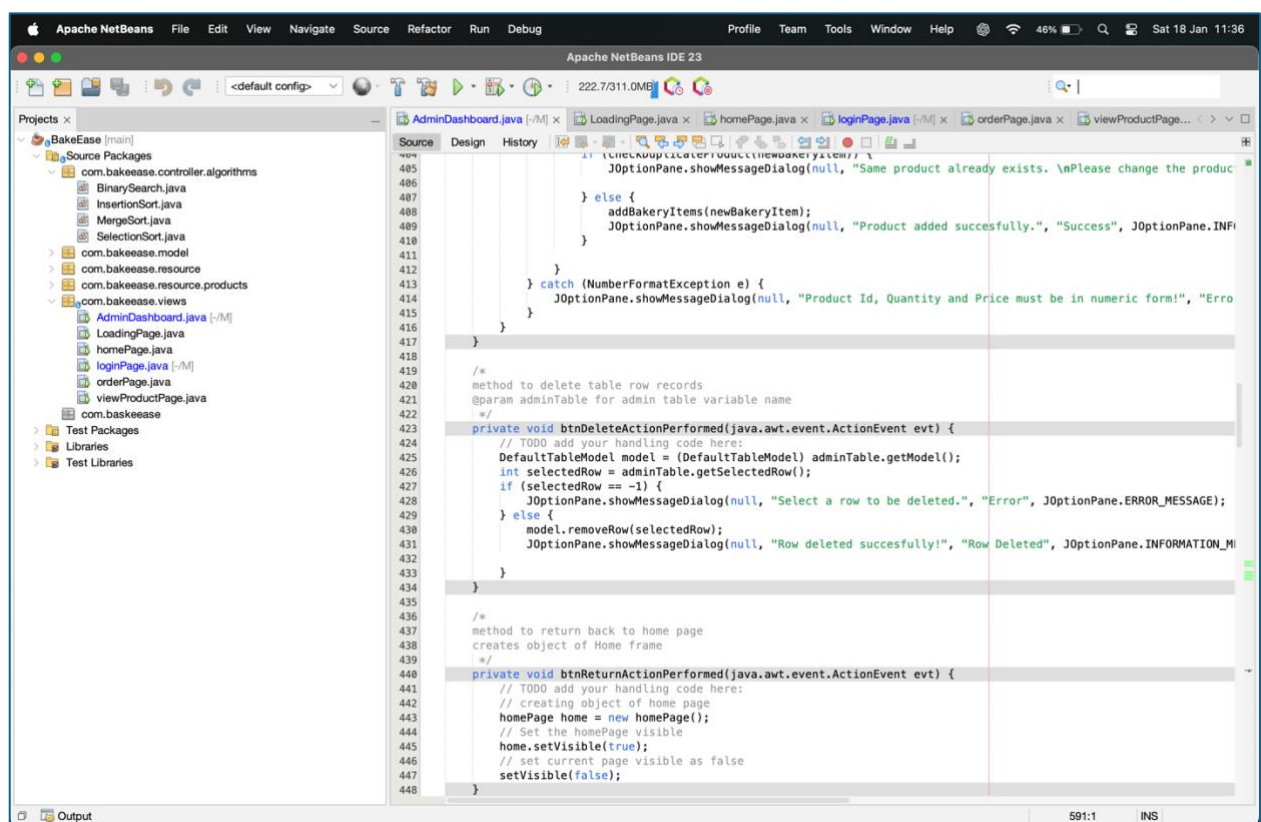


Figure 104: Using apache netbeans

The reasons for choosing Apache Netbeans as IDE for this project are:

- **Integrated GUI builder:** Netbeans contains integrated GUI builder elements which makes it easier to design and implement designs in applications. Also works great with packages.
- **Java Support:** Netbeans provides robust support for Java programming language which means it also contains integrated debugging tools for java.

- **Easy Usage:** Netbeans is user friendly as it has some basic features right implemented in graphical way which makes it easy for the user as the user doesn't need to move around to search for something that he/she requires.
- **Open Source:** Netbeans is built with open-source contribution which has many forum solution for any problem that user faces. Hence quick solution for every problem is provided. (Netbeans, n.d.)
- **Built in Version Control:** Netbeans provides built in version control feature which makes developers easier as they can publish their project right from the IDE itself in version control platforms like GitHub, GitLab etc.

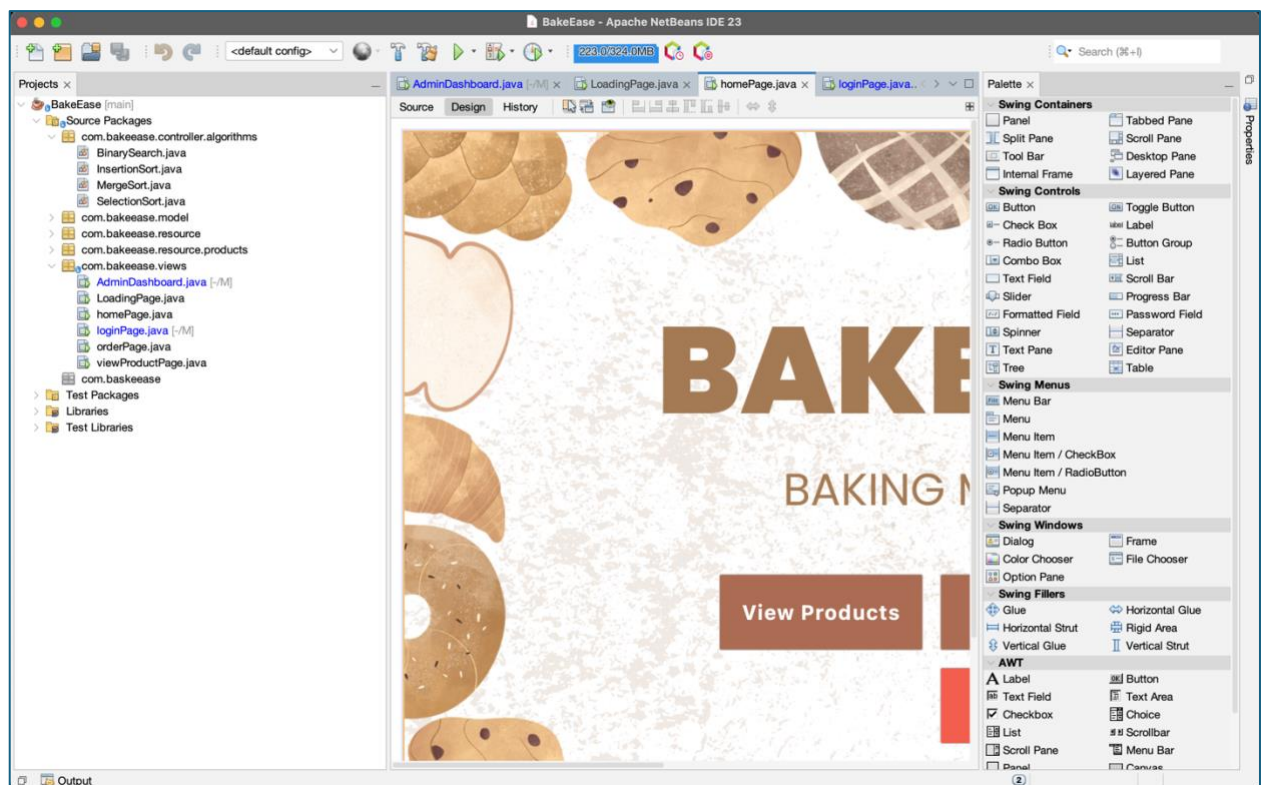


Figure 105: Integrated GUI elements in netbeans

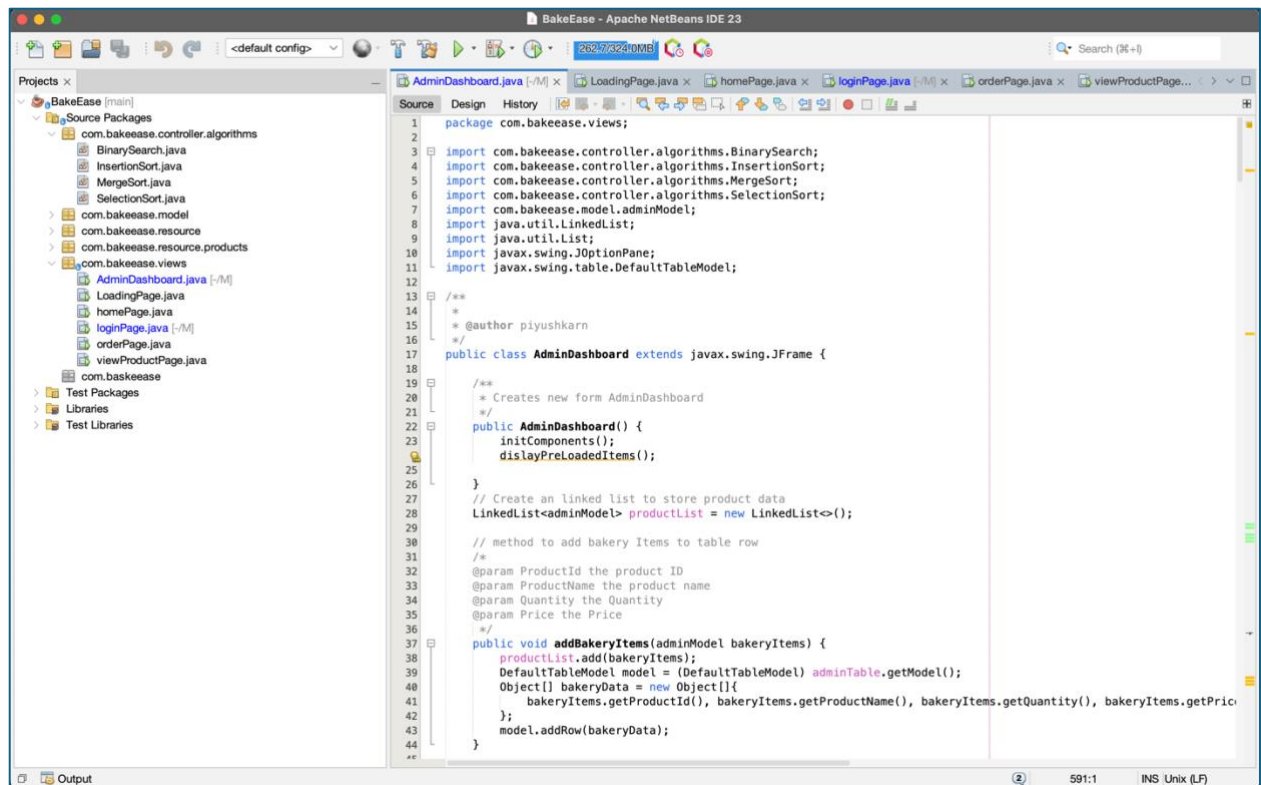


Figure 106: Java support in Netbeans

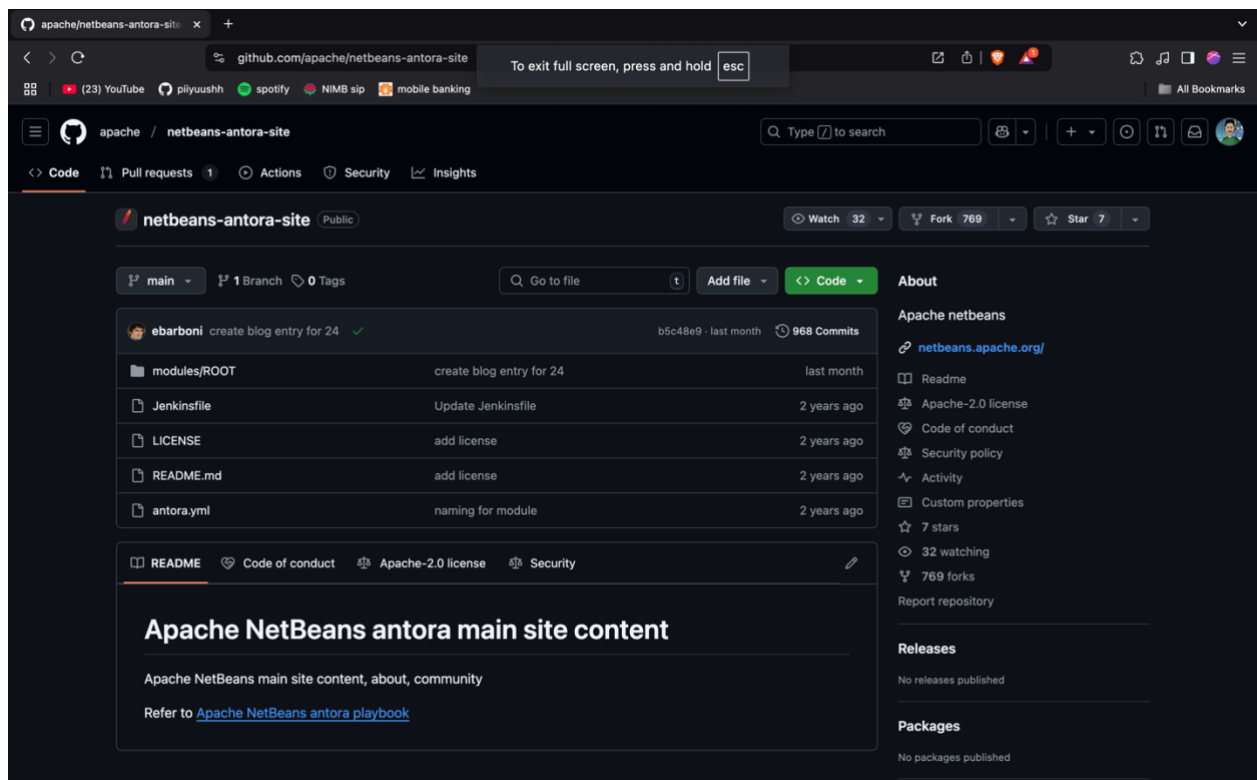


Figure 107: open source link of Netbeans

5.3 Java Programming

5.3.1 Project Development

Bake ease is an application software package designed for bakery management system which includes various features like product display, admin dashboard, order product, login system. The project has been developed using various data structures concepts like linked list, collection frameworks, sorting etc. This is a graphical user interface (GUI) based software which makes it user friendly. GUI elements like label, text fields, panel, frames, buttons, icons, tables, etc. have been used to make this application.

In this project, there are multiple GUI pages with their specific instruction and work to be performed. First, there is a **loading screen** which engages user until the program is fully loaded in the background. After the program is fully loaded, **login page** is shown. There the user is supposed to enter the username and password as credentials. The system has a username and password set by default which is “admin” and “admin”. The user can’t enter the system without entering any credentials and the user can’t enter the system by entering wrong credentials. Each of these cases are validated. Then after the user enters **home screen** the user can access three pages which are **view products**, **order page** and **admin page**. There is also an exit button which terminates the program. The **view products** page shows the available products in the bakery, the **order page** is yet to be built but the page will allow user to order the available products from the bakery and a payment method will also be set for that. The **admin page** allows user to add or delete products to the bakery and the entered data will be shown in a table. The admin page is also validated, and it won’t allow user to enter wrong data in the text fields. Until now the project is being developed at this stage, in future the project will be further developed by the help of individual and open-source contribution.

5.3.2 Implementation of MVC Architecture

MVC architecture stands for model-view-controller architecture is a widely used design pattern which separates application into three components which enhances modularity and makes the application easy to maintain. In the context of this project, the **Model** component helps to store the bakery items details. The **View** component helps to build the GUI interaction elements, and the **Controller** component helps establishing communication between the **View** and **Model** components. It makes sure that the changes done in one component makes impact to other component which ensures user experience.

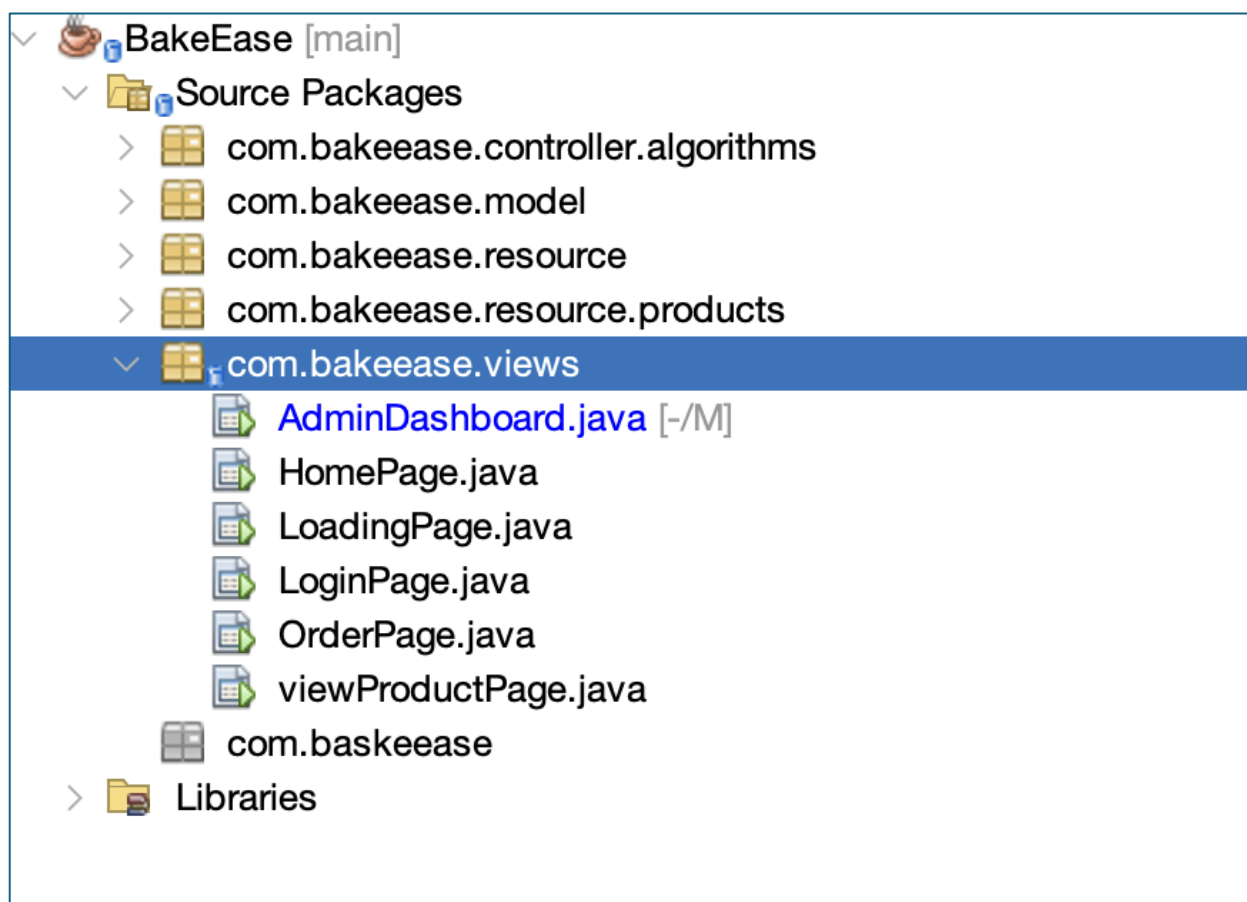


Figure 108: Implementing MVC architecture

5.4 Algorithms

Algorithms play a crucial role in determining functionality of a system. It is a step by step guide for solving a problem. Algorithms follow three logical structure: input, processing, and output. Algorithms are essentials for solving complex computational operations effectively. Types of algorithms used in the context of this project are: Searching algorithms, sorting algorithms and recursive algorithms. (Geeks, n.d.)

5.4.1 Insertion Sort

Insertion sort is a simple sorting algorithm that puts together the final sorted array element by element. It loops through an array and extracts one component from the beginning data to insert it into the right place at the already sorted section of the array. The concept behind insertion sort is to retain a sorted sublist in the lower positions of the array and, at the same time, move the new elements to their right place by inserting these into the sublist. (Javatpoint, n.d.)

5.4.1.1 Evidence of Citation Explanation

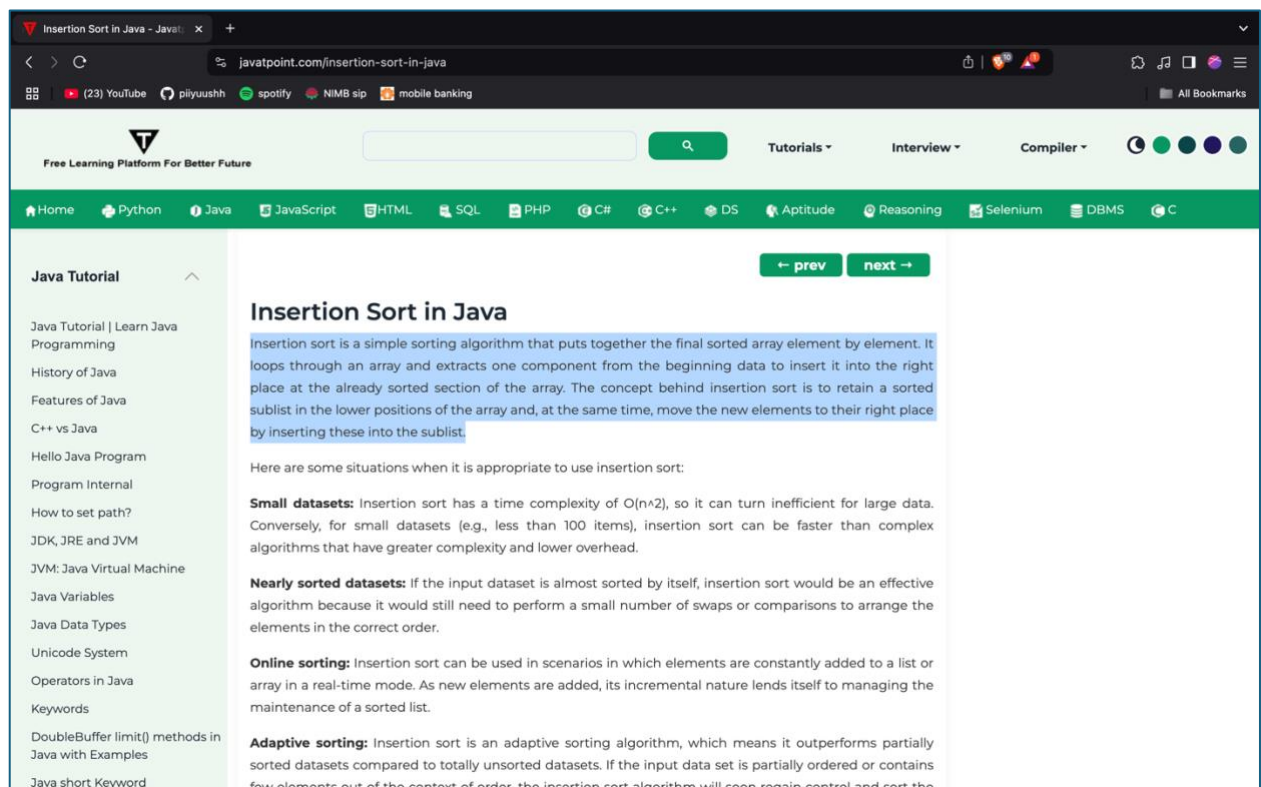


Figure 109: Proof of citation explanation

5.4.1.2 Steps of Insertion Sort

- **Initialization:** Initialized an empty array list to store the sorted list of adminModel objects.
- **Constructor:** Constructor is initialized to form array list.
- **Method:** A method named sortByQuantity is created to perform sorting operations.
- **Input validation:** This method checks for null entries and throws exception if the list is found empty.
- **Sorting preparation:** Clears any existing data in the list.
- **Sorting Algorithm:** Iterates over list, stores the key position of the current element, compares with elements with sorted part of the list, shifts the elements that are greater.
- **Comparison Logic:** The program checks if the items are required to be swapped. If sorting in descending order it moves smaller items forward else if sorting in ascending order it moves larger items forward.
- **Insert element:** Once correct spot is found for the key element, it is put there.
- **Return sorted List:** After sorting is completed, sorted list is returned.
- **Error Handling:** if input list is invalid the program doesn't sort the list and gives an error.

5.4.1.3 Implementation of Insertion Sort

In this project, we have used insertion sort to sort the table items as per quantity which is in descending order. The program implements the Insertion Sort algorithm to sort a list of products based on their quantity in descending order. The class InsertionSort maintains a QuantitySortedList to hold the products during sorting. The main method takes a list of products to indicate the sort order. It clears any previously added data, validates that the input list, and then sorts the list using insertion sort. For each product in the list, it compares the quantity with the quantities of already sorted products. The shouldSwap method decides whether an element should move based on the sorting order. The product is then placed in its correct position within the sorted portion of the list. The sorted list is returned at the end.

```

/**
public class InsertionSort {
    // list for quantity sorting
    List<adminModel> QuantitySortedList;

    /**
     * Constructor initializes the list for storing the sorted results.
     */
    public InsertionSort() {
        // initializing array list for quantity
        QuantitySortedList = new ArrayList<>();
    }

    /**
     * Sorts a list of adminModel objects by quantity.
     *
     * @param productList the list of adminModel objects to be sorted
     * @param isDesc       Boolean indicating whether to sort in descending order (true)
     *                     or ascending order (false)
     * @return the sorted list of adminModel objects
     * @throws IllegalArgumentException if the provided list is null or empty
     */
    public List<adminModel> sortByQuantity(List<adminModel> productList, boolean isDesc) {
        this.QuantitySortedList.clear();
        this.QuantitySortedList.addAll(productList);
        if (QuantitySortedList == null || QuantitySortedList.isEmpty()) {
            throw new IllegalArgumentException("Product list is empty. Insert data!!");
        }

        // Perform the insertion sort
        for (int i = 1; i < QuantitySortedList.size(); i++) {
            adminModel key = QuantitySortedList.get(i);
            int j = i - 1;

            // Move elements of QuantitySortedList[0..i-1] that are greater (or smaller depending on isDesc) than key
            while (j >= 0 && shouldSwap(QuantitySortedList.get(j).getQuantity(), key.getQuantity(), isDesc)) {
                QuantitySortedList.set(j + 1, QuantitySortedList.get(j));
                j = j - 1;
            }
            // place key in correct position
            QuantitySortedList.set(j + 1, key);
        }

        return QuantitySortedList;
    }
}

```

Figure 110: Using insertion sort to sort by quantity

After the program is executed, the table looks like this. The table initially had randomly arranged data but after using sort by price button, the sorting is executed and the sorted is seen in the table.

```
/*
insertion sort for sort by quantity
@param adminModel for model
creates object sortQuantity to handle sorting
Adds data to list
*/
private void btnSortQuantityActionPerformed(java.awt.event.ActionEvent evt) {
    // Create an instance of the InsertionSort class
    InsertionSort sortQuantity = new InsertionSort();

    // Ensure productList is properly populated with product data
    // Sort the product list by quantity (descending order in this case)
    List<adminModel> sortedList = sortQuantity.sortByQuantity(productList, true);

    // Load the sorted data into the table
    loadSortedProductData(sortedList);
}
```

Figure 111: Implementing insertion sort

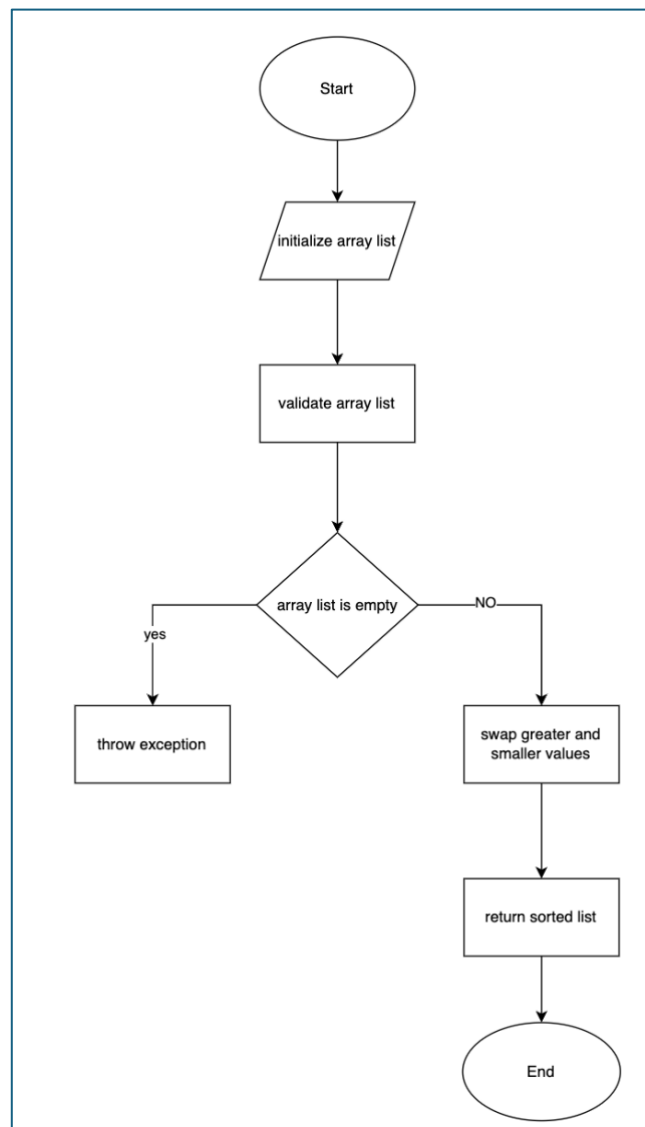
5.4.1.4 Flowchart

Figure 112: flowchart of insertion sort

5.4.2 Selection Sort

Selection sort is an easy-to-understand and straightforward kind of algorithm that sorts elements in a list or array, either ascending or descending. The element idea at the core of selection sort is to obtain repeatedly the minimum (or the maximum) element of the unsorted part and swap it with the element at the beginning of the unsorted part. This process of sorting will continue until the entire inventory is sorted. (JavatPoint, n.d.)

5.4.2.1 Evidence of Citation Explanation

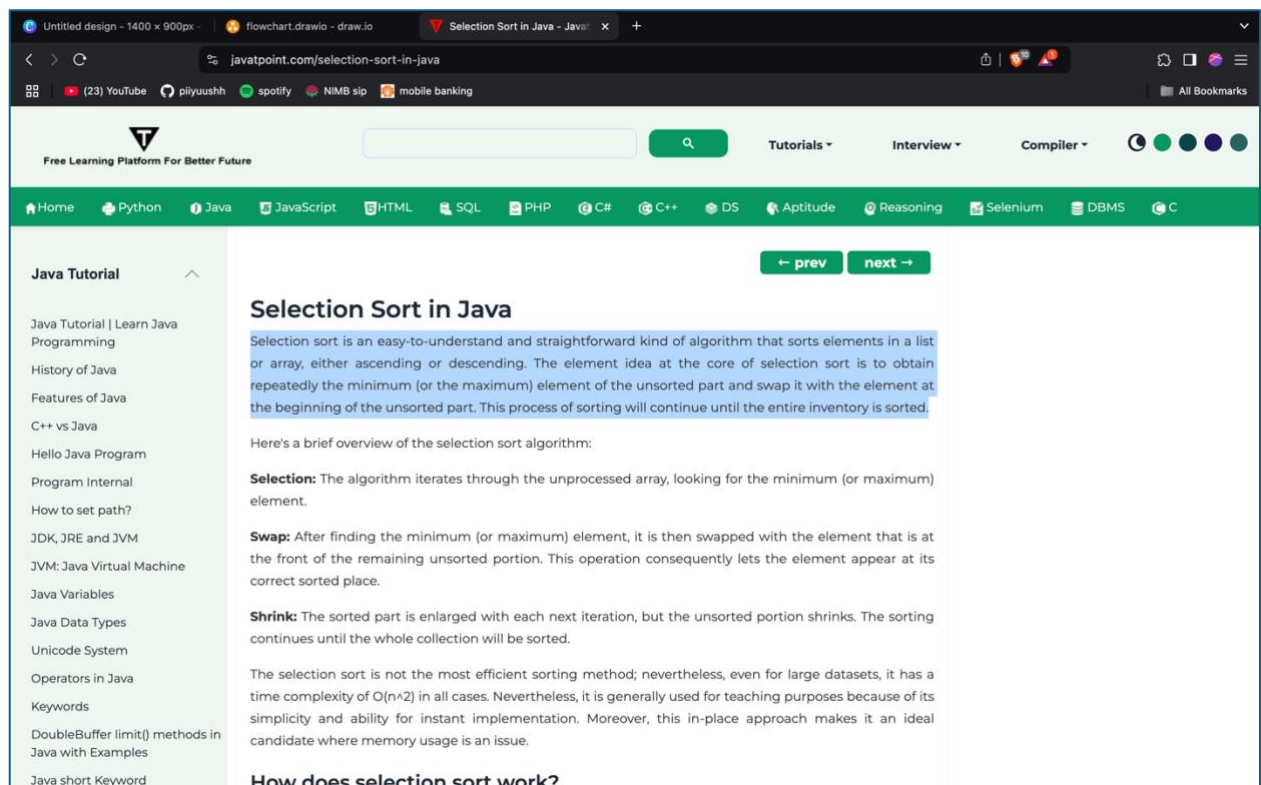


Figure 113: Evidence of selection sort citation explanation

5.4.2.2 Steps of Selection Sort

- **Create a list for sorting:** The program uses a list called PriceSortedList to store the products while sorting them by price.
- **Constructor:** When the SelectionSort class is created, it initializes PriceSortedList as an empty list.
- **Sorting Method:** This method takes a list of products (productList) and sorts them by price.
- **Check Input:** The program checks if the input list is empty or null and shows an error message if list is found empty.

- **Sorting:** Find the position of the smallest (or largest, depending on the order) price in the remaining list and swap the current item with the item at the found position if they are different.
- **Find Price Index:** This helper method goes through the unsorted part of the list to find the index of the largest price (descending order).
- **Comparison Logic:** The program checks if two items should be swapped. Swap if the current price is larger.
- **Swap Items:** A helper method swaps two items in the list to put them in the correct order.
- **Return sorted list:** After sorting all items, the method gives back the sorted list.

5.4.2.3 Implementation of Selection Sort

```

/*
price sorting starts.
@param array list PriceSortedList
validates empty sort
*/
public List<adminModel> sortByPrice(List<adminModel> productList, boolean isDesc) {
    this.PriceSortedList.clear();
    this.PriceSortedList.addAll(productList);
    if (PriceSortedList == null || PriceSortedList.isEmpty()) {
        throw new IllegalArgumentException("Product list is empty. Insert data!!");
    }

    for (int i = 0; i < PriceSortedList.size() - 1; i++) {
        int priceIndex = findpriceIndex(PriceSortedList, i, isDesc);
        if (i != priceIndex) {
            swap(PriceSortedList, i, priceIndex);
        }
    }

    return PriceSortedList;
}

```

Figure 114: method to sort by price

```

/*
method to find price index
@param PriceSortedList is used to get price
*/
private int findpriceIndex(List<adminModel> priceSortList, int startIndex, boolean isDesc) {
    int extremumIndex = startIndex;

    for (int j = startIndex + 1; j < priceSortList.size(); j++) {
        if (shouldSwap(PriceSortedList.get(j).getPrice(), PriceSortedList.get(extremumIndex).getPrice(), isDesc)) {
            extremumIndex = j;
        }
    }

    return extremumIndex;
}

```

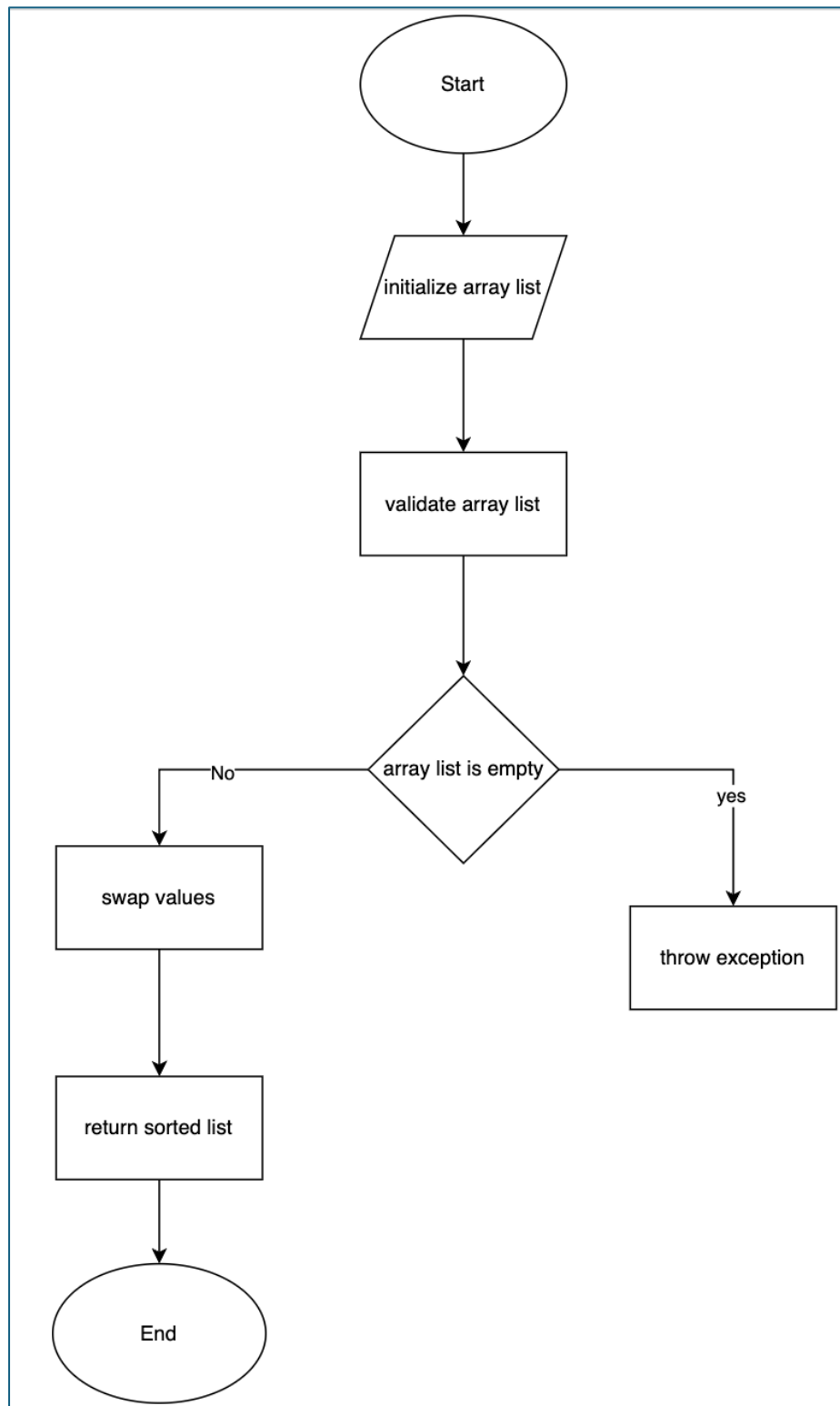
Figure 115: Calculating price index

```
// checks if values are to be swapped or not
private boolean shouldSwap(int current, int extremum, boolean isDesc) {
    return isDesc ? current > extremum : current < extremum;
}
// swaps sorted price value
private void swap(List<adminModel> priceSortList, int i, int j) {
    adminModel temp = PriceSortedList.get(i);
    PriceSortedList.set(i, PriceSortedList.get(j));
    PriceSortedList.set(j, temp);
}
```

Figure 116: swapping

```
/*
method to execute selection sort for price
Arranges item price ranging from high to low
@param adminModel for model
creates a object sort and handles sorting
Adds data to list
*/
private void btnSortPriceActionPerformed(java.awt.event.ActionEvent evt) {
    SelectionSort sort = new SelectionSort();
    List<adminModel> sortedList = sort.sortByPrice(productList, true);
    loadSortedProductData(sortedList);
}
```

Figure 117: implementing Selection sort

5.4.2.4 Flowchart of Selection Sort*Figure 118: flowchart of selection sort*

5.4.3 Merge Sort

Merge sort is a sorting algorithm that follows the divide and merge approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array. (Geeks, n.d.)

5.4.3.1 Evidence of Citation Explanation

Merge Sort – Data Structure and Algorithms Tutorials
Last Updated : 02 Jan, 2025

Merge sort is a sorting algorithm that follows the [divide-and-conquer](#) approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array.

In simple terms, we can say that the process of **merge sort** is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

Merge Sort Algorithm

Table of Content

- [How does Merge Sort work?](#)

Figure 119: Evidence of citation explanation of merge sort

5.4.3.2 Steps of Merge Sort

- **Validate Input:** The program checks if the input list (product) is empty or null and throws an error message if list is found empty.
- **Sorting:** Calls the mergeSort method to recursively divide and sort the list based on the product name.
- **Divide the list:** Split the list into two sub lists, Left and right.
- **Merge sorted List:** Combines sorted sub lists.
- **Comparison Logic:** Uses compareToIgnoreCase to compare product names alphabetically. Places the items in descending order.
- **Return sorted list:** Returns sorted list after merging two sub lists.

5.4.3.3 Implementation of Merge Sort

```
*/  
  
public List<adminModel> sortByName(List<adminModel> product, boolean isDesc) {  
    if (product == null || product.isEmpty()) {  
        throw new IllegalArgumentException("Product list is empty. Fix it!");  
    }  
  
    return mergeSort(product, isDesc);  
}
```

Figure 120: validating empty fields

```
/**  
 * Recursively divides and sorts the list of adminModel objects using Merge Sort.  
 *  
 * @param product the list of adminModel items to be sorted  
 * @param isDesc Boolean indicating the sort order  
 * @return the sorted list of adminModel objects  
 */  
private List<adminModel> mergeSort(List<adminModel> product, boolean isDesc) {  
    if (product.size() <= 1) {  
        return product;  
    }  
  
    int mid = product.size() / 2;  
    List<adminModel> left = new ArrayList<>(product.subList(0, mid));  
    List<adminModel> right = new ArrayList<>(product.subList(mid, product.size()));  
  
    // Recursively split and merge  
    left = mergeSort(left, isDesc);  
    right = mergeSort(right, isDesc);  
  
    return merge(left, right, isDesc);  
}
```

Figure 121: Splitting into two sub lists

```

/**
 * Merges two sorted sub lists into a single sorted list.
 *
 * @param left the left sub list
 * @param right the right sub list
 * @param isDesc Boolean indicating the sort order
 * @return the mergedList and sorted list
 */
private List<adminModel> merge(List<adminModel> left, List<adminModel> right, boolean isDesc) {
    List<adminModel> mergedList = new ArrayList<>();
    int i = 0, j = 0;
    // Merge based on the model names
    while (i < left.size() && j < right.size()) {
        if (shouldPlaceLeft(left.get(i).getProductName(), right.get(j).getProductName(), isDesc)) {
            mergedList.add(left.get(i));
            i++;
        } else {
            mergedList.add(right.get(j));
            j++;
        }
    }

    // Add remaining elements
    while (i < left.size()) {
        mergedList.add(left.get(i));
        i++;
    }

    while (j < right.size()) {
        mergedList.add(right.get(j));
        j++;
    }

    return mergedList;
}

```

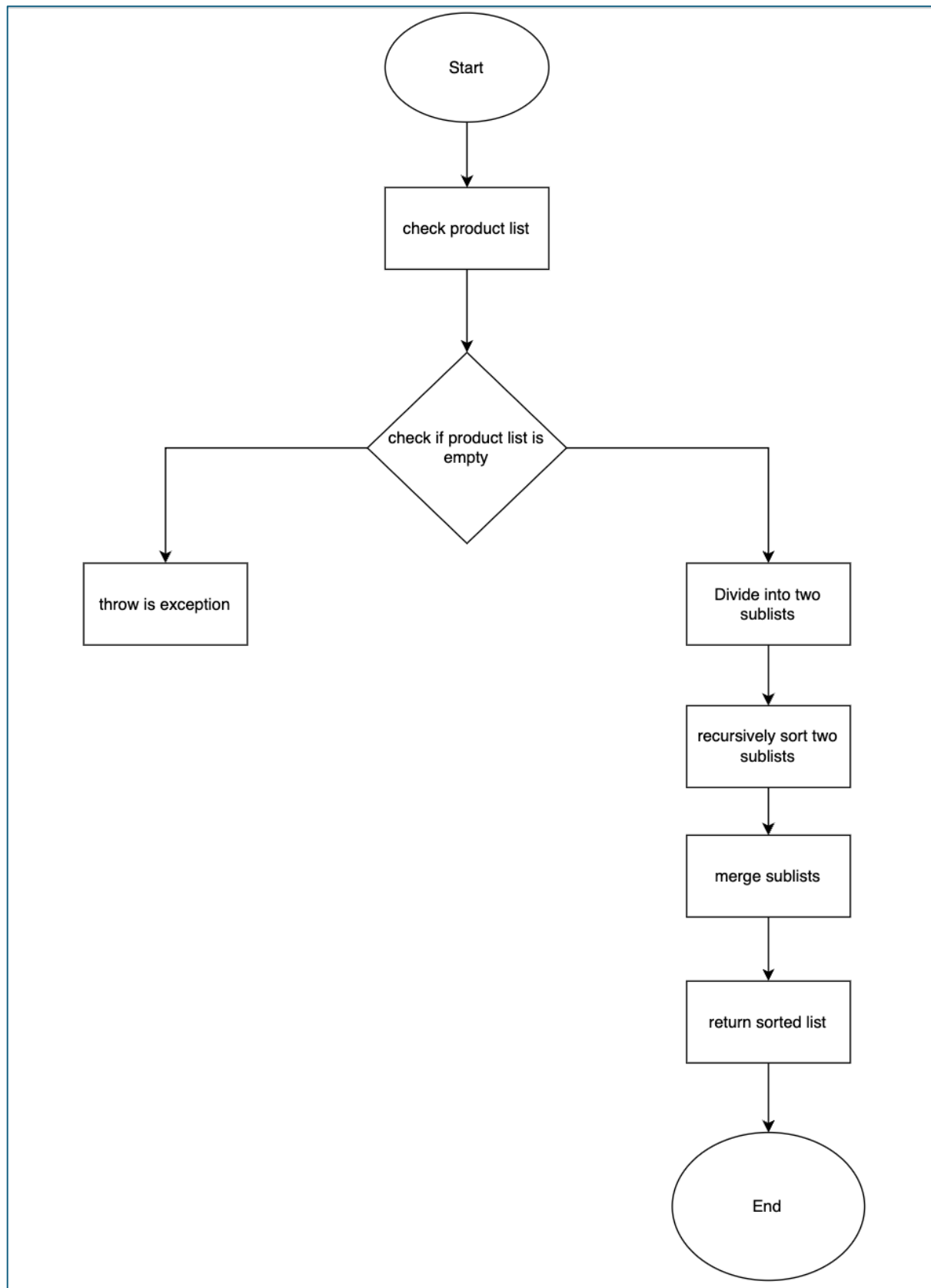
Figure 122: Merging sub lists

```

/**
 method to sort by name
 @param adminModel for model
 creates object merge to handle sorting
 adds data to list
 */
private void btnSortNameActionPerformed(java.awt.event.ActionEvent evt) {
    MergeSort sort = new MergeSort();
    List<adminModel> merge = sort.sortByName(productList, true);
    loadSortedProductData(merge);
}

```

Figure 123: Implementing merge sort

5.4.3.4 Flowchart of Merge Sort*Figure 124: flow chart of merge sort*

5.4.4 Binary Search

Binary search is one of the searching techniques applied when the input is sorted here we are focusing on finding the middle element that acts as a reference frame whether to go left or right to it as the elements are already sorted. This searching helps in optimizing the search technique with every iteration is referred to as binary search and readers do stress over it as it is indirectly applied in solving questions. (Geeks, n.d.)

5.4.4.1 Evidence of Citation Explanation

Binary Search in Java
Last Updated : 23 Oct, 2024

Binary search is one of the searching techniques applied when the input is sorted here we are focusing on finding the middle element that acts as a reference frame whether to go left or right to it as the elements are already sorted. This searching helps in optimizing the search technique with every iteration is referred to as binary search and readers do stress over it as it is indirectly applied in solving questions.

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
23 > 16 take 2nd half	L=0	1	2	3	M=4	5	6	7	8	9
23 < 56 take 1st half	0	1	2	3	4	L=5	6	M=7	8	H=9
Found 23, Return 5	0	1	2	3	4	L=5, M=5	H=6	7	8	9

Binary Search Algorithm in Java
Below is the Algorithm designed for Binary Search:

1. Start

Figure 125: Evidence of citation explanation of binary search

5.4.4.2 Steps of Binary Search

- **Input Validation:** checks if the search range is invalid, if $right < left$ then return null.
- **Calculate middle index:** calculate middle index using formulae $\frac{(left+right)}{2}$
- **Compare search values:** It compares the search value (searchValue) with the product name of the element at the middle index. If they are equal, that means search value has been found.
- **Search in left Sub list:** Search for corresponding value in left sub list. If value is less than middle index, calls itself recursively with updated range left to mid -1.

- **Search in right sub list:** If value is greater than middle index, calls itself recursively with updated range left to mid +1 to right.
- **Return sorted list:** returns sorted list after index is found.

5.4.4.3 Implementation of Binary Search

```
public adminModel searchByProductName(String searchValue, List<adminModel> studentList,
    int leftIndex, int rightIndex) {

    // if the search range is invalid, return null
    if (rightIndex < leftIndex) {
        return null;
    }

    // mid value of current search range
    int mid = (leftIndex + rightIndex) / 2;
```

Figure 126: initializing binary search

```
// Compare the search value with the product name at the middle index
if (searchValue.compareToIgnoreCase(studentList.get(midIndex).getProductName()) == 0) {
    // If the search value matches the product name, return the object
    return studentList.get(midIndex);
} else if (searchValue.compareToIgnoreCase(studentList.get(midIndex).getProductName()) < 0) {
    // If the search value is less than the middle product name, search in the left sublist
    return searchByProductName(searchValue, studentList, leftIndex, midIndex - 1);
} else {
    // If the search value is greater than the middle product name, search in the right sublist
    return searchByProductName(searchValue, studentList, midIndex + 1, rightIndex);
}
}
```

Figure 127: comparing search values

```
private void btnSearchActionPerformed(java.awt.event.ActionEvent evt) {
    MergeSort searchProduct = new MergeSort();
    List<adminModel> sortedList = searchProduct.sortByName(productList, false);
    BinarySearch search = new BinarySearch();
    adminModel searchedData = search.searchByProductName(txtFldSearchBar.getText().trim().toLowerCase(), sortedList, 0, sortedList.size() - 1);
    if (searchedData != null) {
        System.out.println(searchedData.getProductName() + " is found.");
        JOptionPane.showMessageDialog(null, searchedData.getProductName() + " is found.", "Item Found", JOptionPane.INFORMATION_MESSAGE);
    } else {
        System.out.println("Product not found.");
        JOptionPane.showMessageDialog(null, "Item not found. Please check the table or try changing keywords.", "Product Not Found", JOptionPane.WARNING_MESSAGE);
    }
}
```

Figure 128: implementing binary search

5.4.4.3 Flowchart of Binary Search

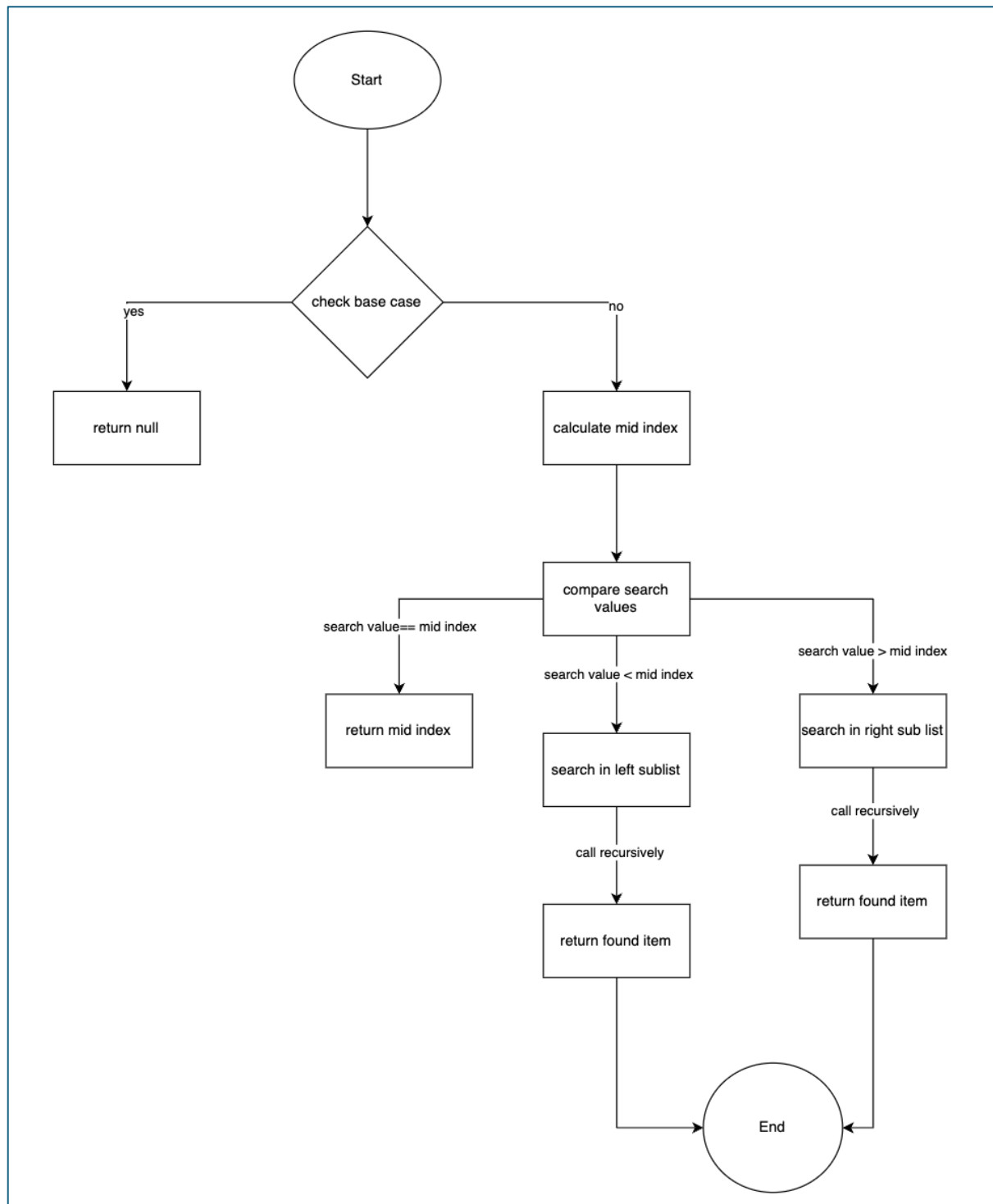


Figure 129: flow chart of binary search

5.5 Data Structure

5.5.1 Linked List

In Java, a **LinkedList** is a class that implements the **List** interface and represents a linked list data structure. Unlike arrays, which store elements in contiguous memory locations, a linked list stores elements as nodes, where each node contains the element itself and a reference to the next node in the sequence. (Javatpoint, n.d.)

5.5.1 Evidence of Citation Explanation of Linked List

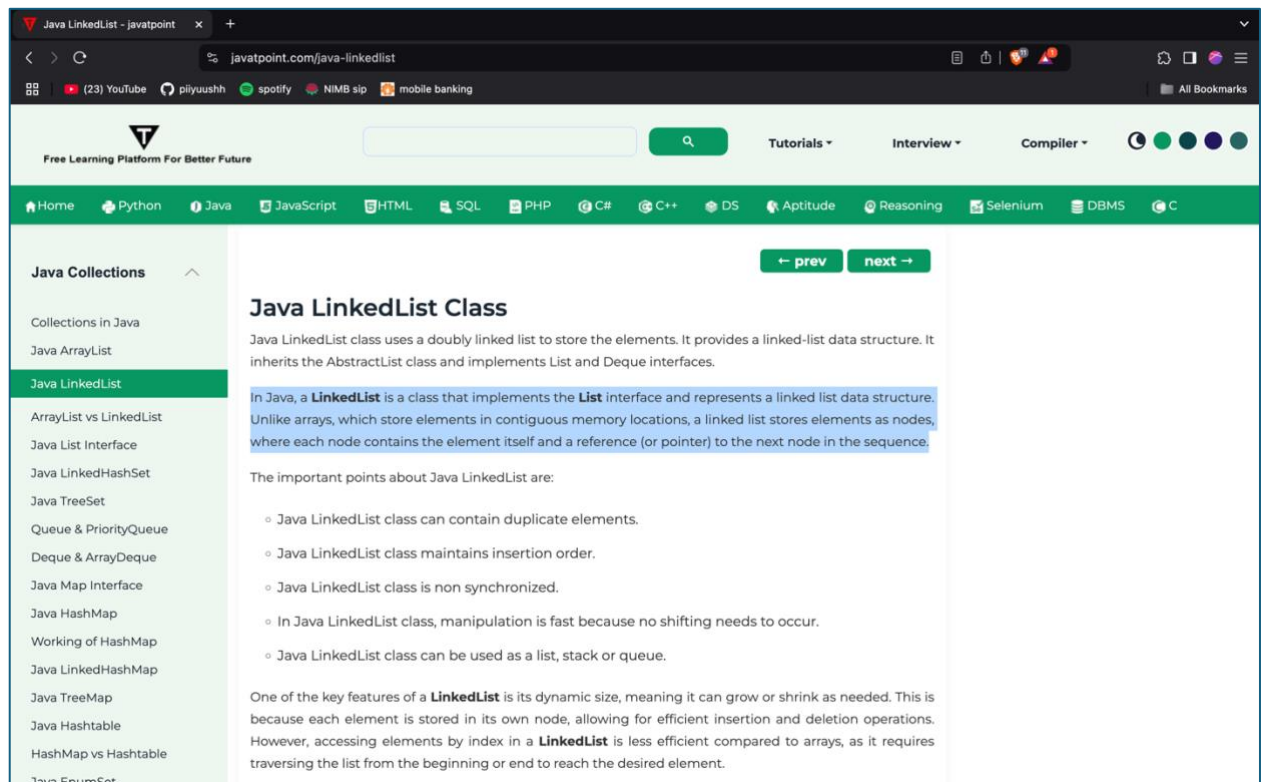


Figure 130: Evidence of citation explanation of linked list

5.5.2 Implementation of Linked List

Linked list is created at the beginning of the program. Further it is used in the applicable place in the program. In the program, linked list is created by the name **productList**. Hence, all the spots where **productList** is used are highlighted.

```

/**
 *
 * @author piyushkarn
 */
public class AdminDashboard extends javax.swing.JFrame {

    /**
     * Creates new form AdminDashboard
     */
    public AdminDashboard() {
        initComponents();
        displayPreLoadedItems();
    }

    // Create an linked list to store product data
    LinkedList<adminModel> productList = new LinkedList<>();
}

```

Figure 131: Initializing linked list in this program

Firstly, it is used at table model to add items to admin dashboard table as pre-loaded data in the table. It is to show engagement in the software, as items are seen in the bakery.

```

// Create an linked list to store product data
LinkedList<adminModel> productList = new LinkedList<>();

// method to add bakery Items to table row
/**
 * @param ProductId the product ID
 * @param ProductName the product name
 * @param Quantity the Quantity
 * @param Price the Price
 */
public void addBakeryItems(adminModel bakeryItems) {
    productList.add(bakeryItems);
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    Object[] bakeryData = new Object[]{
        bakeryItems.getProductId(), bakeryItems.getProductName(), bakeryItems.getQuantity(), bakeryItems.getPrice()
    };
    model.addRow(bakeryData);
}
}

```

Figure 132: using Linked list to display bakery items

Further, productList is used to check duplicate product in the table in the GUI. It uses product Id to check any repeating data in the table. If the product with same product Id is found, it returns some kind of response in the GUI.

```
// method to check duplicate data in the data
private boolean checkDuplicateProduct(adminModel bakeryItems) {
    for (adminModel existingBakeryItem : productList) {
        if (existingBakeryItem.getProductid() == bakeryItems.getProductid()) {
            return true;
        }
    }
    return false;
}
```

Figure 133: using linked list to check duplicate data

Product list is also used to update the information of currently added items in the table. As the items which are displayed in the table in the GUI are retrieved from the list itself. Hence, product list is also used to update items information in the GUI.

```
/*
method to update table item
@param product id
@param product name
@param product quantity
@param price
*/
private void updateItems() {
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    // Clear the table before loading new data
    // set model row count to 0 to locate to exact updation location
    model.setRowCount(0);
    // add row for every fields
    for (adminModel products : productList) {
        model.addRow(new Object[]{
            products.getProductid(),
            products.getProductname(),
            products.getQuantity(),
            products.getPrice()
        });
    }
}
```

Figure 134: using linked list to update items information

5.5.2 Array List

ArrayList in Java is a dynamic array implementation that belongs to the Java Collections Framework. This is a big array that grows on its own as more elements are added to it. (Javatpoint, n.d.)

5.5.2 Evidence of Citation Explanation of Array List

The screenshot shows the Javatpoint website's page for 'Java ArrayList'. The page title is 'Java ArrayList - javatpoint'. The main content area is titled 'Java ArrayList' and contains the following text:

ArrayList in Java is a dynamic array implementation that belongs to the Java Collections Framework. This is a big array that grows on its own as more elements are added to it.

ArrayLists come from the java.util package and are quite commonly used for their ease of use and flexibility. They offer flexibility in that you do not need to determine the size of the ArrayList at the time of its creation, which is similar to standard arrays in Java. So, it is much more flexible than the traditional array. It is like the Vector in C++.

The ArrayList in Java can also have duplicate elements. It implements the List interface so that we can use all the methods of the List interface here. The ArrayList maintains the insertion order internally.

It inherits the AbstractList class and implements List interface.

The important points about the Java ArrayList class are:

- **Maintains Insertion Order:** Java ArrayList guarantees the order in which elements are fed into it. When going through the ArrayList with the iterating process, elements are accessed in the same sequence they were added.
- **Non-Synchronized:** Unlike some other Java collection classes (for instance, Vector), ArrayList is

To the right of the text is a class hierarchy diagram:

```
graph BT
    Iterable -- extends --> Collection
    Collection -- extends --> List
    List -- implements --> AbstractList
    AbstractList -- extends --> ArrayList
```

Figure 135: evidence of explanation of citation of array list

5.5.3 Implementation of Array List

In this program, array list is created and implemented in sorting and searching algorithms. In selection sort algorithm array list is created and implemented by the name **PriceSortedList**. In insertion sort algorithm, array list is created and implemented by the name **QuantitySortedList**.

```

/**
 *
 * @author piyushkarn
 */
public class SelectionSort {
    // list for price sorting
    List<adminModel> PriceSortedList;

    public SelectionSort() {
        // initializing array list for price
        PriceSortedList = new ArrayList<>();
    }
}

```

Figure 136: initializing array list in selection sort

Here, the PriceSortedList is implemented to sort the items based in their price high to low. First the list is cleared to ensure no data is entered in the list by any means. Then, items from product list are added to the array list and items are swapped based on their price.

```

/*
price sorting starts.
@param array list priceSortedList
validates empty sort
*/
public List<adminModel> sortByPrice(List<adminModel> productList, boolean isDesc) {
    this.PriceSortedList.clear();
    this.PriceSortedList.addAll(productList);
    if (PriceSortedList == null || PriceSortedList.isEmpty()) {
        throw new IllegalArgumentException("Product list is empty. Insert data!!");
    }

    for (int i = 0; i < PriceSortedList.size() - 1; i++) {
        int priceIndex = findpriceIndex(PriceSortedList, i, isDesc);
        if (i != priceIndex) {
            swap(PriceSortedList, i, priceIndex);
        }
    }

    return PriceSortedList;
}

```

Figure 137: using array list to sort by price

Here, the priceSortedList is used to get price index from the data entered into the list.


```

/*
method to find price index
@param priceSortedList is used to get price
*/
private int findpriceIndex(List<adminModel> priceSortList, int startIndex, boolean isDesc) {
    int extremumIndex = startIndex;

    for (int j = startIndex + 1; j < priceSortList.size(); j++) {
        if (shouldSwap(PriceSortedList.get(j).getPrice(), PriceSortedList.get(extremumIndex).getPrice(), isDesc)) {
            extremumIndex = j;
        }
    }

    return extremumIndex;
}

```

Figure 138: calculating price index

Checking if the values are swapped according to their price or not.

```

// checks is values are to be swapped or not
private boolean shouldSwap(int current, int extremum, boolean isDesc) {
    return isDesc ? current > extremum : current < extremum;
}

// swaps sorted price value
private void swap(List<adminModel> priceSortList, int i, int j) {
    adminModel temp = PriceSortedList.get(i);
    PriceSortedList.set(i, PriceSortedList.get(j));
    PriceSortedList.set(j, temp);
}
}

```

Figure 139: checking swapped values

Now in insertion sort, the array list is created and implemented by the name **QuantitySortedList**.

```

/**
 *
 * @author piyushkarn
 */
public class InsertionSort {
    // list for quantity sorting
    List<adminModel> QuantitySortedList;

    /**
     * Constructor initializes the list for storing the sorted results.
     */
    public InsertionSort() {
        // initializing array list for quantity
        QuantitySortedList = new ArrayList<>();
    }
}

```

Figure 140: Initializing array list in insertion sort

Here, **QuantitySortedList** is first cleared and then all the data is added into the array list retrieved from the admin model. Then empty list is validated then insertion sort is performed.

```
/**
 * Sorts a list of adminModel objects by quantity.
 *
 * @param productList the list of adminModel objects to be sorted
 * @param isDesc      Boolean indicating whether to sort in descending order (true)
 *                   or ascending order (false)
 * @return the sorted list of adminModel objects
 * @throws IllegalArgumentException if the provided list is null or empty
 */
public List<adminModel> sortByQuantity(List<adminModel> productList, boolean isDesc) {
    this.QuantitySortedList.clear();
    this.QuantitySortedList.addAll(productList);
    if (QuantitySortedList == null || QuantitySortedList.isEmpty()) {
        throw new IllegalArgumentException("Product list is empty. Insert data!!");
    }

    // Perform the insertion sort
    for (int i = 1; i < QuantitySortedList.size(); i++) {
        adminModel key = QuantitySortedList.get(i);
        int j = i - 1;

        // Move elements of QuantitySortedList[0..i-1] that are greater (or smaller depending on isDesc) than key
        while (j >= 0 && shouldSwap(QuantitySortedList.get(j).getQuantity(), key.getQuantity(), isDesc)) {
            QuantitySortedList.set(j + 1, QuantitySortedList.get(j));
            j = j - 1;
        }
        // place key in correct position
        QuantitySortedList.set(j + 1, key);
    }

    return QuantitySortedList;
}
```

Figure 141: performing insertion sort in QuantitySortedList

Here, array list is implemented to divide into two sublists left and right. They are used to sort the items within them, and return sorted item in descending alphabetical order.

```

/**
 * Recursively divides and sorts the list of adminModel objects using Merge Sort.
 *
 * @param product the list of adminModel items to be sorted
 * @param isDesc Boolean indicating the sort order
 * @return the sorted list of adminModel objects
 */
private List<adminModel> mergeSort(List<adminModel> product, boolean isDesc) {
    if (product.size() <= 1) {
        return product;
    }

    int mid = product.size() / 2;
    List<adminModel> left = new ArrayList<>(product.subList(0, mid));
    List<adminModel> right = new ArrayList<>(product.subList(mid, product.size()));

    // Recursively split and merge
    left = mergeSort(left, isDesc);
    right = mergeSort(right, isDesc);

    return merge(left, right, isDesc);
}

/**
 * Merges two sorted sub lists into a single sorted list.
 *
 * @param left the left sub list
 * @param right the right sub list
 * @param isDesc Boolean indicating the sort order
 * @return the merged and sorted list
 */
private List<adminModel> merge(List<adminModel> left, List<adminModel> right, boolean isDesc) {
    List<adminModel> merged = new ArrayList<>();
    int i = 0, j = 0;
    // Merge based on the model names
    while (i < left.size() && j < right.size()) {
        if (shouldPlaceLeft(left.get(i).getProductName(), right.get(j).getProductName(), isDesc)) {
            merged.add(left.get(i));
            i++;
        } else {
            merged.add(right.get(j));
            j++;
        }
    }
    merged.addAll(left.subList(i, left.size()));
    merged.addAll(right.subList(j, right.size()));
    return merged;
}

```

Figure 142: implementing array list in merge sort

6. Critical Analysis

6.1 UI/UX Design & Development

UI/UX Design, referred to as “User Interface/User Experience design,” has become very important in the field of application design focusing on creating a visual interface and improving the experience of the user. The role of a UI/UX designer is to ensure that the application, or software is user-friendly and visually appealing for the user. (Geeks, 2023)

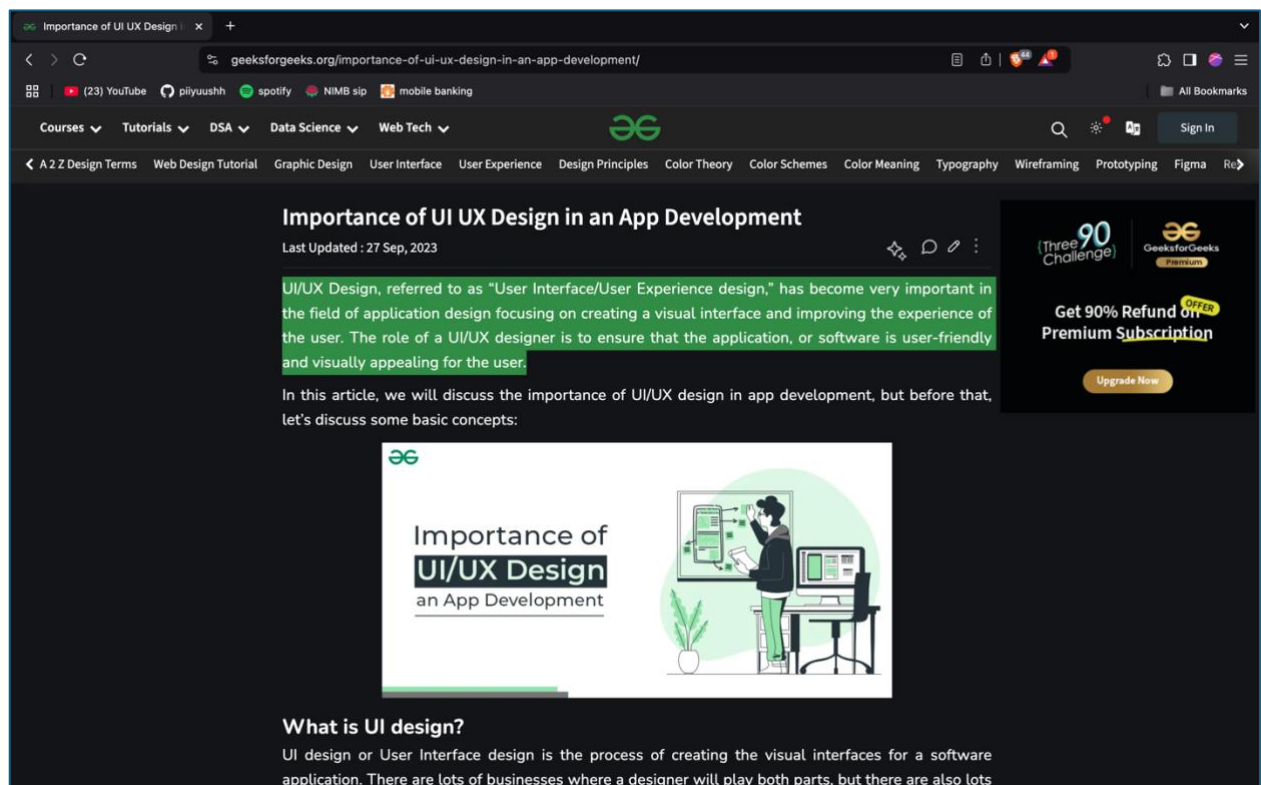


Figure 143: evidence of citation of UI/UX development

In developing the GUI for this system application, we have used Canva to design the wireframe and to implement the design, we have used Netbeans IDE. Netbeans contains inbuilt GUI builder elements which can be used to create the GUI as required.

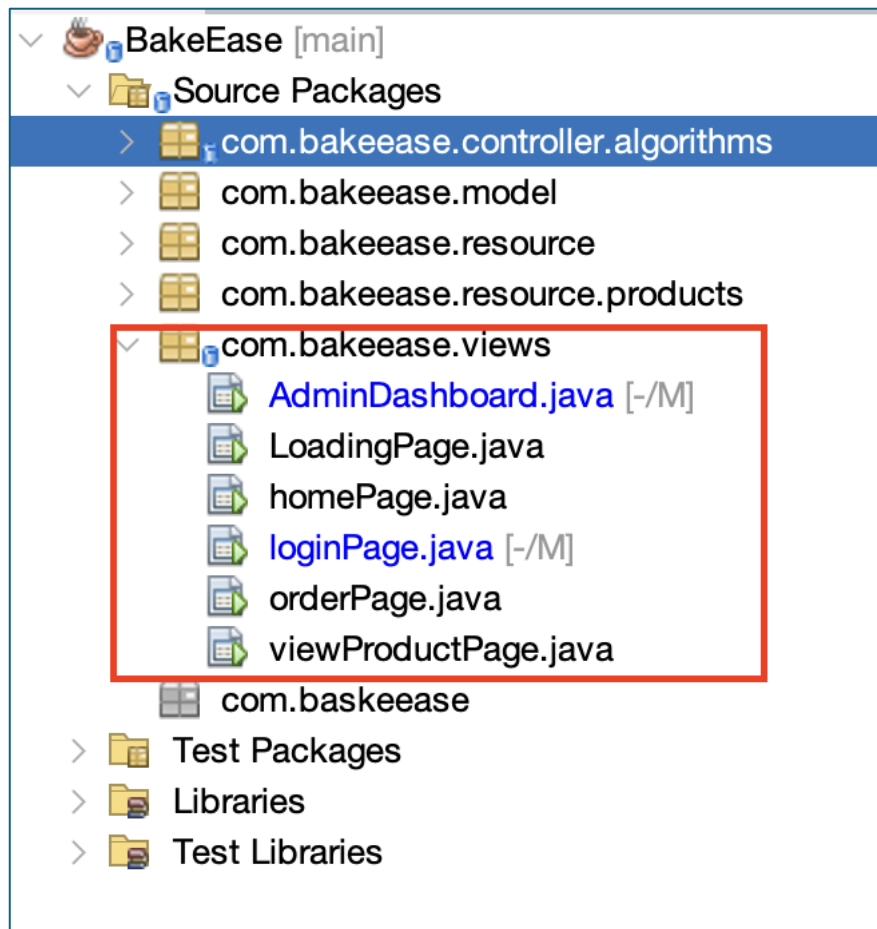


Figure 144: Views package

Above we can see that this project follows MVC architecture. Inside views package it contains all the JFrame forms which contains GUI interfaces used in the system.

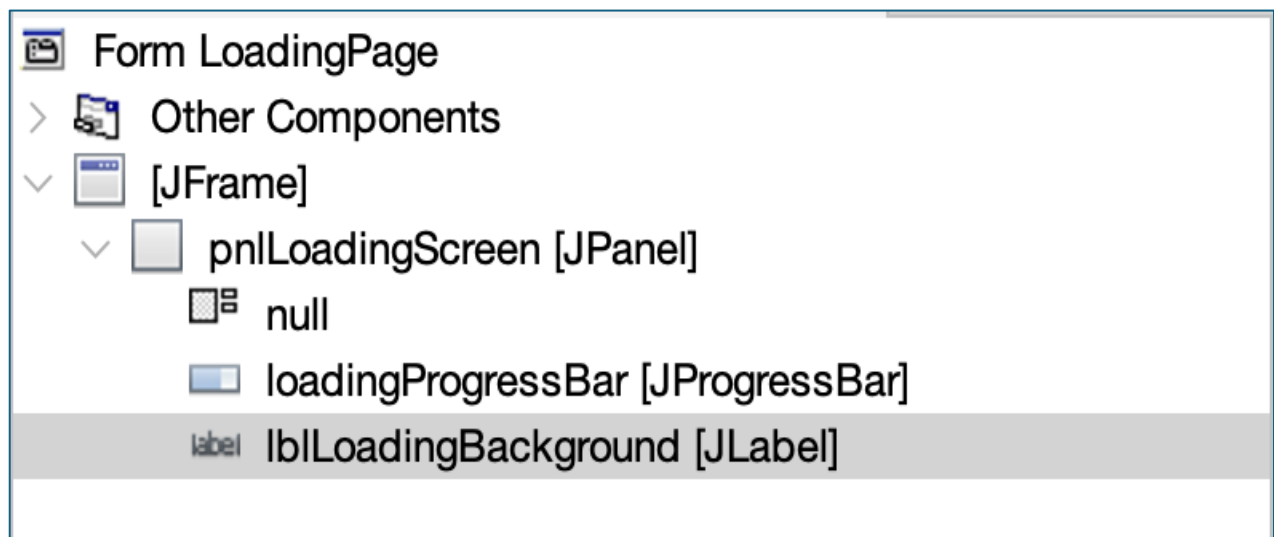


Figure 145: contents of loading screen

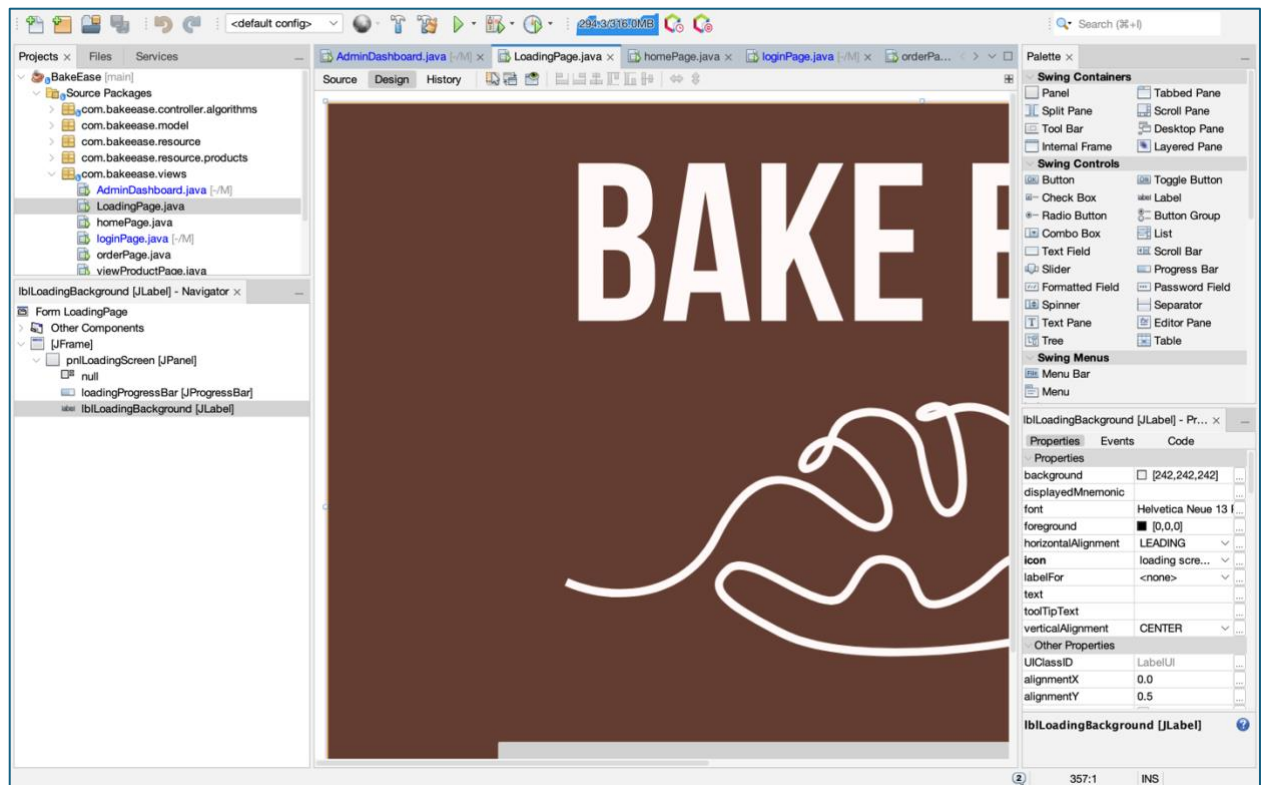


Figure 146: designing loading screen

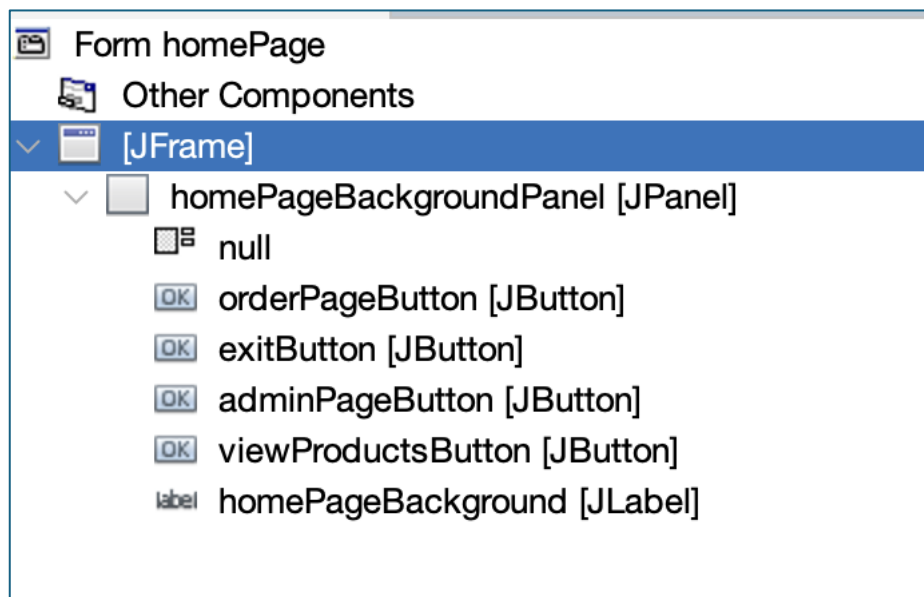


Figure 147: contents of home page

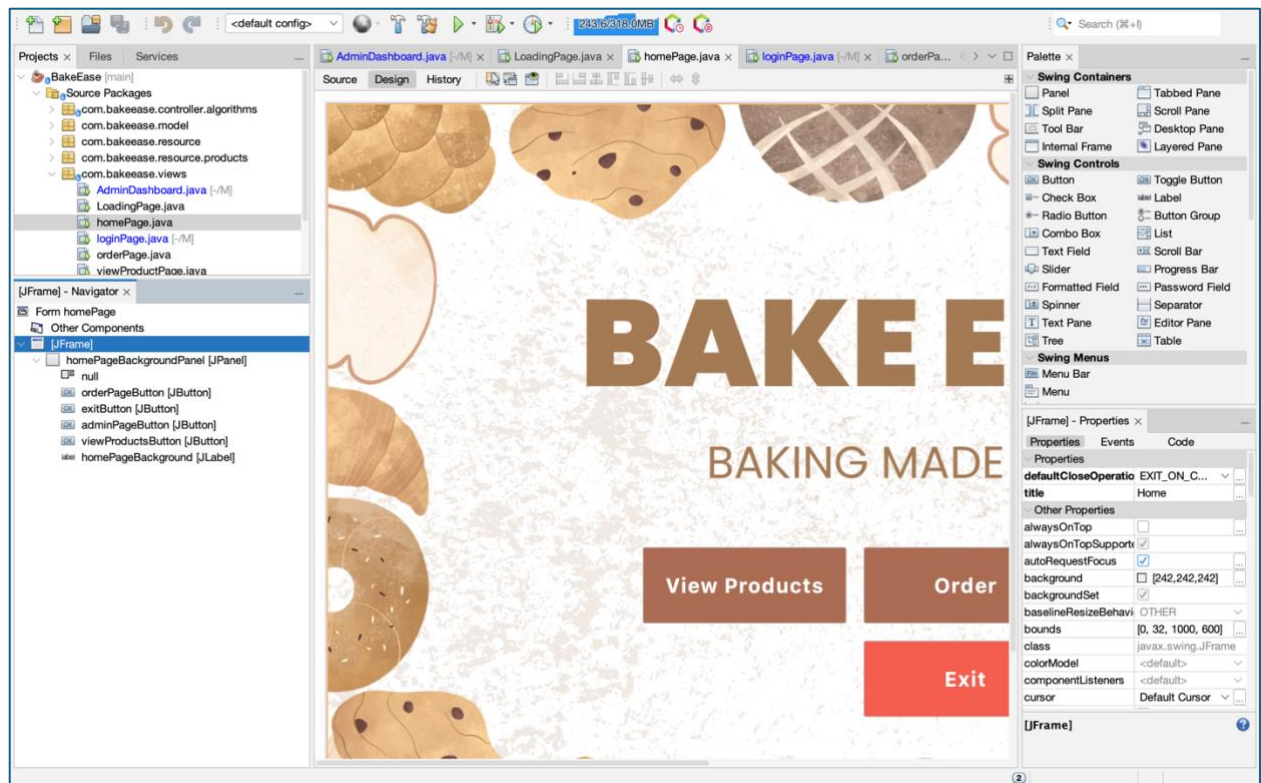


Figure 148: designing home page

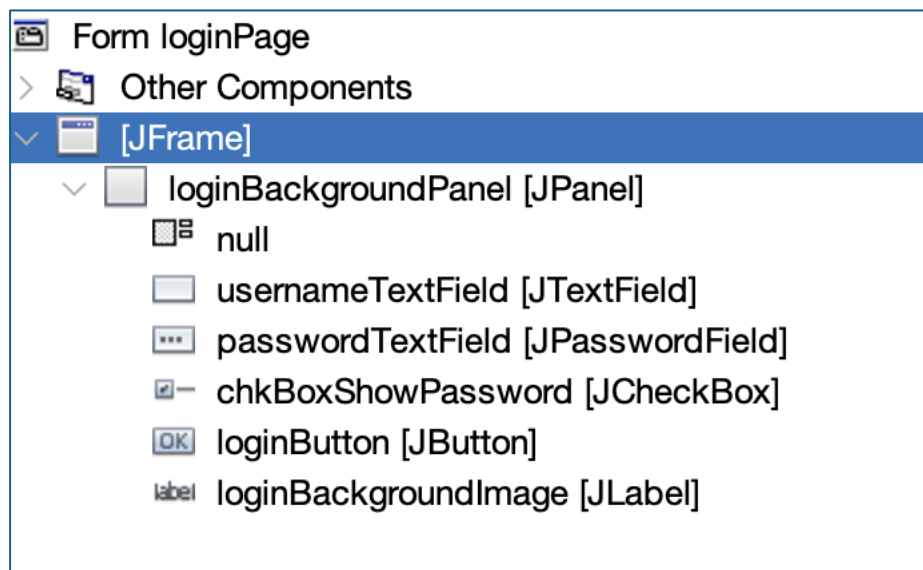


Figure 149: contents of login page

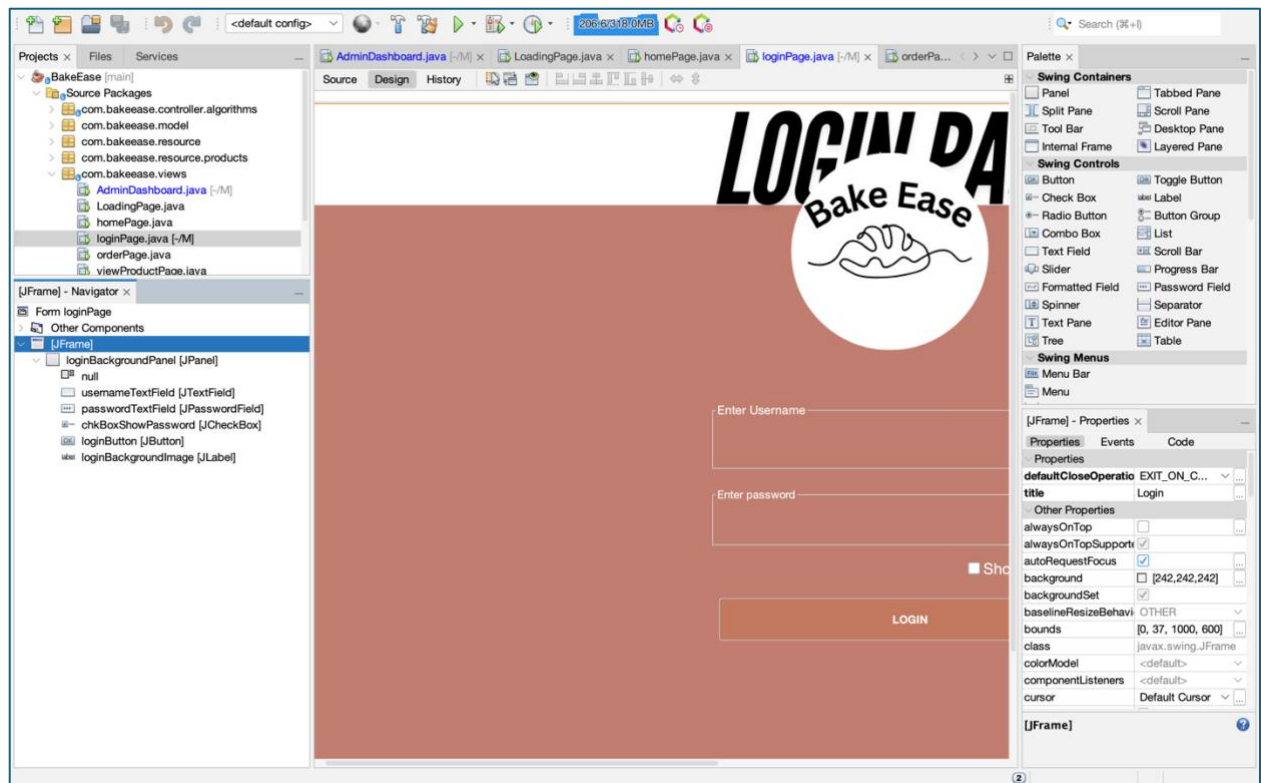


Figure 150: designing home page

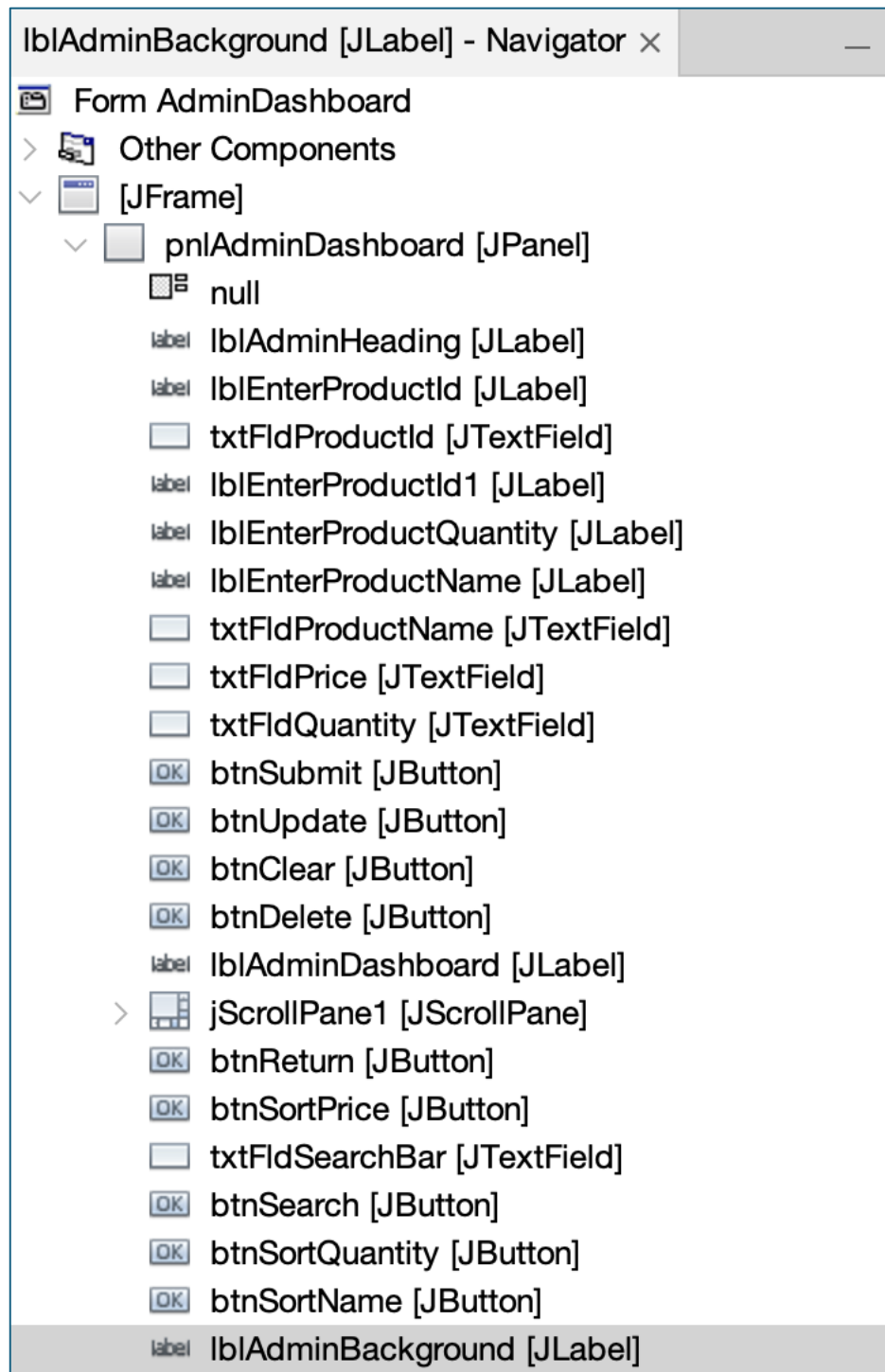


Figure 151: Contents of admin page

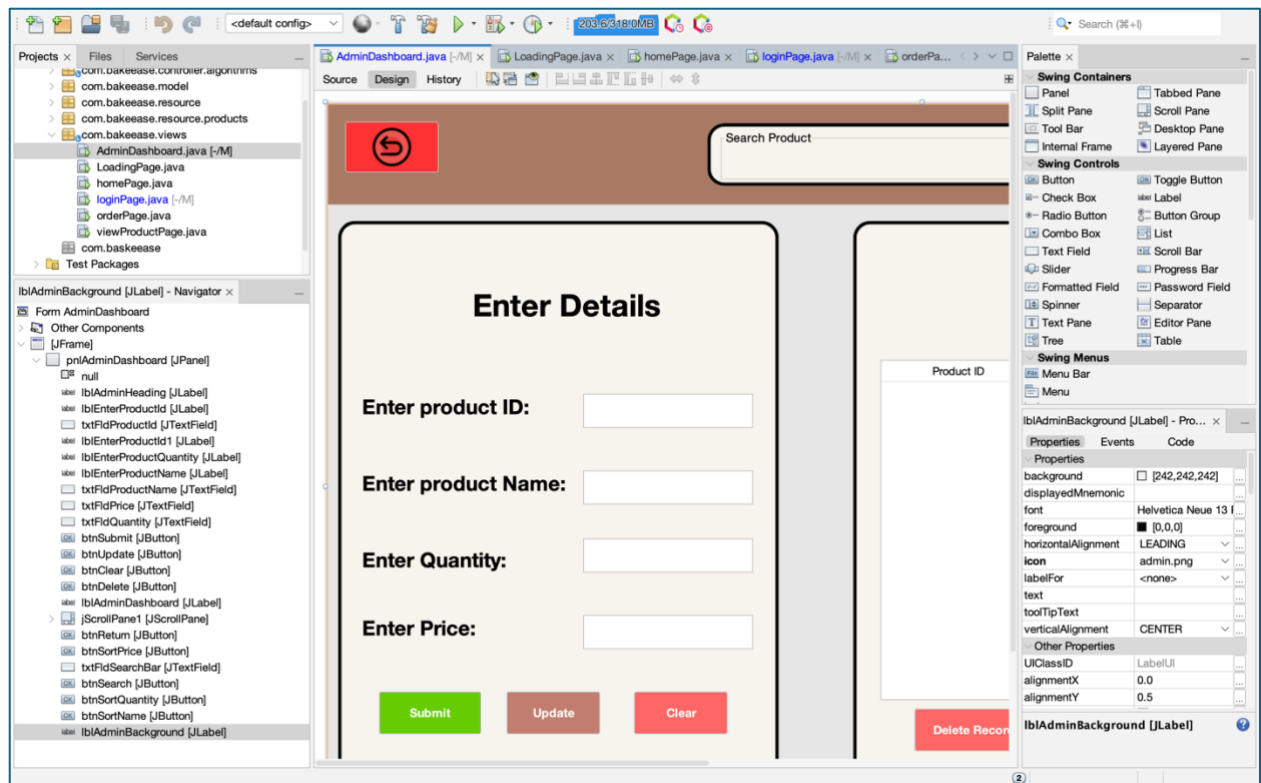


Figure 152: designing admin page

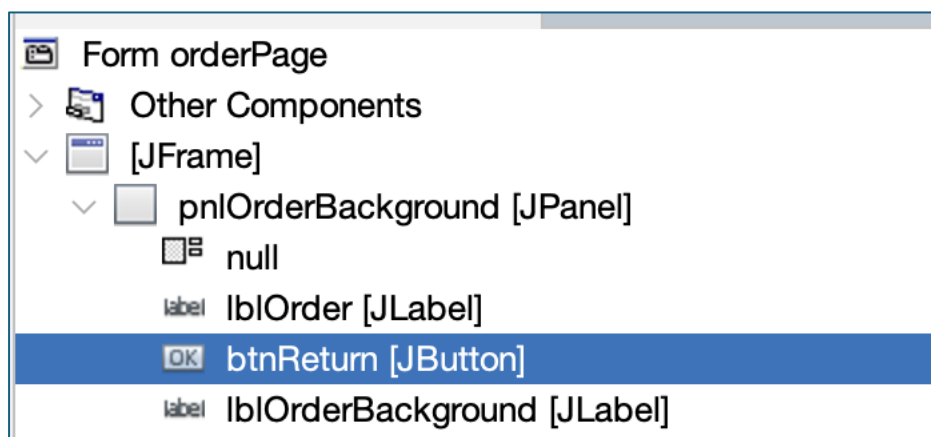


Figure 153: contents of order page

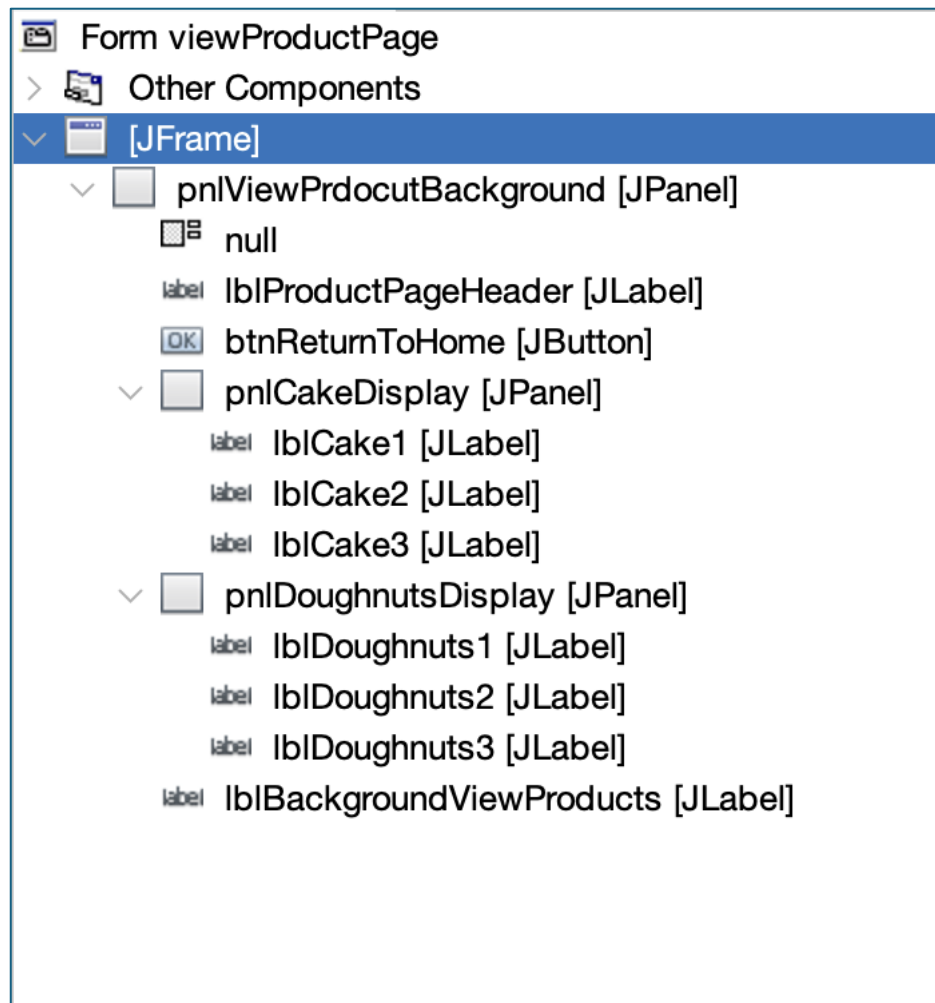


Figure 154: Contents of product page

The order page and product page are yet to be built. In the current context this coursework doesn't require the GUI to be built entirely and this project could not be completed solely. So, further this application will be built by the help of open-source contribution from other developers.

6.2 CRUD Operations

CRUD operations play a crucial role in developing a dynamic system. CRUD operations bring features like creating, reading, updating and deleting which makes system more user friendly and helps bring customers. While performing CRUD operations in the system, adding data to player table, removing data from the table and updating existing information from the table were some of the features and following evidence show the problems encountered and their solutions.

6.2.1 Adding data to admin dashboard

While developing the admin page of the system, it felt difficult to insert data into the system. Initially we created a linked list where all the data retrieved from the text fields are stored into an object and that object adds the retrieved data to the model which inserts into table row. Then, to insert the data into the table, it is supposed to import default table model component of java swing collection framework, create a model of admin table and then we are supposed to enter the retrieved data into that specific model.

Initially I faced problem regarding using table model, and didn't understand the concept of using table model. So, I went on internet to search solutions for this problem, and I ended up taking reference from a YouTube tutorial that explained how to insert data into the table (blog, n.d.) and took reference from the GitHub repository provided to us by our tutor which had a similar project package that used same architecture. (Maharjan, n.d.)

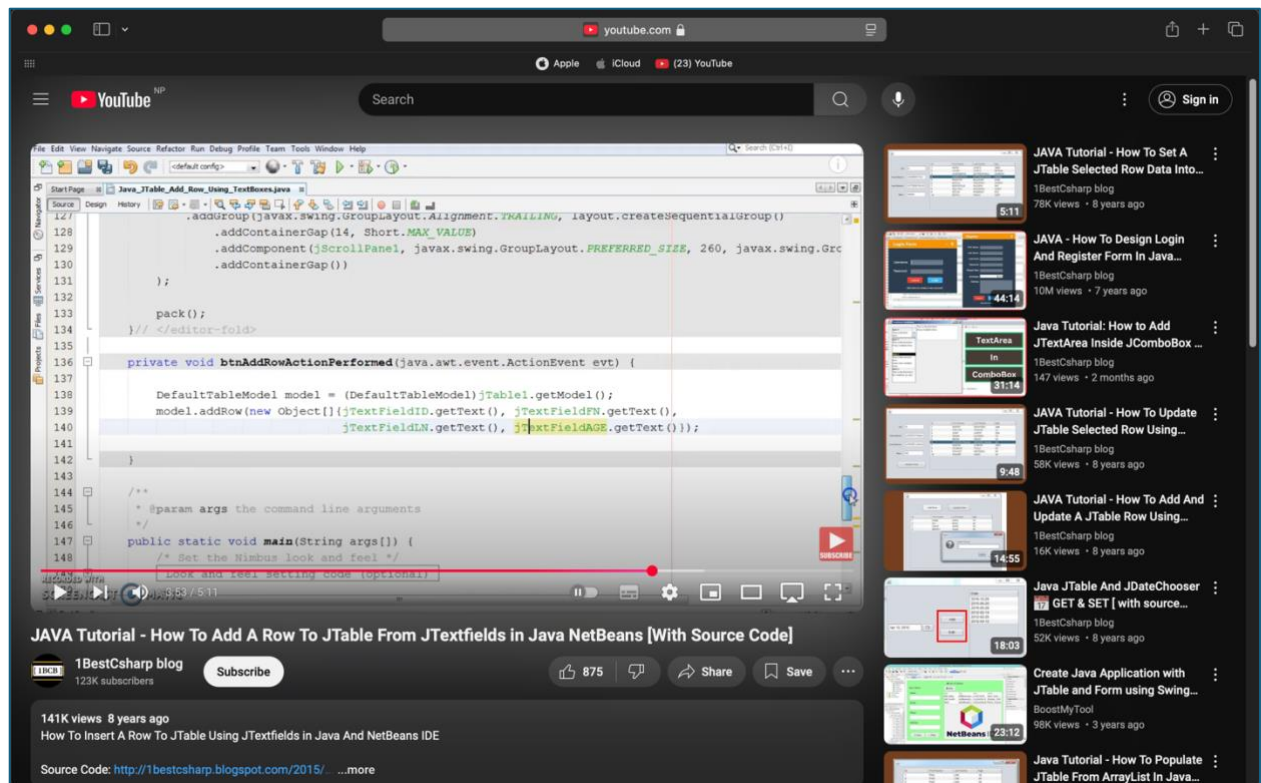


Figure 155: Evidence of using youtube tutorial

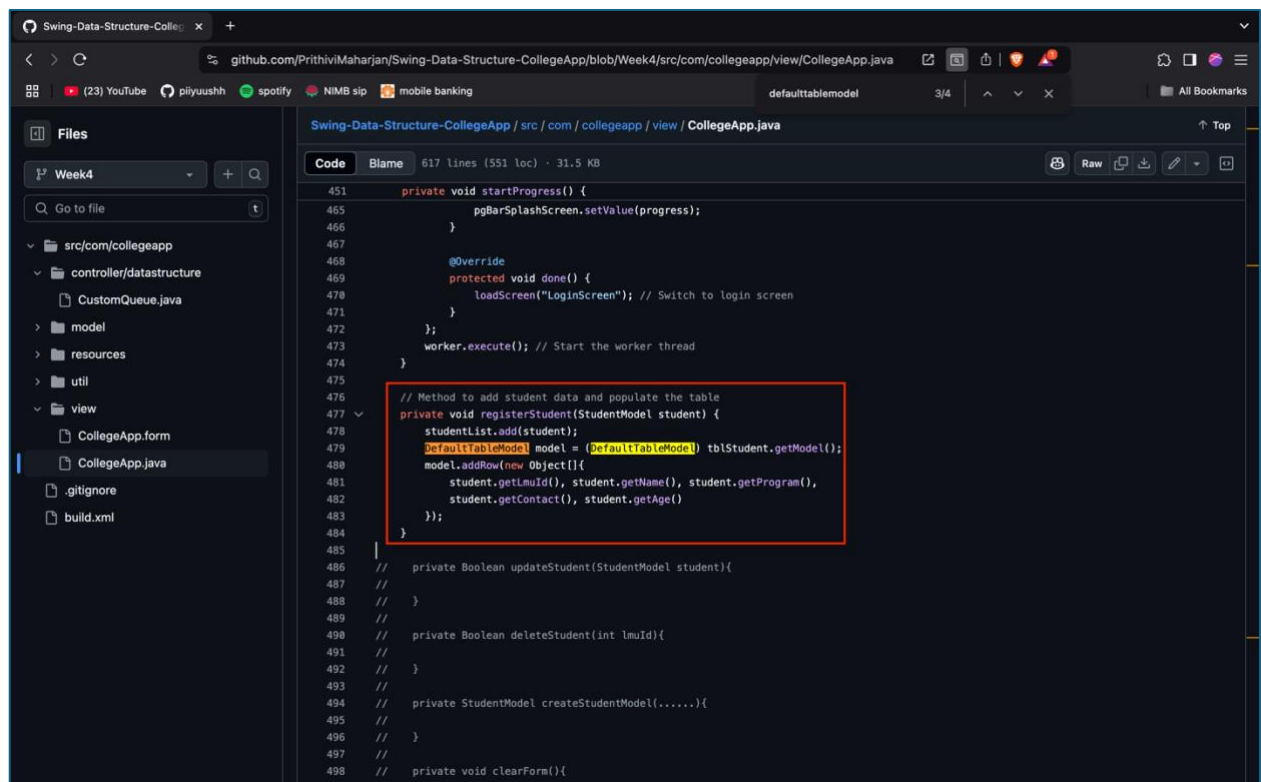


Figure 156: evidence of using GitHub as reference

6.2.1.1 Solution to the challenge

First, we import DefaultTableModel java swing component.

```
package com.bakeease.views;

import com.bakeease.model.adminModel;
import java.util.Iterator;
import javax.swing.JOptionPane;
import java.util.LinkedList;
import javax.swing.table.DefaultTableModel;
```

Figure 157: importing default table model

After that we use that component to create a table model for admin table.

```
// method to add bakery Items to table row
public void addBakeryItems(adminModel bakeryItems) {
    productList.add(bakeryItems);
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    Object[] bakeryData = new Object[]{
        bakeryItems.getProductName(), bakeryItems.getQuantity(), bakeryItems.getPrice()
    };
    model.addRow(bakeryData);
}
```

Figure 158: creating table model

Syntax breakdown: **DefaultTableModel** is a class in Swing's javax.swing.table package that provides a default implementation for a table model. It stores data in a tabular format.

Model is a local variable of **DefaultTableModel**. It helps to perform table actions like adding data into rows and columns. **adminTable** is the variable name of the table in which data is supposed to be inserted.

After the data is stored in the model then the data is further added to the table row using a function "addRow" function.

```
// method to add bakery Items to table row
public void addBakeryItems(adminModel bakeryItems) {
    productList.add(bakeryItems);
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    Object[] bakeryData = new Object[]{
        bakeryItems.getProductName(), bakeryItems.getQuantity(), bakeryItems.getPrice()
    };
    model.addRow(bakeryData);
}
```

Figure 159: adding data to model

Here, the data is retrieved from the private method of the button action performed. The data is further sent to model and further an object is created which stores information retrieved from admin table to store information and then further that object is passed to the linked list.

6.2.1.2 Screenshots of adding data

Admin

Search Product

Enter Details

Enter product ID:

Enter product Name:

Enter Quantity:

Enter Price:

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Figure 160: screenshot before adding data

Admin

Search Product

Enter Details

Enter product ID:

Enter product Name:

Enter Quantity:

Enter Price:

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
49	Banana	2	120

Success
Product added successfully.

Figure 161: Evidence of adding data

6.2.2 Deleting data from admin dashboard

To delete data from the table row it requires a function named “getSelectedRow()” to select the row from the table which is supposed to be deleted. To delete the data from the table, row the user is supposed to select the row from the table and then press the delete button which deletes the data from the table. There are many ways to delete data from the table but using this method felt practical and more useful in the current scenario of this project.

Just like adding data in the table. I didn't know about this process and according to our project requirement, we are supposed to perform CRUD operation which includes deletion of data too and then again, I went on internet to find the solution to this problem. After few hours of searching for solutions, I came across a YouTube tutorial to solve this problem. (YouTube, n.d.) After taking help from the YouTube tutorial, I tried to test it out by myself.

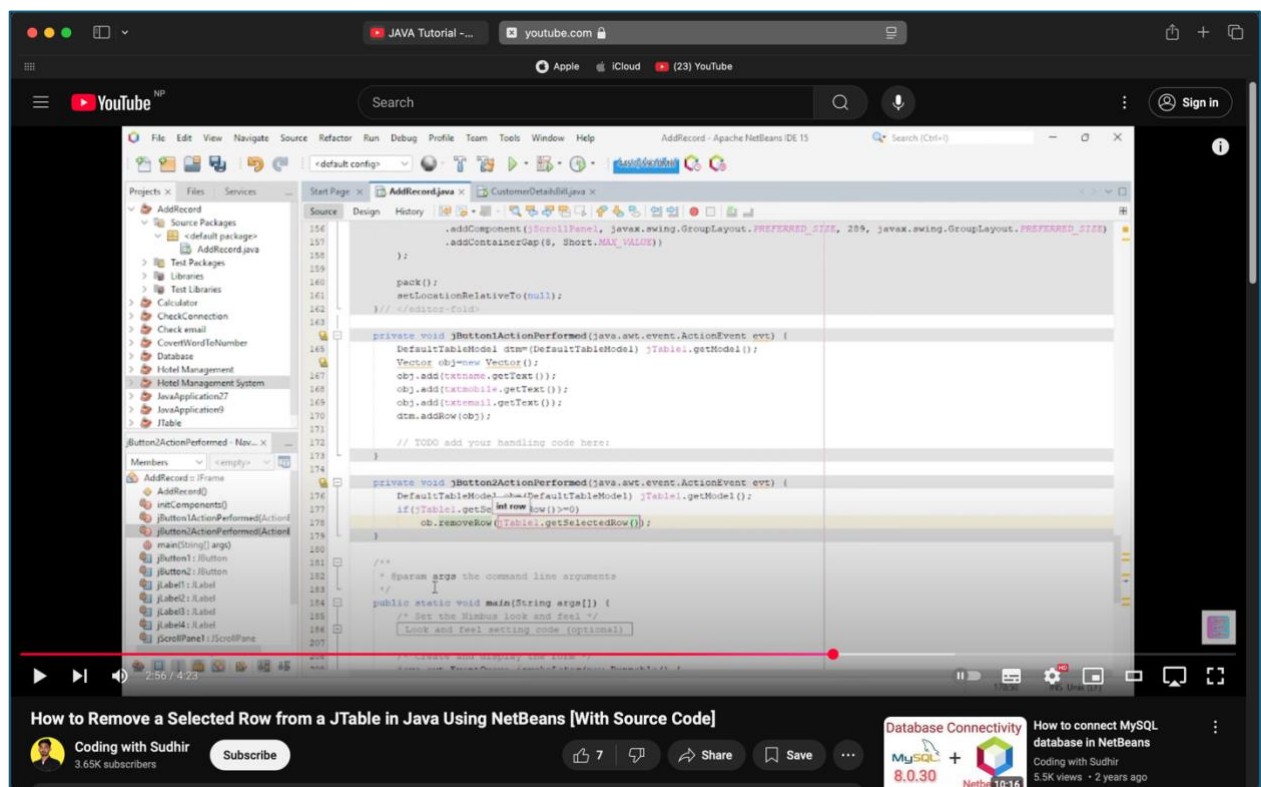


Figure 162: evidence of using youtube tutorial for deleting data

6.2.2.1 Solution to the problem

Just like adding data to the table, here also it requires to import defaultTableModel. It is essential for these operations to be performed.


```

package com.bakeease.views;

import com.bakeease.model.adminModel;
import java.util.Iterator;
import javax.swing.JOptionPane;
import java.util.LinkedList;
import javax.swing.table.DefaultTableModel;

```

Figure 163: importing default table model

After importing defaultTableModel, we created a private method which handles deletion of data from the row.

```

// using a private method to delete data from table row
private void btnDeleteRowActionPerformed(java.awt.event.ActionEvent evt) {
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    int selectedRow = adminTable.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(null, "Select a row to be deleted.", "Error", JOptionPane.ERROR_MESSAGE);
    } else {
        model.removeRow(selectedRow);
        JOptionPane.showMessageDialog(null, "Row deleted succesfully!", "Row Deleted", JOptionPane.INFORMATION_MESSAGE);
    }
}

```

Figure 164: creation of table method

Admin

Search Product

Enter Details

Enter product ID: 49

Enter product Name: Banana

Enter Quantity: 2

Enter Price: 120

Submit Update Clear

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
49	Banana	2	120

Delete Record Sort by Quantity Sort by Name Sort by Price

Error: Select a row to be deleted.

Figure 165: select a row to be deleted





Enter Details

Enter product ID:

Enter product Name:

Enter Quantity:

Enter Price:

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Row Deleted

 Row deleted successfully!

Figure 166: deleting data from table row

6.2.3 Updating data from admin dashboard

To update existing data from the table, it requires to check duplicate data in the table first just to be confirm and then it changes the data in the list.

6.2.3.1 Solution to the problem

```
/*
method to update table item
@param product id
@param product name
@param product quantity
@param price
*/
private void updateItems() {
    DefaultTableModel model = (DefaultTableModel) adminTable.getModel();
    // Clear the table before loading new data
    // set model row count to 0 to locate to exact updation location
    model.setRowCount(0);
    // add row for every fields
    for (adminModel products : productList) {
        model.addRow(new Object[]{
            products.getProductid(),
            products.getProductName(),
            products.getQuantity(),
            products.getPrice()
        });
    }
}
```

Figure 167: creating update method

Here, we first set the model count to 0. Then, we retrieve the product id, product name, product quantity and product price. After that we move to action listener method to change the current existing product information.

```

/*
update button method
*/
private void btnUpdateActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int Id = Integer.parseInt(txtFldProductId.getText());
    String Name = txtFldProductName.getText();
    int price = Integer.parseInt(txtFldPrice.getText());
    int quantity = Integer.parseInt(txtFldQuantity.getText());

    boolean isValid = true;

    if (isValid) {
        for (adminModel menu : productList) {
            // Matching product id
            if (menu.getProductID() == Id) {
                menu.setProductName(Name);
                menu.setQuantity(quantity);
                menu.setPrice(price);
                menu.setProductID(Id);

                // updates product details
                // calling above update method
                updateItems();
                JOptionPane.showMessageDialog(null, "Product information updated succesfully.", "Updated", JOptionPane.INFORMATION_MESSAGE);
                return;
            }
        }
    }
}

```

Figure 168: update button action listener method

Admin

Search Product

Enter Details

Enter product ID:

Enter product Name:

Enter Quantity:

Enter Price:

Admin Dashboard

Product ID	Name	Quantity	Price
1	Crossiant	22	420
2	Cake	12	300
3	Pastry	10	200
4	Cheese Cake	14	69
5	Rumballs	7	690
6	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
49	apple	2	120

Figure 169: targeted product to update information

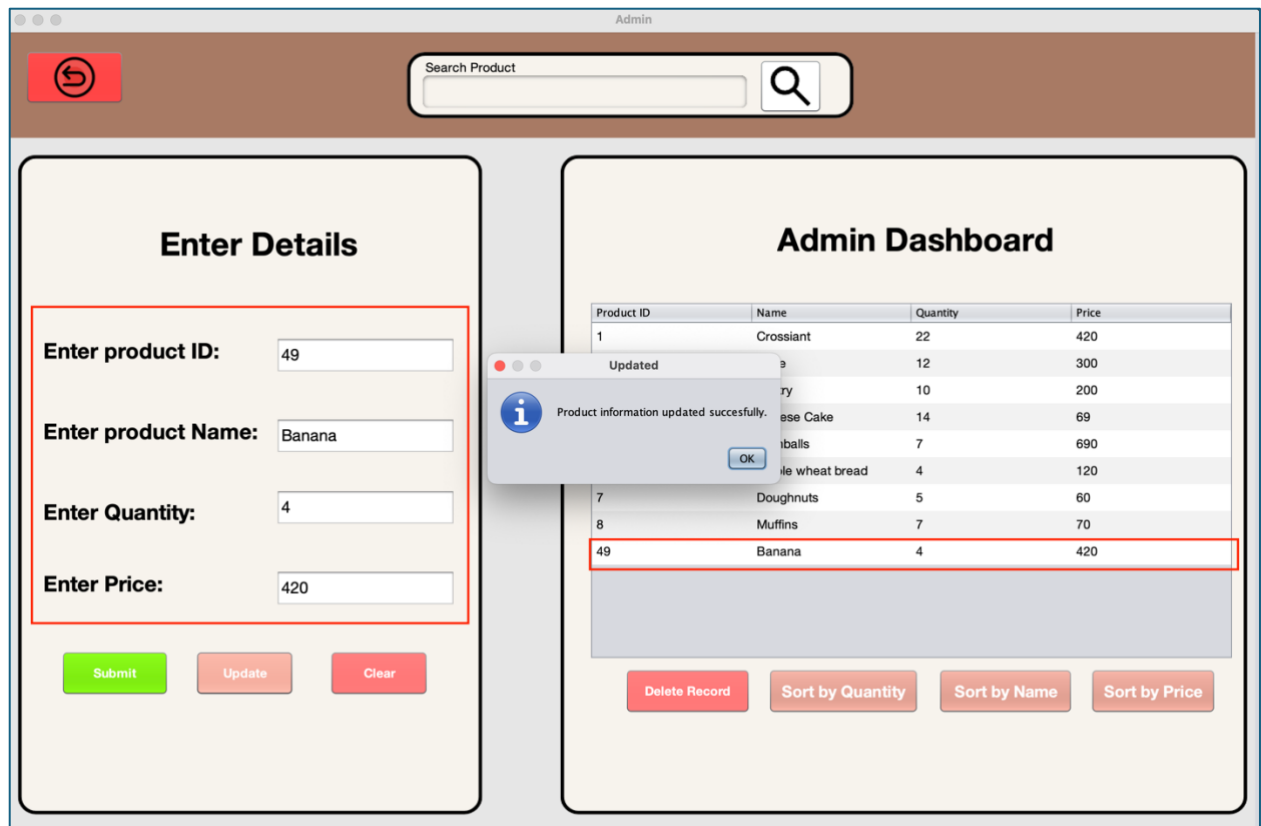


Figure 170: updating product information from GUI

6.3 Validations

Validations play a crucial role in improving user experience. Validations improve user experience by instructing user to enter correct credentials. Validations ensure that only required information goes into the system and ensures data integrity.

6.3.1 Login Page Validation

In the login page, there are three text fields regarding username and password and a checkbox to show password and a login button. The system default username is set to “admin” and default password is also set to “admin”. Hence if user doesn’t enter anything in the text fields a message will appear which will instruct user to enter credentials. Similarly, if wrong credentials are entered the user will be instructed to enter correct credentials and after user enters correct credentials, welcome message is shown when home screen appears.

6.3.2 Empty Fields Validation

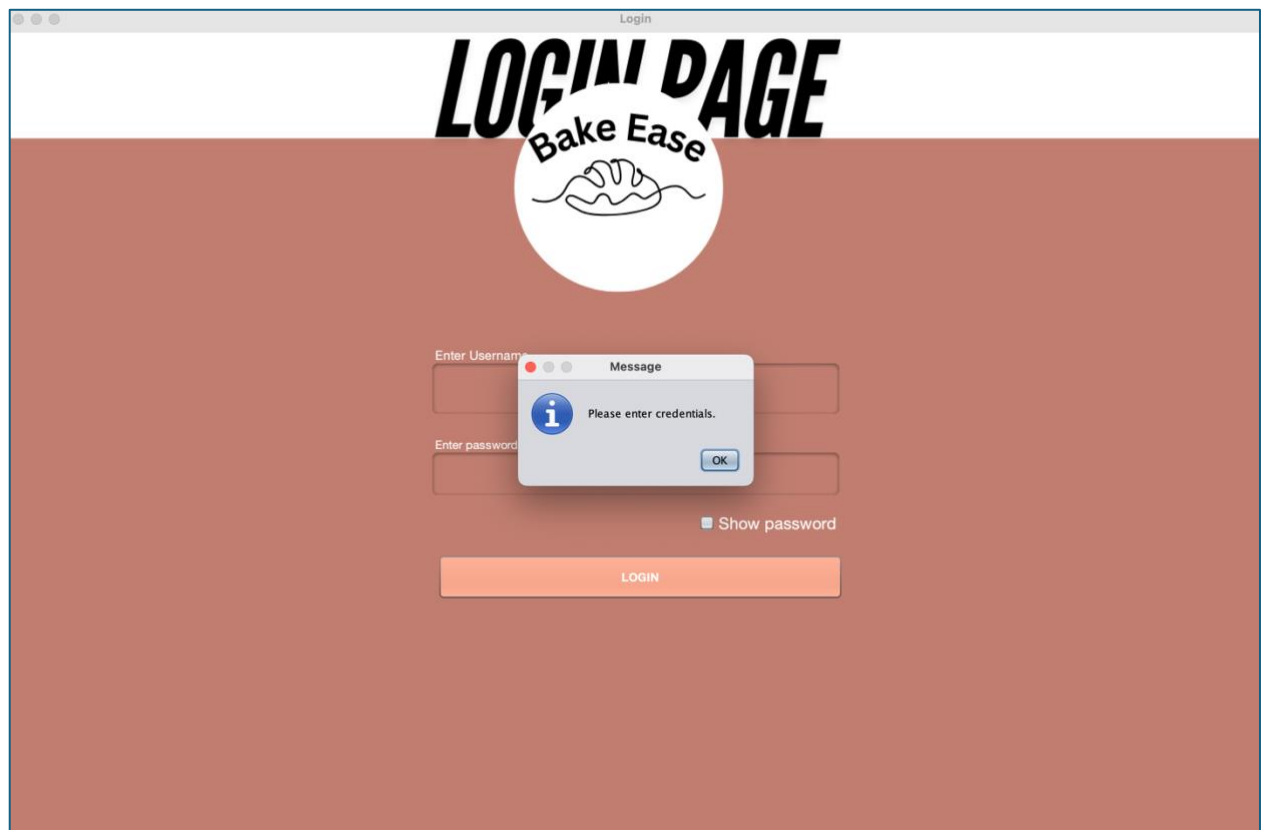


Figure 171: empty fields validation in login page

6.3.3 Incorrect Credentials Validation

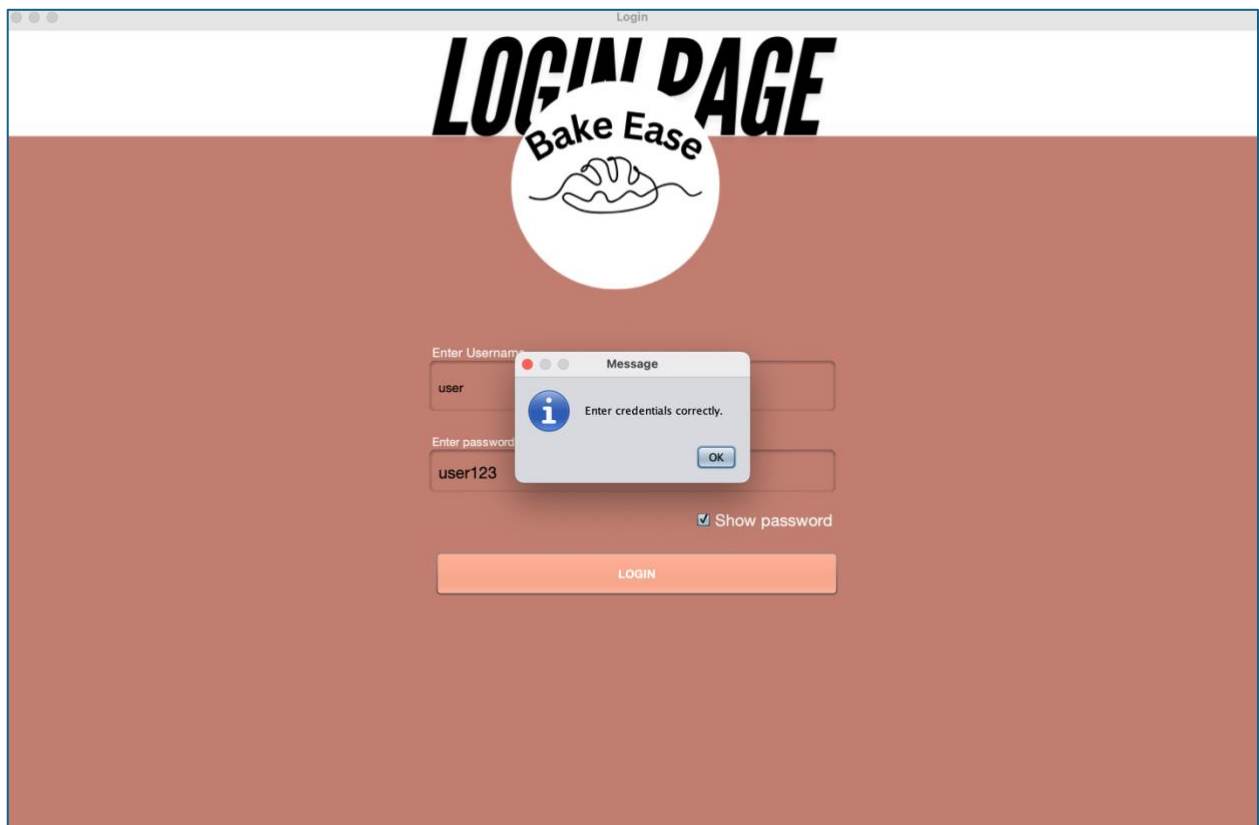


Figure 172: incorrect credentials validation

6.3.4 Correct Credentials Validation

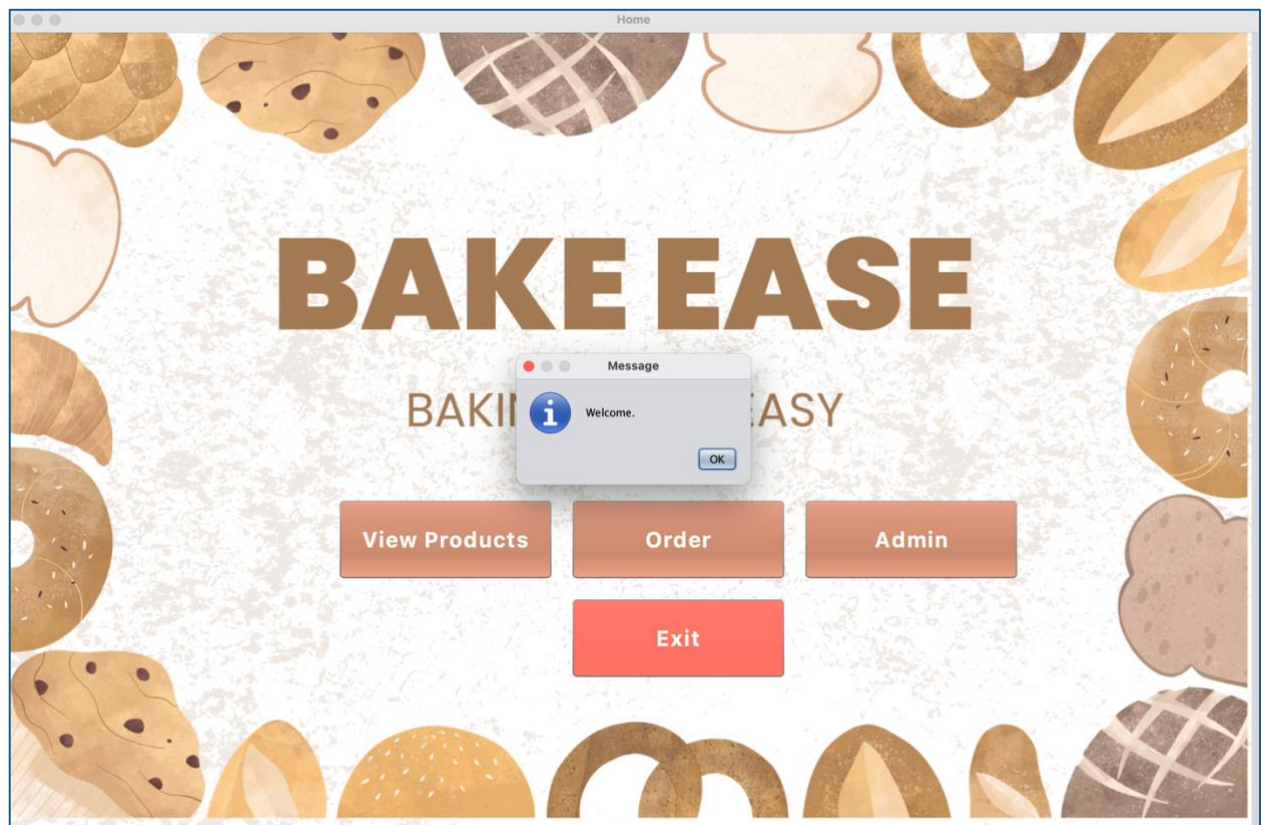
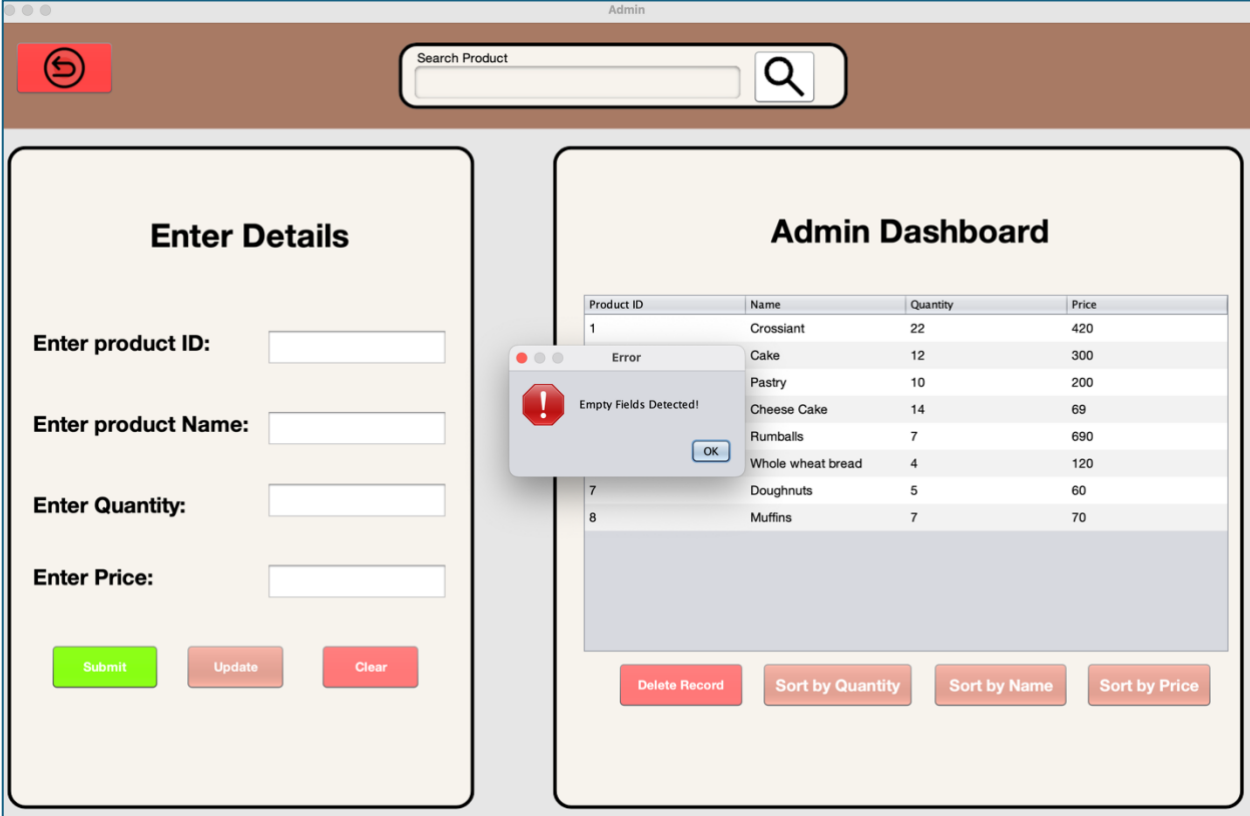


Figure 173: After correct credentials validation

6.3.2 Admin Dashboard Validation

In admin dashboard page we have validated the text fields which are supposed to contain integer values such as product id, product price and product quantity. Then, further validation is done in clear button and delete button. In clear button, user is confirmed if text fields are to be cleared or not. If user chooses YES, the text fields are cleared else text fields are not cleared. In delete button the user is instructed to select a row to be deleted else data cannot be deleted. Messages are shown for each validation performed.

6.3.2.1 Admin Empty Fields Validation



The screenshot shows the Admin Dashboard interface. On the left is the 'Enter Details' form with four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). On the right is the 'Admin Dashboard' section featuring a table of products. An 'Error' dialog box is overlaid on the table, displaying a red exclamation mark icon and the message 'Empty Fields Detected!' with an 'OK' button.

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange).

Figure 174: empty fields validation in admin page

6.3.2.2 Incorrect data type validation

The screenshot displays an Admin Dashboard with two main sections: 'Enter Details' and 'Admin Dashboard'.

Enter Details Section:

- Enter product ID:** user
- Enter product Name:** user
- Enter Quantity:** user
- Enter Price:** user

Buttons: Submit (green), Update (orange), Clear (red).

Admin Dashboard Section:

Product ID	Name	Quantity	Price
1	Crossiant	22	420
		12	300
		10	200
	ke	14	69
		7	690
	at bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70

Buttons: Delete Record (red), Sort by Quantity (orange), Sort by Name (orange), Sort by Price (orange).

Error Message: Product Id, Quantity and Price must be in numeric form! (OK)

Figure 175: incorrect data type validation

6.3.2.3 Correct data type validation

The screenshot displays an Admin Dashboard with two main sections: 'Enter Details' and 'Admin Dashboard'. The 'Enter Details' section contains four input fields: 'Enter product ID:' (12), 'Enter product Name:' (banana), 'Enter Quantity:' (2), and 'Enter Price:' (120). Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). The 'Admin Dashboard' section features a table with columns: Product ID, Name, Quantity, and Price. The table lists several products, including Crossiant, Cake, Pastry, Cheese Cake, Rumballs, Whole wheat bread, Doughnuts, and Muffins. A modal window titled 'Success' is overlaid on the table, displaying the message 'Product added succesfully.' with an 'OK' button. At the bottom of the dashboard, there are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange).

Product ID	Name	Quantity	Price
1	Crossiant	22	420
	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
7	Doughnuts	5	60
8	Muffins	7	70
12	banana	2	120

Figure 176: correct data type validation in admin page

6.3.2.4 Clear text fields validation

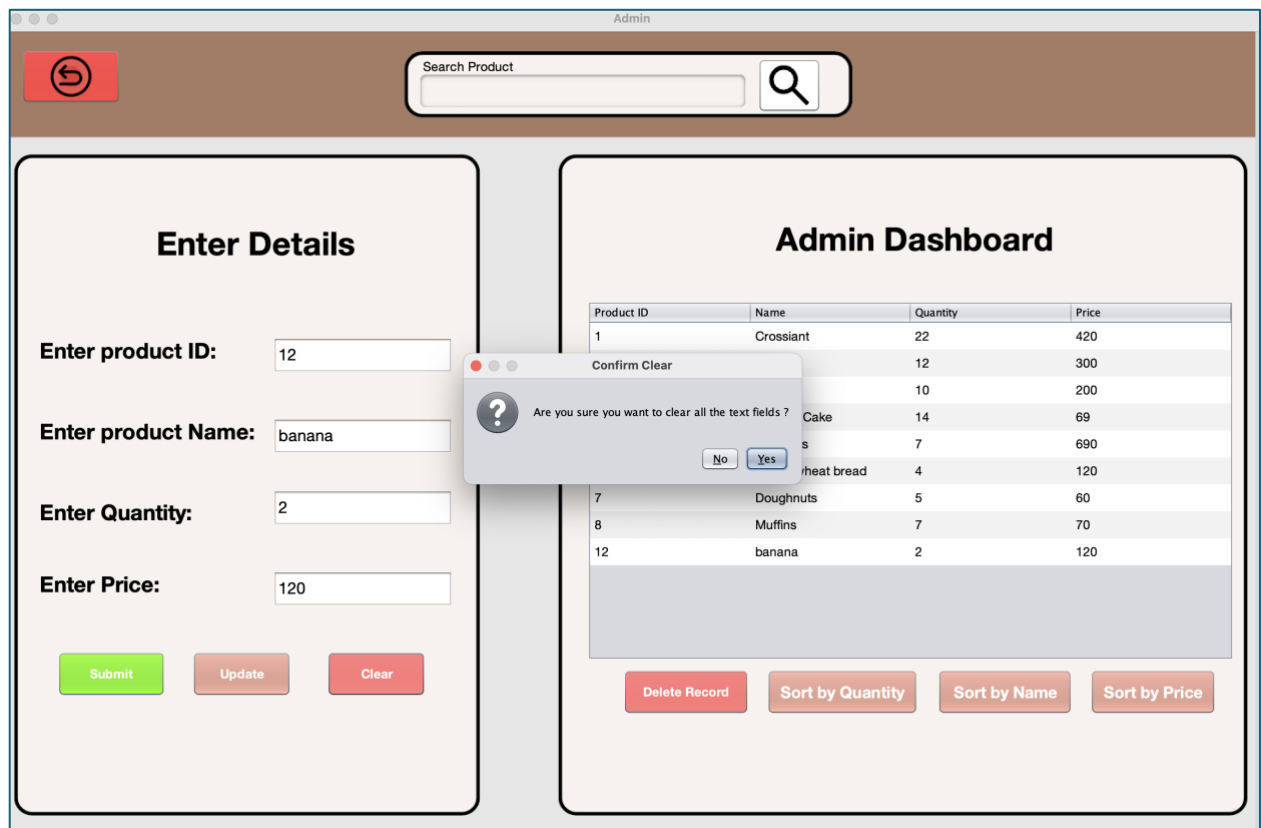
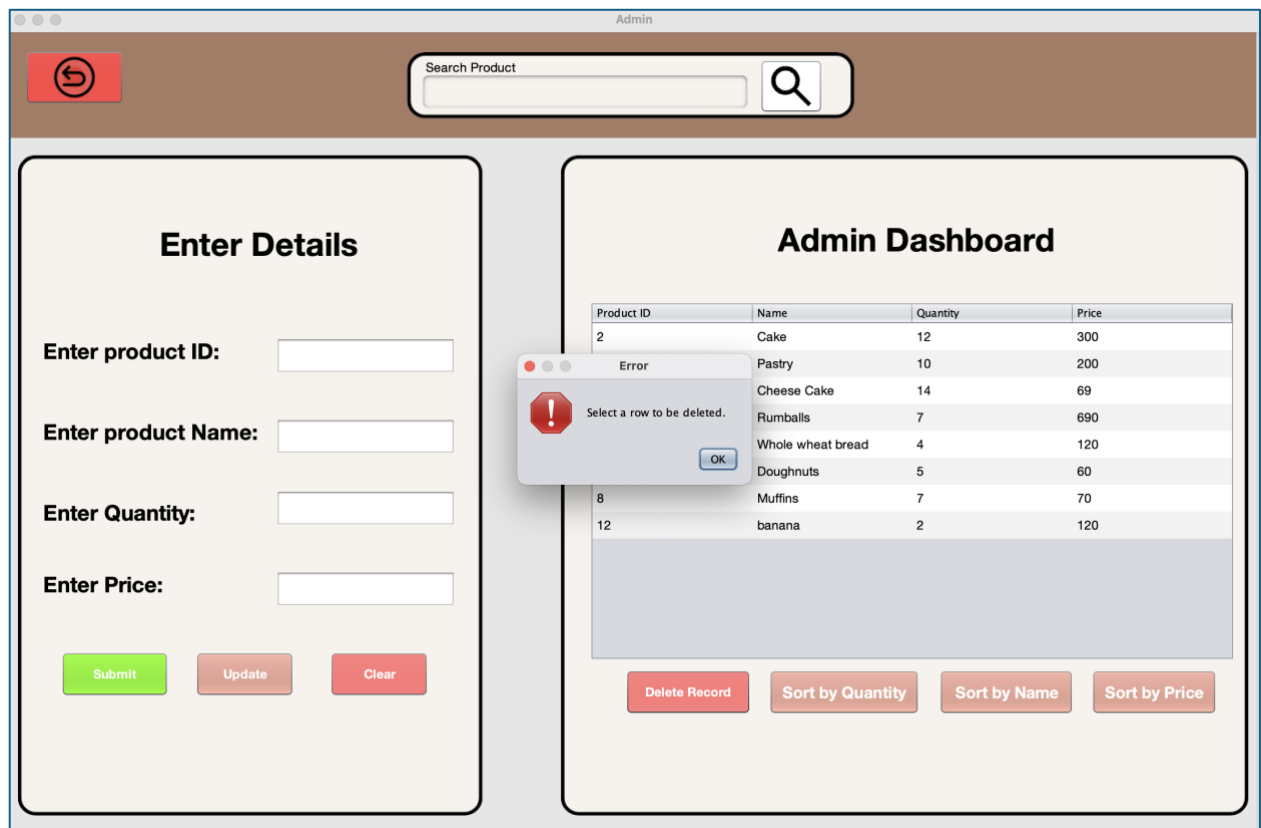


Figure 177: clear text fields validation in admin page

6.3.2.5 Delete data validation



The screenshot displays an 'Admin' interface with two main panels. The left panel, titled 'Enter Details', contains four input fields: 'Enter product ID:', 'Enter product Name:', 'Enter Quantity:', and 'Enter Price:'. Below these fields are three buttons: 'Submit' (green), 'Update' (orange), and 'Clear' (red). The right panel, titled 'Admin Dashboard', features a table with the following data:

Product ID	Name	Quantity	Price
2	Cake	12	300
	Pastry	10	200
	Cheese Cake	14	69
	Rumballs	7	690
	Whole wheat bread	4	120
	Doughnuts	5	60
8	Muffins	7	70
12	banana	2	120

Below the table are four buttons: 'Delete Record' (red), 'Sort by Quantity' (orange), 'Sort by Name' (orange), and 'Sort by Price' (orange). An 'Error' dialog box is overlaid on the table, displaying a red exclamation mark icon and the text 'Select a row to be deleted.' with an 'OK' button.

Figure 178: delete data validation in admin page

6.3.2.6 Search data validation

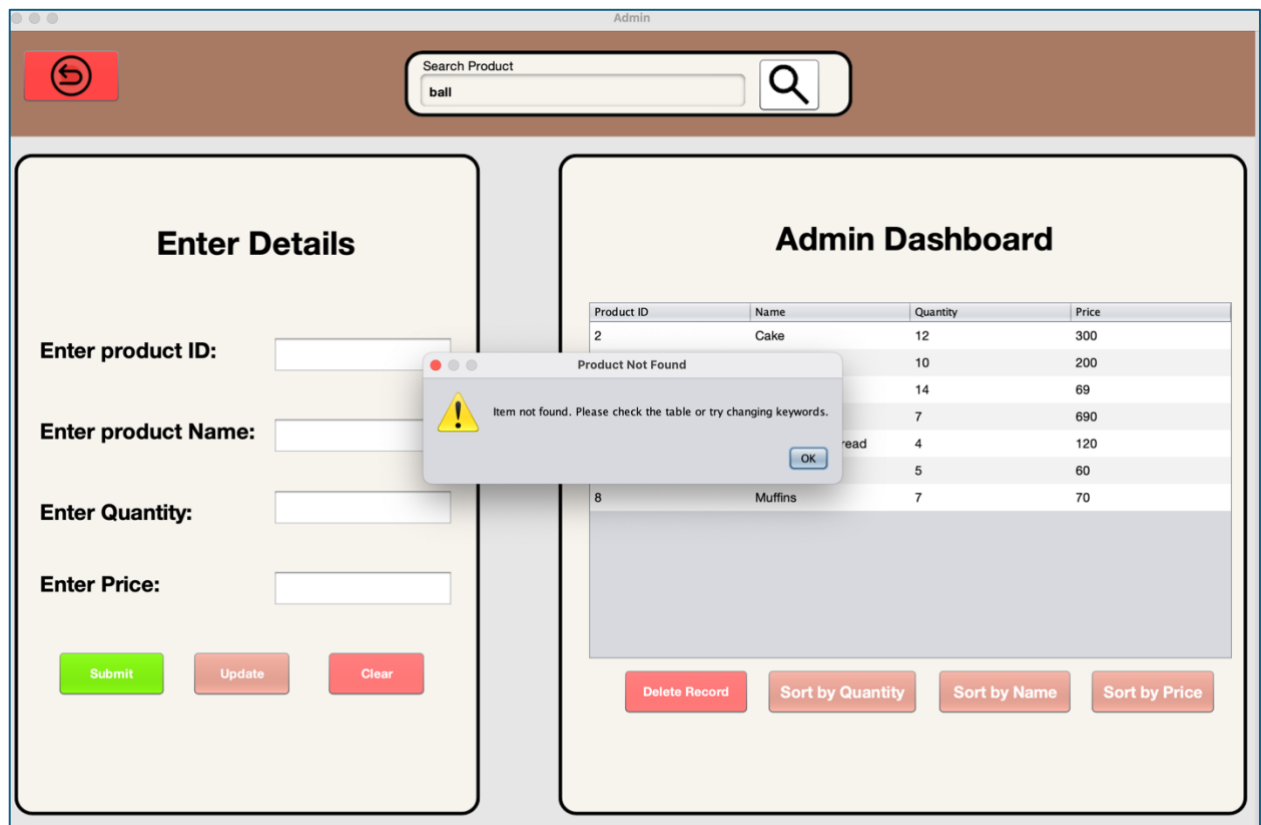


Figure 179: item not found in search bar

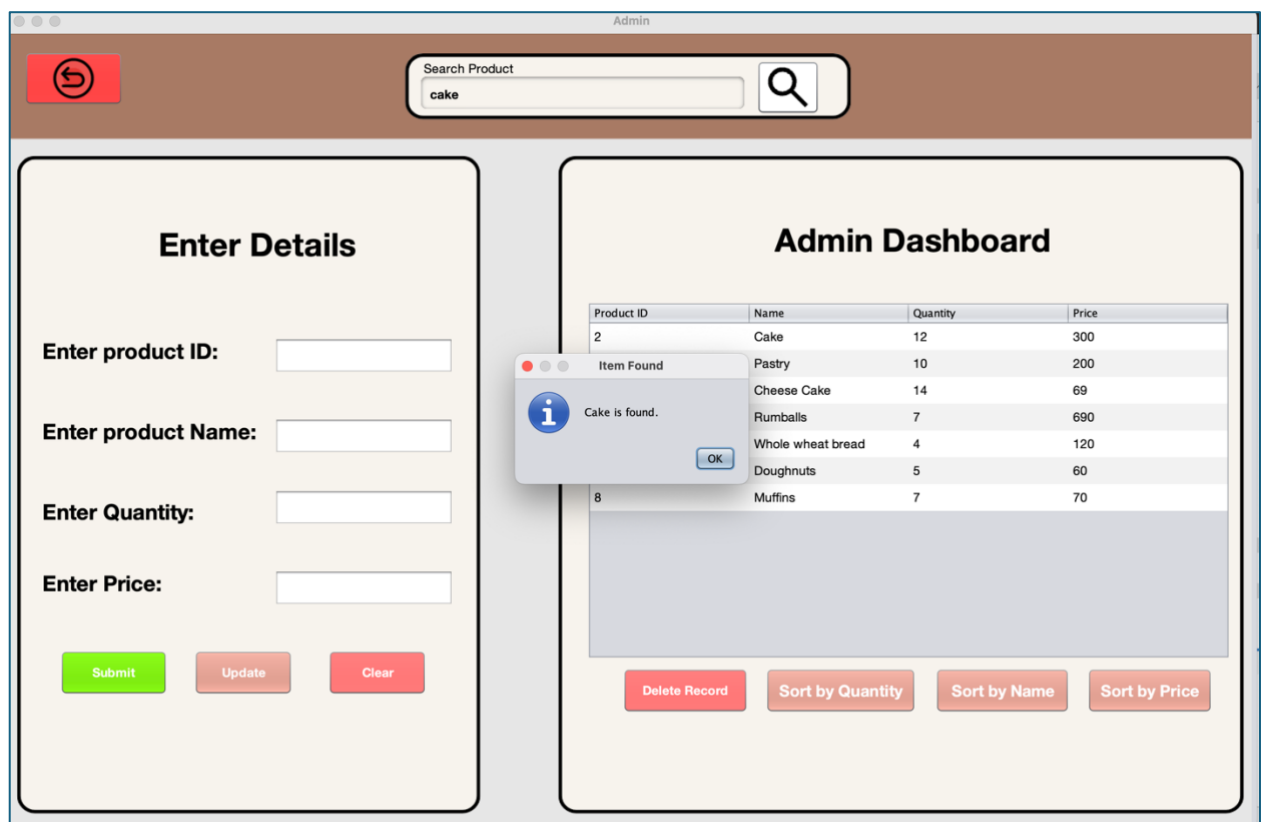


Figure 180: item found validation in search bar

6.4 Sorting

Sorting algorithm is used to rearrange the set of data in an order. It is library implementation in Java program. (Geeks, n.d.)

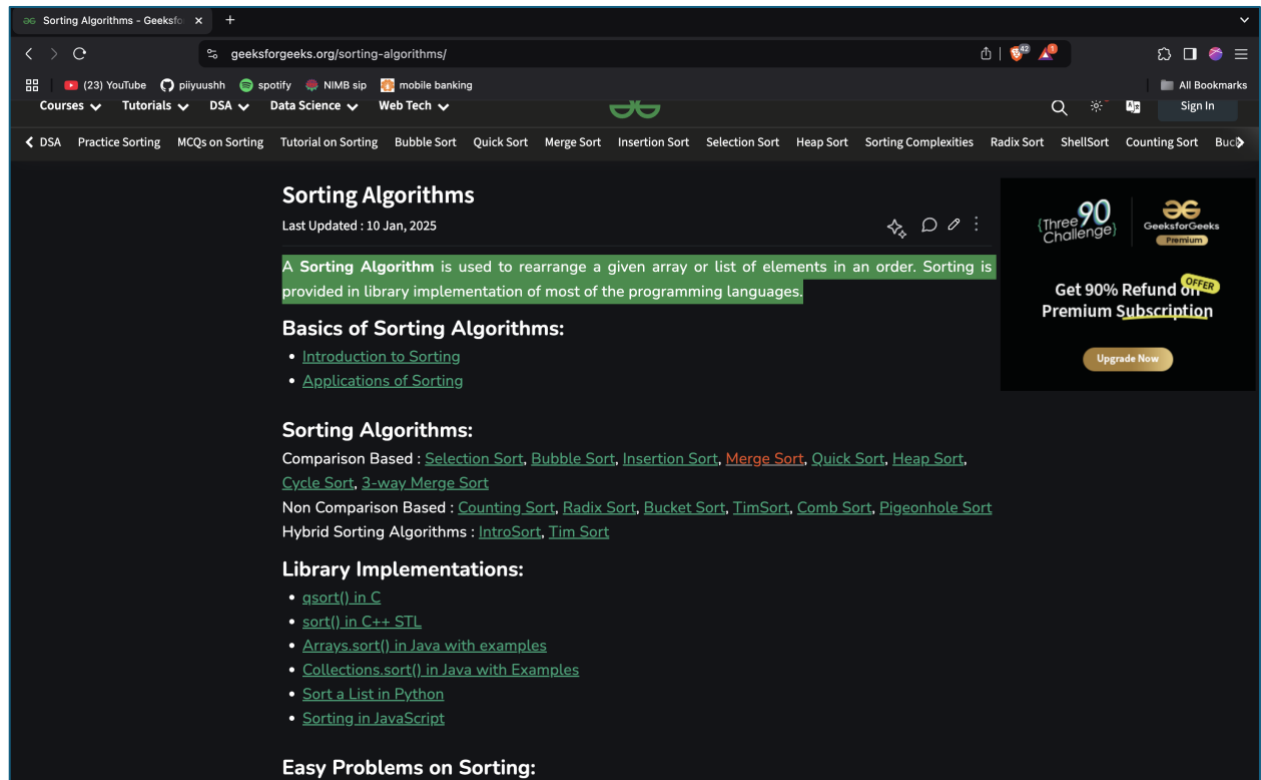


Figure 181: evidence of sorting algorithm citation

6.4.1 Merge Sort

Merge sort is a sorting algorithm that follows the divide and merge approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array. (Geeks, n.d.)

```

/**
public class MergeSort {
    /**
     * Sorts a list of adminModel objects by product name.
     *
     * @param product the list of adminModel objects to be sorted
     * @param isDesc Boolean indicating whether to sort in descending order (true)
     *                or ascending order (false)
     * @return the sorted list of adminModel objects
     * @throws IllegalArgumentException if the provided list is null or empty
     */
    public List<adminModel> sortByName(List<adminModel> product, boolean isDesc) {
        if (product == null || product.isEmpty()) {
            throw new IllegalArgumentException("Product list is empty. Fix it!");
        }

        return mergeSort(product, isDesc);
    }

    /**
     * Recursively divides and sorts the list of adminModel objects using Merge Sort.
     *
     * @param product the list of adminModel items to be sorted
     * @param isDesc Boolean indicating the sort order
     * @return the sorted list of adminModel objects
     */
    private List<adminModel> mergeSort(List<adminModel> product, boolean isDesc) {
        if (product.size() <= 1) {
            return product;
        }

        int mid = product.size() / 2;
        List<adminModel> left = new ArrayList<>(product.subList(0, mid));
        List<adminModel> right = new ArrayList<>(product.subList(mid, product.size()));

        // Recursively split and merge
        left = mergeSort(left, isDesc);
        right = mergeSort(right, isDesc);

        return merge(left, right, isDesc);
    }
}

```

Figure 182: implementing Merge sort in program (i)


```
/**
 * Merges two sorted sub lists into a single sorted list.
 *
 * @param left the left sub list
 * @param right the right sub list
 * @param isDesc Boolean indicating the sort order
 * @return the merged and sorted list
 */
private List<adminModel> merge(List<adminModel> left, List<adminModel> right, boolean isDesc) {
    List<adminModel> merged = new ArrayList<>();
    int i = 0, j = 0;
    // Merge based on the model names
    while (i < left.size() && j < right.size()) {
        if (shouldPlaceLeft(left.get(i).getProductName(), right.get(j).getProductName(), isDesc)) {
            merged.add(left.get(i));
            i++;
        } else {
            merged.add(right.get(j));
            j++;
        }
    }

    // Add remaining elements
    while (i < left.size()) {
        merged.add(left.get(i));
        i++;
    }

    while (j < right.size()) {
        merged.add(right.get(j));
        j++;
    }

    return merged;
}
```

Figure 183: creating sub lists

6.4.2 Insertion Sort

Insertion sort is used in this program to sort the bakery items based on their quantity from high to low. Insertion sort is a simple sorting algorithm that puts together the final sorted array element by element. It loops through an array and extracts one component from the beginning data to insert it into the right place at the already sorted section of the array. The concept behind insertion sort is to retain a sorted sublist in the lower positions of the array and, at the same time, move the new elements to their right place by inserting these into the sublist.

```
/*
insertion sort for sort by quantity
@param adminModel for model
creates object sortQuantity to handle sorting
Adds data to list
*/
private void btnSortQuantityActionPerformed(java.awt.event.ActionEvent evt) {
    // Create an instance of the InsertionSort class
    InsertionSort sortQuantity = new InsertionSort();

    // Ensure productList is properly populated with product data
    // Sort the product list by quantity (descending order in this case)
    List<adminModel> sortedList = sortQuantity.sortByQuantity(productList, true);

    // Load the sorted data into the table
    loadSortedProductData(sortedList);
}
```

Figure 184: implementing selection sort in the program

6.4.3 Selection Sort

Selection sort is used in this program to sort the items based on their price from high to low. As this is the default view in customer point of view, prices which are high are placed on the top and prices which are low are placed in the bottom.

```

/*
method to execute selection sort for price
Arranges item price ranging from high to low
@param adminModel for model
creates a object sort and handles sorting
Adds data to list
*/
private void btnSortPriceActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    SelectionSort sort = new SelectionSort();
    List<adminModel> sortedList = sort.sortByPrice(productList, true);
    loadSortedProductData(sortedList);
}

```

Figure 185: Implementing selection sort

6.4.4 Searching Algorithm

Searching algorithm is implemented to find the item based on its name despite its casing as every text is default converted into lower case. The searching algorithm implemented in this project is binary search. "Binary search is used to search a key element from multiple elements. Binary search is faster than linear search." (JavatPoint, 2025)

```

/*
button to execute search feature
instructs user to provide proper details in case product is not found
Else shows success message
Also prints message in console
*/
private void btnSearchActionPerformed(java.awt.event.ActionEvent evt) {
    MergeSort selectionSort = new MergeSort();
    List<adminModel> sortedList = selectionSort.sortByName(productList, false);
    BinarySearch search = new BinarySearch();
    adminModel searchedData = search.searchByProductName(txtFldSearchBar.getText().trim().toLowerCase(), sortedList, 0, sortedList.size() - 1);
    if (searchedData != null) {
        System.out.println(searchedData.getProductName());
        JOptionPane.showMessageDialog(null, searchedData.getProductName() + " is found.", "Item Found", JOptionPane.INFORMATION_MESSAGE);
    } else {
        System.out.println("Product not found.");
        JOptionPane.showMessageDialog(null, "Item not found. Please check the table or try changing keywords.", "Product Not Found", JOptionPane.ERROR_MESSAGE);
    }
}

```

Figure 186: implementing binary search

7. Conclusion

The **Bake Ease** project is a bakery management system developed using Java, incorporating data structures such as linked lists and array lists alongside sorting algorithms like insertion, selection, and merge sort. Built with a graphical user interface using Java Swing and AWT, the project adheres to the Model-View-Controller (**MVC**) architecture for modularity and maintainability. The system addresses key challenges faced by bakery owners by streamlining inventory management and improving user experience through features such as CRUD operations, sorting, and searching. Comprehensive testing and validation ensure the robustness and usability of the application, while the use of open-source tools like Apache NetBeans and GitHub fosters scalability and future enhancements. With its focus on user convenience and efficient management, **Bake Ease** achieves its objectives and demonstrates the effective application of data structures and algorithms in solving real-world problems.

8. Reference

NetBeans, n.d. *NetBeans*. [Online]
 Available at: <https://netbeans.apache.org/front/main/index.html>

Pintrest, 2024. *Pintrest*. [Online]
 Available at: <https://www.pinterest.com/pin/9218374223184999/>

Canva, 2024. *Canva*. [Online]
 Available at: <https://www.canva.com/>

reference, L. P., 2024. *Login Page reference*. [Online]
 Available at: <https://www.pinterest.com/pin/648025833897562195/>

Canva, n.d. *Canva*. [Online]
 Available at: <https://www.canva.com/>

Overflow, S., n.d. *Refernce of home page*. [Online]
 Available at: <https://stackoverflow.com/questions/76299413/how-can-i-put-one-treeview-in-multiple-windows-but-it-will-have-the-same-databas>

Canva, 3034. *Canva*. [Online]
 Available at: <https://www.canva.com/>

Canva, 2025. *Canva*. [Online]
 Available at: <https://www.canva.com/>

Anon., n.d. *Netbeans*. [Online]
 Available at: <https://netbeans.apache.org/front/main/index.html>

Netbeans, n.d. *Netbeans*. [Online]
 Available at: <https://github.com/apache/netbeans-antora-site>

Geeks, G. F., n.d. *Geeks For Geeks*. [Online]
 Available at: <https://www.geeksforgeeks.org/fundamentals-of-algorithms/>

Javatpoint, n.d. *Javatpoint*. [Online]
 Available at: <https://www.javatpoint.com/insertion-sort-in-java>

JavatPoint, n.d. *JavatPoint*. [Online]
 Available at: <https://www.javatpoint.com/selection-sort-in-java>

Geeks, G. f., n.d. *Geeks for Geeks*. [Online]
 Available at: <https://www.geeksforgeeks.org/merge-sort/>

Geeks, G. F., n.d. *Geeks For Geeks*. [Online]
 Available at: <https://www.geeksforgeeks.org/binary-search-in-java/>

Javatpoint, n.d. *Javatpoint*. [Online]
 Available at: <https://www.javatpoint.com/java-linkedlist>

Javatpoint, n.d. *Array list in java*. [Online]
 Available at: <https://www.javatpoint.com/java-arraylist>

blog, 1., n.d. *YouTube*. [Online]
 Available at: <https://youtu.be/F0Zq2fAUpXg?si=Lkg5YPKpGFaS5MTu>

Maharjan, P., n.d. *GitHub*. [Online]
 Available at: <https://github.com/PrithiviMaharjan/Swing-Data-Structure-CollegeApp>

YouTube, n.d. *YouTube*. [Online]
 Available at: <https://youtu.be/OICnHcXXmp8?si=AKNd70Qj2Srppuxz>

Geeks, G. f., n.d. *Sorting algorithm*. [Online]
 Available at: <https://www.geeksforgeeks.org/sorting-algorithms/>

JavatPoint, 2025. *Binary search*. [Online]
 Available at: <https://www.javatpoint.com/binary-search-in-java>

Geeks, G. f., 2023. *UI/UX design*. [Online]
Available at: <https://www.geeksforgeeks.org/importance-of-ui-ux-design-in-an-app-development/>
[Accessed 19 Jan 2025].